



Alma Mater Studiorum – Università di Bologna

Corso di Laurea Magistrale in Ingegneria Meccanica

LABORATORIO DI MECCANICA DELLE VIBRAZIONI E
SPERIMENTAZIONE SUI SISTEMI MECCANICI M

Sviluppo e verifica di un codice MATLAB per analisi modale agli elementi finiti mediante elementi Hexa20

Docente: Prof. Giuseppe Catania

Studente: Federico Nanni

Anno Accademico 2025/2026

Sommario

Nel presente lavoro viene sviluppato un codice MATLAB per l'analisi dinamica e vibratoria di strutture tridimensionali mediante il metodo degli elementi finiti. Il modello numerico è basato sull'impiego di elementi solidi isoparametrici quadratici *serendipity* a 20 nodi, indicati come *Hexa20*. Le matrici elementari di rigidezza e massa vengono calcolate mediante integrazione numerica di Gauss e successivamente assemblate nelle corrispondenti matrici globali del sistema. Il problema agli autovalori generalizzato viene risolto per determinare le frequenze naturali e le forme modali della struttura. La risposta dinamica forzata smorzata e non viene calcolata secondo due approcci: il metodo diretto, che integra numericamente le equazioni del moto nel dominio del tempo, e il metodo modale, che sfrutta la sovrapposizione dei modi propri per disaccoppiare il sistema. La correttezza del codice viene valutata attraverso una serie di verifiche numeriche — conservazione della massa, semidefinita positività della matrice di rigidezza, invarianza delle frequenze proprie per rotazione del modello e verifica tramite funzione di risposta in frequenza — e mediante il confronto con la soluzione analitica di Donnell per gusci cilindrici sottili.

Indice

1	Introduzione	4
2	Modellazione agli elementi finiti	6
2.1	Definizione del problema elastico	6
2.2	Approccio di Ritz	9
2.3	Approccio Lagrangiano	11
2.4	Approccio isoparametrico	12
2.5	Jacobiano della trasformazione	13
2.6	Integrazione delle matrici elementari	14
2.7	Integrazione numerica di Gauss	15
2.8	Modellazione di un vincolo elastico superficiale	16
3	Elemento solido isoparametrico Hexa20	18
3.1	Descrizione dell'elemento	18
3.2	Shape functions	19
4	Implementazione dello Script MATLAB	21
4.1	Leggi modello	21
4.2	Plotta modello	22
4.3	Plotta modi propri	23
4.4	Calcola risposta forzata	23
4.4.1	Metodo diretto (modello strutturale)	23
4.4.2	Metodo per sovrapposizione modale	24
4.5	Ruota modello	25
4.6	Crea Shell	25
4.7	Verifica analitica Shell	28
4.8	Plotta risposta forzata con chirp	30
5	Risultati numerici e verifiche	32
5.1	Verifica della matrice di massa: conservazione della massa totale	32
5.2	Verifica della matrice di rigidezza: semidefinita positività	34
5.3	Verifica per rotazione del modello: invarianza delle frequenze proprie	35
5.4	Confronto con la teoria analitica di Donnell	37
5.5	Verifica della risposta forzata tramite FRF chirp	41
6	Conclusioni	42
A	Codice MATLAB completo	45

Nomenclatura

Simbolo	Descrizione
$\mathbf{K}$	matrice di rigidezza globale
$\mathbf{M}$	matrice di massa globale
$\mathbf{K}_e$	matrice di rigidezza elementare
$\mathbf{M}_e$	matrice di massa elementare
$\mathbf{u}$	vettore degli spostamenti
$\boldsymbol{\varepsilon}$	vettore delle deformazioni
$\boldsymbol{\sigma}$	vettore degli sforzi
$\mathbf{D}$	matrice costitutiva elastica
$\mathbf{B}$	matrice deformazione-spostamento
$\mathbf{N}$	matrice delle funzioni di forma
ξ, η, ζ	coordinate naturali dell'elemento
ρ	densità del materiale
E	modulo di Young
ν	coefficiente di Poisson
ω	pulsazione naturale
f	frequenza naturale

1 Introduzione

Il metodo degli elementi finiti rappresenta uno degli strumenti numerici più utilizzati nell'ambito della progettazione e dell'analisi strutturale. Esso consente di studiare il comportamento statico e dinamico di componenti meccanici caratterizzati da geometrie complesse, materiali differenti e condizioni al contorno non sempre riconducibili a soluzioni analitiche semplici. Il dominio continuo viene suddiviso in un numero finito di elementi, all'interno dei quali il campo di spostamento è approssimato mediante opportune funzioni di interpolazione. In questo modo è possibile ricondurre il problema continuo originario a un sistema discreto di equazioni algebriche.

Nell'ambito della dinamica strutturale, il metodo degli elementi finiti permette di valutare le frequenze naturali e le corrispondenti forme modali di una struttura, fornendo informazioni fondamentali per l'identificazione di possibili fenomeni di risonanza. Inoltre, una volta costruite le matrici globali del sistema, è possibile studiare la risposta dinamica della struttura soggetta a forzanti esterne variabili nel tempo.

Nel presente lavoro viene sviluppato un codice MATLAB per l'analisi dinamica di strutture tridimensionali discretizzate mediante elementi solidi isoparametrici quadratici a 20 nodi *Serendipity*, indicati nel seguito come elementi *Hexa20*. Il codice è stato implementato con l'obiettivo di costruire in modo autonomo la matrice globale di rigidezza, la matrice globale di massa e il vettore globale delle forze nodali equivalenti, a partire dalla definizione della geometria, della connettività degli elementi, delle proprietà meccaniche del materiale, dei carichi distribuiti e dei vincoli elastici applicati al modello.

L'approccio seguito prevede la formulazione dell'elemento nel sistema di coordinate naturali, la definizione delle funzioni di forma quadratiche, il calcolo della matrice Jacobiana e la costruzione della matrice deformazione-spostamento. Le matrici elementari di rigidezza e massa vengono valutate mediante integrazione numerica di Gauss nel volume dell'elemento.

Il modello consente inoltre l'introduzione di carichi distribuiti per unità di superficie e di vincoli elastici superficiali. I carichi superficiali vengono trasformati nei corrispondenti vettori delle forze nodali equivalenti, mentre i vincoli elastici forniscono un contributo aggiuntivo alla matrice di rigidezza elementare. Entrambi i contributi vengono valutati mediante integrazione numerica sulla superficie dell'elemento interessata.

Le matrici e i vettori elementari così ottenuti vengono successivamente assemblati nelle corrispondenti quantità globali del sistema. A partire dalle matrici globali di rigidezza e massa viene risolto il problema agli autovalori generalizzato associato all'analisi modale libera non smorzata. Le frequenze naturali e le forme modali così ottenute rappresentano inoltre la base per la successiva valutazione della risposta dinamica mediante approccio modale.

Il codice consente anche il calcolo della risposta dinamica forzata secondo due differenti formulazioni. Nel primo caso, indicato come modello strutturale, l'equazione del moto viene integrata direttamente nel dominio fisico, mantenendo come incognite gli spostamenti nodali del sistema. Nel secondo caso, indicato come modello modale, la risposta viene approssimata mediante una combinazione lineare di modi propri, riducendo il numero di gradi di libertà del problema e consentendo una valutazione più efficiente della risposta nel tempo.

Particolare attenzione è stata dedicata alla struttura del codice e alla fase di post-processing. Il programma consente infatti di leggere un modello da file, costruire le matrici del sistema, calcolare le frequenze naturali, valutare la risposta dinamica forzata

e visualizzare sia la mesh indeformata sia le forme modali della struttura. Tali strumenti grafici risultano utili non solo per l'interpretazione dei risultati, ma anche per il controllo della correttezza della discretizzazione e della connettività degli elementi.

La verifica del codice viene condotta attraverso una serie di test numerici indipendenti. In primo luogo, la correttezza della matrice di massa viene accertata verificando che l'energia cinetica associata a una traslazione rigida unitaria sia uguale alla massa totale della struttura. In secondo luogo, la matrice di rigidezza viene testata confermandone la semidefinita positività: i modi di corpo rigido producono energia di deformazione nulla a livello di precisione macchina, mentre vettori di spostamento generici producono energia strettamente positiva. In terzo luogo, si verifica che le frequenze proprie siano invarianti per rotazione rigida del modello, confermando che l'assemblaggio delle matrici globali è indipendente dall'orientamento della struttura nello spazio. Le frequenze naturali ottenute numericamente vengono inoltre confrontate con valori ricavati dalla teoria analitica di Donnell per gusci cilindrici, al fine di validare esternamente il modello FEM. Infine, la consistenza interna del solver viene verificata tramite la funzione di risposta in frequenza calcolata a partire da una forzante chirp: la presenza di picchi della FRF in corrispondenza di ciascuna frequenza naturale conferma la coerenza tra la soluzione agli autovalori e l'integrazione temporale delle equazioni del moto.

Il lavoro è organizzato come segue. Nella Sezione 2 vengono richiamati i principali aspetti teorici del metodo degli elementi finiti applicato all'elasticità tridimensionale, inclusa la modellazione dei carichi superficiali e dei vincoli elastici. Nella Sezione 3 viene descritto l'elemento *Hexa20*, con particolare riferimento alle coordinate naturali, alle funzioni di forma e alla costruzione della matrice Jacobiana. Nella Sezione 4 viene descritta l'implementazione MATLAB del codice, compresi i due approcci per il calcolo della risposta dinamica forzata e il generatore di mesh per gusci cilindrici. Nella Sezione 5 vengono presentati i risultati numerici e le verifiche condotte, tra cui il confronto con la teoria analitica di Donnell e la verifica tramite FRF chirp. Nella Sezione 6 vengono infine discusse le conclusioni e i possibili sviluppi futuri.

2 Modellazione agli elementi finiti

Il metodo degli elementi finiti è riconosciuto come uno degli strumenti numerici più diffusi ed efficaci per l'analisi di problemi del continuo. Esso può essere applicato a numerosi ambiti dell'ingegneria, tra cui la meccanica dei solidi, la trasmissione del calore e la fluidodinamica. Tuttavia, uno dei suoi impieghi principali è rappresentato dall'analisi strutturale di componenti meccanici.

Il caso di interesse del presente lavoro riguarda l'utilizzo di questo metodo per la risoluzione di sistemi dinamici che esibiscono un comportamento elastico lineare, caratterizzato da piccoli spostamenti e piccole deformazioni. In particolare, l'attenzione è rivolta allo studio delle vibrazioni libere e forzate di strutture tridimensionali, con l'obiettivo di determinarne le frequenze naturali, le forme modali e la risposta dinamica nel tempo.

Nel seguito vengono quindi richiamate le relazioni fondamentali dell'elasticità lineare tridimensionale, a partire dalla descrizione del campo di spostamento, delle deformazioni e degli sforzi, fino alla formulazione discreta delle equazioni del moto.

2.1 Definizione del problema elastico

Campo di spostamento

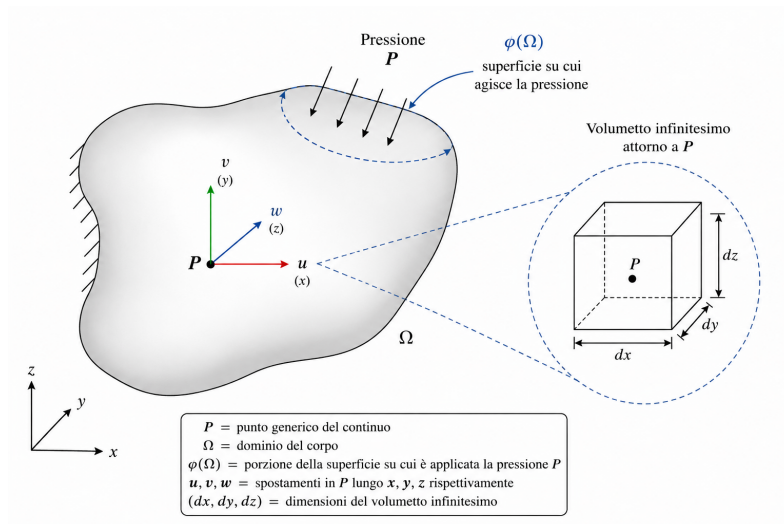


Figura 2.1: Il problema elastico.

Si consideri un corpo continuo che occupa il dominio Ω delimitato dalla superficie $\phi(\Omega)$. Per un generico punto $P \in \Omega$ possiamo definire il campo di spostamento come

$$\mathbf{u}(x, y, z) = \begin{bmatrix} u \\ v \\ w \end{bmatrix} \quad (2.1)$$

dove u , v e w rappresentano gli spostamenti lungo gli assi cartesiani.

Campo di tensione

Sempre per lo stesso punto possiamo definire anche lo stato tensionale, descritto dal tensore delle tensioni:

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{xz} \\ \sigma_{yx} & \sigma_{yy} & \sigma_{yz} \\ \sigma_{zx} & \sigma_{zy} & \sigma_{zz} \end{bmatrix} \quad (2.2)$$

Poiché il tensore delle tensioni è simmetrico, vale

$$\sigma_{ij} = \sigma_{ji} \quad (2.3)$$

e posso scrivermi il tensore delle tensioni come un vettore:

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{zz} \\ \sigma_{xy} \\ \sigma_{yz} \\ \sigma_{xz} \end{bmatrix} \quad (2.4)$$

Campo di deformazione

Le componenti del tensore delle deformazioni sono legate al campo di spostamento mediante le relazioni

$$\varepsilon_{xx} = \frac{\partial u}{\partial x} \quad (2.5)$$

$$\varepsilon_{yy} = \frac{\partial v}{\partial y} \quad (2.6)$$

$$\varepsilon_{zz} = \frac{\partial w}{\partial z} \quad (2.7)$$

$$\gamma_{xy} = \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \quad (2.8)$$

$$\gamma_{yz} = \frac{\partial v}{\partial z} + \frac{\partial w}{\partial y} \quad (2.9)$$

$$\gamma_{xz} = \frac{\partial u}{\partial z} + \frac{\partial w}{\partial x} \quad (2.10)$$

le relazioni deformazione-spostamento possono essere raccolte nella forma compatta

$$\boldsymbol{\varepsilon} = \mathbf{L}\mathbf{u} \quad (2.11)$$

dove l'operatore differenziale $\mathbf{L}$ è definito da

$$\mathbf{L} = \begin{bmatrix} \frac{\partial}{\partial x} & 0 & 0 \\ 0 & \frac{\partial}{\partial y} & 0 \\ 0 & 0 & \frac{\partial}{\partial z} \\ \frac{\partial}{\partial y} & \frac{\partial}{\partial x} & 0 \\ 0 & \frac{\partial}{\partial z} & \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} & 0 & \frac{\partial}{\partial x} \end{bmatrix} \quad (2.12)$$

Equilibrio sul solido infinitesimo

L'equilibrio infinitesimo di un volume elementare, dovuto a tutti gli sforzi agente su di esso, da fuori a queste equazioni:

$$f_x + \frac{\partial \sigma_x}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} + \frac{\partial \sigma_{xz}}{\partial z} = 0 \quad (2.13)$$

$$f_y + \frac{\partial \sigma_y}{\partial y} + \frac{\partial \sigma_{xy}}{\partial x} + \frac{\partial \sigma_{yz}}{\partial z} = 0 \quad (2.14)$$

$$f_z + \frac{\partial \sigma_z}{\partial z} + \frac{\partial \sigma_{xz}}{\partial x} + \frac{\partial \sigma_{yz}}{\partial y} = 0 \quad (2.15)$$

facendo le opportune semplificazioni, in forma matriciale, le equazioni di equilibrio del solido infinitesimo possono essere scritte come

$$\begin{bmatrix} \frac{\partial}{\partial x} & 0 & 0 & \frac{\partial}{\partial y} & 0 & \frac{\partial}{\partial z} \\ 0 & \frac{\partial}{\partial y} & 0 & \frac{\partial}{\partial x} & \frac{\partial}{\partial z} & 0 \\ 0 & 0 & \frac{\partial}{\partial z} & 0 & \frac{\partial}{\partial y} & \frac{\partial}{\partial x} \end{bmatrix} \begin{bmatrix} \sigma_x \\ \sigma_y \\ \sigma_z \\ \sigma_{xy} \\ \sigma_{yz} \\ \sigma_{zx} \end{bmatrix} + \begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix} = \mathbf{0} \quad (2.16)$$

e quindi usando l'operatore $\mathbf{L}$ definito nelle relazioni di spostamento-deformazione:

$$\mathbf{L}^T \boldsymbol{\sigma} + \mathbf{f} = \mathbf{0} \quad (2.17)$$

Legge costitutiva del materiale

Nel caso di un materiale isotropo, conservativo e lineare, possiamo poi andare a costruirci la legge costitutiva del materiale che lega le deformazioni alle tensioni:

$$\boldsymbol{\sigma} = \mathbf{D} \boldsymbol{\varepsilon} \quad (2.18)$$

dove la matrice costitutiva $\mathbf{D}$ vale:

$$\mathbf{D} = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1-\nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1-\nu & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix} \quad (2.19)$$

Problema Elastico

Combinando le equazioni (2.11), (2.17) ed (2.18), il problema elastico lineare può essere espresso mediante il sistema:

$$\begin{cases} \boldsymbol{\varepsilon} = \mathbf{L}\mathbf{u} & (6 \text{ equazioni}) \\ \mathbf{L}^T \boldsymbol{\sigma} + \mathbf{f} = \mathbf{0} & (3 \text{ equazioni}) \\ \boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\varepsilon} & (6 \text{ equazioni}) \end{cases} \quad (2.20)$$

Le precedenti relazioni costituiscono il problema elastico lineare espresso in termini delle tre incognite fondamentali: campo di spostamento $\mathbf{u}$, campo di deformazione $\boldsymbol{\varepsilon}$ e campo di tensione $\boldsymbol{\sigma}$.

2.2 Approccio di Ritz

Il problema elastico lineare può essere scritto, in forma compatta, come

$$\mathbf{L}^T \mathbf{D} \mathbf{L} \mathbf{u} + \mathbf{f} = \mathbf{0} \quad (2.21)$$

a cui devono essere associate le opportune condizioni al contorno. In generale, la risoluzione diretta di tale problema differenziale può risultare complessa; per questo motivo è conveniente introdurre una formulazione energetica.

L'energia potenziale totale del sistema può essere scritta come

$$\Pi(\mathbf{u}) = U(\mathbf{u}) + W(\mathbf{u}) + W_{in}(\mathbf{u}) \quad (2.22)$$

dove U rappresenta l'energia elastica interna, W il lavoro delle forze esterne e W_{in} il contributo associato alle forze inerziali.

L'energia elastica interna è pari a

$$U = \frac{1}{2} \int_{\Omega} \boldsymbol{\sigma}^T \boldsymbol{\varepsilon} d\Omega \quad (2.23)$$

Sostituendo le relazioni

$$\boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} = \mathbf{L}\mathbf{u}$$

si ottiene

$$U = \frac{1}{2} \int_{\Omega} (\mathbf{L}\mathbf{u})^T \mathbf{D} (\mathbf{L}\mathbf{u}) d\Omega \quad (2.24)$$

Il lavoro delle forze esterne risulta invece

$$W = - \int_{\Omega} \mathbf{u}^T \mathbf{f} d\Omega - \int_{\partial\Omega_p} \mathbf{u}^T \mathbf{p} dS \quad (2.25)$$

mentre il contributo inerziale può essere scritto come

$$W_{in} = \int_{\Omega} \rho \mathbf{u}^T \ddot{\mathbf{u}} d\Omega \quad (2.26)$$

La funzionale da minimizzare diventa quindi

$$\Pi(\mathbf{u}) = \frac{1}{2} \int_{\Omega} (\mathbf{L}\mathbf{u})^T \mathbf{D} (\mathbf{L}\mathbf{u}) d\Omega + \int_{\Omega} \rho \mathbf{u}^T \ddot{\mathbf{u}} d\Omega - \int_{\Omega} \mathbf{u}^T \mathbf{f} d\Omega - \int_{\partial\Omega_p} \mathbf{u}^T \mathbf{p} dS \quad (2.27)$$

L'approccio di Ritz consiste nel ricercare una soluzione approssimata del campo di spostamento come combinazione lineare di funzioni assegnate nello spazio e coefficienti incogniti dipendenti dal tempo:

$$\mathbf{u}(x, y, z, t) \simeq \sum_{i=1}^N \phi_i(x, y, z) \delta_i(t) \quad (2.28)$$

In forma matriciale:

$$\mathbf{u}(x, y, z, t) \simeq \mathbf{\Phi}(x, y, z) \boldsymbol{\delta}(t) \quad (2.29)$$

dove

$$\mathbf{\Phi} = [\phi_1 \quad \phi_2 \quad \cdots \quad \phi_N] \quad (2.30)$$

e

$$\boldsymbol{\delta} = [\delta_1 \quad \delta_2 \quad \cdots \quad \delta_N]^T \quad (2.31)$$

Sostituendo l'approssimazione di Ritz nella funzionale energetica e imponendo la condizione di stazionarietà

$$\frac{\partial \Pi}{\partial \boldsymbol{\delta}} = \mathbf{0} \quad (2.32)$$

si ottiene il sistema discreto del moto

$$\mathbf{M} \ddot{\boldsymbol{\delta}} + \mathbf{K} \boldsymbol{\delta} = \mathbf{F} \quad (2.33)$$

dove

$$\mathbf{K} = \int_{\Omega} (\mathbf{L}\mathbf{\Phi})^T \mathbf{D} (\mathbf{L}\mathbf{\Phi}) d\Omega \quad (2.34)$$

$$\mathbf{M} = \int_{\Omega} \rho \mathbf{\Phi}^T \mathbf{\Phi} d\Omega \quad (2.35)$$

$$\mathbf{F} = \int_{\Omega} \mathbf{\Phi}^T \mathbf{f} d\Omega + \int_{\partial\Omega_p} \mathbf{\Phi}^T \mathbf{p} dS \quad (2.36)$$

Su domini curvi l'integrazione di queste matrici può risultare assai complicato, motivo per il quale bisogna cercare un approccio tale per cui si riesca a semplificare queste procedure

2.3 Approccio Lagrangiano

Le matrici di rigidezza, massa e il vettore delle forze richiedono l'integrazione su domini che, in generale, possono avere una geometria arbitraria. L'integrazione diretta su tali domini può risultare particolarmente complessa.

L'idea alla base dell'approccio lagrangiano consiste nell'introdurre una mappa che associ ad ogni punto del dominio fisico un punto appartenente ad un dominio parametrico di forma semplice.

Nel caso monodimensionale si consideri una curva arbitraria immersa nello spazio tridimensionale:

$$\Omega_e \subset \mathbb{R}^3$$

e si introduca una coordinata parametrica $\xi \in [0, 1]$.

Si definisce quindi una mappa

$$P = \phi(\xi) \quad (2.37)$$

che associa ad ogni valore della coordinata parametrica un punto della curva fisica. Esistono numerose possibili scelte per la funzione di mappatura ϕ . Una delle più utilizzate nel metodo degli elementi finiti è l'interpolazione polinomiale di Lagrange.

Si scelgono $n + 1$ punti della curva, detti nodi geometrici,

$$Q_0, Q_1, \dots, Q_n$$

ai quali vengono associati i valori parametrici

$$\xi_0, \xi_1, \dots, \xi_n.$$

La mappa viene quindi approssimata mediante

$$P(\xi) = \sum_{i=0}^n L_i(\xi) Q_i \quad (2.38)$$

dove $L_i(\xi)$ sono i polinomi di Lagrange. I polinomi di Lagrange sono definiti come

$$L_i(\xi) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{\xi - \xi_j}{\xi_i - \xi_j} \quad (2.39)$$

La caratteristica più importante di tali funzioni è la proprietà interpolante

$$L_i(\xi_j) = \delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \quad (2.40)$$

Grazie alla proprietà precedente, la curva ottenuta mediante la mappa (2.38) passa esattamente per tutti i nodi assegnati.

Infatti, valutando la mappa in corrispondenza del nodo ξ_k , si ottiene

$$P(\xi_k) = \sum_{i=0}^n L_i(\xi_k) Q_i = Q_k \quad (2.41)$$

poiché tutti i termini della sommatoria risultano nulli ad eccezione di quello corrispondente all'indice k .

Ne consegue che la curva approssimata interpola esattamente tutti i punti geometrici assegnati.

Nel caso tridimensionale si introducono tre coordinate parametriche (ξ, η, ζ) e la geometria del solido viene rappresentata mediante

$$P(\xi, \eta, \zeta) = \sum_{i=0}^{n_\xi} \sum_{j=0}^{n_\eta} \sum_{k=0}^{n_\zeta} L_i(\xi) L_j(\eta) L_k(\zeta) Q_{ijk}. \quad (2.42)$$

Grazie all'utilizzo di questa formulazione non abbiamo dei vincoli sulla forma del solido e possiamo modellare anche delle geometrie complesse.

2.4 Approccio isoparametrico

L'approccio lagrangiano permette di descrivere la geometria dell'elemento fisico tramite una mappa costruita a partire da funzioni di forma. In generale, però, si devono ancora scegliere separatamente le funzioni utilizzate per descrivere la geometria e quelle utilizzate per approssimare il campo di spostamento.

Nell'approccio isoparametrico si assume che la stessa famiglia di funzioni di forma venga utilizzata sia per interpolare la geometria dell'elemento sia per interpolare il campo di spostamento.

Indicando con $N_i(\xi, \eta, \zeta)$ le funzioni di forma definite nel dominio naturale, la geometria dell'elemento viene descritta come

$$\mathbf{x}(\xi, \eta, \zeta) = \sum_{i=1}^n N_i(\xi, \eta, \zeta) \mathbf{x}_i \quad (2.43)$$

dove

$$\mathbf{x}_i = \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix} \quad (2.44)$$

sono le coordinate del nodo i -esimo dell'elemento fisico.

Il campo di spostamento viene interpolato mediante le stesse funzioni di forma:

$$\mathbf{u}(\xi, \eta, \zeta, t) = \sum_{i=1}^n N_i(\xi, \eta, \zeta) \mathbf{u}_i(t) \quad (2.45)$$

dove

$$\mathbf{u}_i(t) = \begin{bmatrix} u_i(t) \\ v_i(t) \\ w_i(t) \end{bmatrix} \quad (2.46)$$

In forma matriciale si può scrivere

$$\mathbf{u}(\xi, \eta, \zeta, t) = \mathbf{N}_s(\xi, \eta, \zeta) \boldsymbol{\delta}(t) \quad (2.47)$$

con

$$\boldsymbol{\delta}(t) = [u_1 \quad v_1 \quad w_1 \quad u_2 \quad v_2 \quad w_2 \quad \cdots \quad u_n \quad v_n \quad w_n]^T \quad (2.48)$$

$$\mathbf{N}_s = \begin{bmatrix} N_1 & 0 & 0 & N_2 & 0 & 0 & \cdots & N_n & 0 & 0 \\ 0 & N_1 & 0 & 0 & N_2 & 0 & \cdots & 0 & N_n & 0 \\ 0 & 0 & N_1 & 0 & 0 & N_2 & \cdots & 0 & 0 & N_n \end{bmatrix} \quad (2.49)$$

Questa scelta prende il nome di formulazione isoparametrica poiché la geometria e il campo di spostamento sono descritti mediante lo stesso insieme di parametri e le stesse funzioni di forma.

In particolare, ponendo

$$\mathbf{N}_s \equiv \boldsymbol{\phi} \quad (2.50)$$

si ottiene una descrizione coerente tra geometria e cinematica dell'elemento.

Il principale vantaggio dell'approccio isoparametrico è che la continuità geometrica tra elementi adiacenti si traduce anche in continuità del campo di spostamento. Infatti, se due elementi condividono uno stesso lato, una stessa superficie o gli stessi nodi, l'utilizzo delle medesime funzioni di forma garantisce che la deformata lungo il bordo comune sia descritta in modo compatibile da entrambi gli elementi.

Questo permette di evitare discontinuità spurie tra elementi adiacenti e rende la formulazione particolarmente adatta alla costruzione automatica delle matrici elementari nel metodo agli elementi finiti.

2.5 Jacobiano della trasformazione

Nell'approccio isoparametrico il dominio fisico dell'elemento viene descritto a partire dal dominio naturale mediante la mappa

$$\mathbf{x} = \boldsymbol{\phi}(\xi, \eta, \zeta) \quad (2.51)$$

ovvero

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x(\xi, \eta, \zeta) \\ y(\xi, \eta, \zeta) \\ z(\xi, \eta, \zeta) \end{bmatrix}. \quad (2.52)$$

Il Jacobiano della trasformazione è la matrice che lega le variazioni infinitesime nelle coordinate naturali (ξ, η, ζ) alle variazioni infinitesime nelle coordinate fisiche (x, y, z) :

$$\mathbf{J} = \frac{\partial(x, y, z)}{\partial(\xi, \eta, \zeta)} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} & \frac{\partial z}{\partial \eta} \\ \frac{\partial x}{\partial \zeta} & \frac{\partial y}{\partial \zeta} & \frac{\partial z}{\partial \zeta} \end{bmatrix}. \quad (2.53)$$

Il determinante del Jacobiano consente di trasformare il differenziale di volume dal dominio naturale al dominio fisico:

$$d\Omega = \det(\mathbf{J}) d\xi d\eta d\zeta. \quad (2.54)$$

Pertanto, un integrale sul dominio fisico può essere riscritto come

$$\int_{\Omega_e} (\cdot) d\Omega = \int_{\hat{\Omega}_e} (\cdot) \det(\mathbf{J}) d\xi d\eta d\zeta. \quad (2.55)$$

Inoltre, poiché le funzioni di forma sono definite rispetto alle coordinate naturali, è necessario trasformare le derivate rispetto a x, y, z . Applicando la regola della catena si ottiene

$$\begin{bmatrix} \frac{\partial}{\partial \xi} \\ \frac{\partial}{\partial \eta} \\ \frac{\partial}{\partial \zeta} \end{bmatrix} = \mathbf{J} \begin{bmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{bmatrix}. \quad (2.56)$$

Di conseguenza,

$$\begin{bmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{bmatrix} = \mathbf{J}^{-1} \begin{bmatrix} \frac{\partial}{\partial \xi} \\ \frac{\partial}{\partial \eta} \\ \frac{\partial}{\partial \zeta} \end{bmatrix}. \quad (2.57)$$

Questo passaggio è fondamentale perché la matrice deformazione-spostamento $\mathbf{B}$ richiede le derivate delle funzioni di forma rispetto alle coordinate fisiche x, y, z , mentre le funzioni di forma vengono costruite nel sistema naturale ξ, η, ζ .

2.6 Integrazione delle matrici elementari

Le matrici elementari di rigidezza e di massa sono definite nel dominio fisico dell'elemento Ω_e come

$$\mathbf{K}_e = \int_{\Omega_e} \mathbf{B}^T \mathbf{D} \mathbf{B} d\Omega \quad (2.58)$$

$$\mathbf{M}_e = \int_{\Omega_e} \rho \mathbf{N}_s^T \mathbf{N}_s d\Omega \quad (2.59)$$

Il vettore delle forze elementari può essere scritto come

$$\mathbf{F}_e = \int_{\Omega_e} \mathbf{N}_s^T \mathbf{f} d\Omega + \int_{\partial\Omega_{p,e}} \mathbf{N}_s^T \mathbf{p} dS. \quad (2.60)$$

Attraverso la mappa isoparametrica, il dominio fisico Ω_e viene trasformato nel dominio naturale $\hat{\Omega}_e$. Utilizzando il determinante del Jacobiano, vale

$$d\Omega = \det(\mathbf{J}) d\xi d\eta d\zeta. \quad (2.61)$$

Pertanto, la matrice di rigidezza diventa

$$\mathbf{K}_e = \int_{\hat{\Omega}_e} \mathbf{B}^T \mathbf{D} \mathbf{B} \det(\mathbf{J}) d\xi d\eta d\zeta. \quad (2.62)$$

Analogamente, la matrice di massa diventa

$$\mathbf{M}_e = \int_{\hat{\Omega}_e} \rho \mathbf{N}_s^T \mathbf{N}_s \det(\mathbf{J}) d\xi d\eta d\zeta. \quad (2.63)$$

e il vettore delle forze di volume diventa

$$\mathbf{F}_{e,\Omega} = \int_{\hat{\Omega}_e} \mathbf{N}_s^T \mathbf{f} \det(\mathbf{J}) d\xi d\eta d\zeta. \quad (2.64)$$

Nel caso di elementi tridimensionali con dominio naturale definito in

$$-1 \leq \xi \leq 1, \quad -1 \leq \eta \leq 1, \quad -1 \leq \zeta \leq 1,$$

gli integrali assumono la forma

$$\mathbf{K}_e = \int_{-1}^1 \int_{-1}^1 \int_{-1}^1 \mathbf{B}^T \mathbf{D} \mathbf{B} \det(\mathbf{J}) d\xi d\eta d\zeta. \quad (2.65)$$

$$\mathbf{M}_e = \int_{-1}^1 \int_{-1}^1 \int_{-1}^1 \rho \mathbf{N}_s^T \mathbf{N}_s \det(\mathbf{J}) d\xi d\eta d\zeta. \quad (2.66)$$

2.7 Integrazione numerica di Gauss

Gli integrali precedenti sono generalmente difficili da calcolare in forma analitica, soprattutto nel caso di elementi curvi o di ordine elevato. Per questo motivo si ricorre all'integrazione numerica di Gauss.

Nel caso monodimensionale, un integrale nell'intervallo $[-1, 1]$ viene approssimato come

$$\int_{-1}^1 f(\xi) d\xi \simeq \sum_{i=1}^{n_g} w_i f(\xi_i) \quad (2.67)$$

dove ξ_i sono i punti di Gauss, w_i i relativi pesi e n_g il numero di punti di integrazione. Nel caso tridimensionale si applica il prodotto tensoriale della formula monodimensionale:

$$\int_{-1}^1 \int_{-1}^1 \int_{-1}^1 f(\xi, \eta, \zeta) d\xi d\eta d\zeta \simeq \sum_{i=1}^{n_g} \sum_{j=1}^{n_g} \sum_{k=1}^{n_g} w_i w_j w_k f(\xi_i, \eta_j, \zeta_k). \quad (2.68)$$

Applicando la quadratura di Gauss alla matrice di rigidezza elementare si ottiene

$$\mathbf{K}_e \simeq \sum_{i=1}^{n_g} \sum_{j=1}^{n_g} \sum_{k=1}^{n_g} w_i w_j w_k \mathbf{B}^T(\xi_i, \eta_j, \zeta_k) \mathbf{D} \mathbf{B}(\xi_i, \eta_j, \zeta_k) \det[\mathbf{J}(\xi_i, \eta_j, \zeta_k)]. \quad (2.69)$$

Analogamente, per la matrice di massa:

$$\mathbf{M}_e \simeq \sum_{i=1}^{n_g} \sum_{j=1}^{n_g} \sum_{k=1}^{n_g} w_i w_j w_k \rho \mathbf{N}_s^T(\xi_i, \eta_j, \zeta_k) \mathbf{N}_s(\xi_i, \eta_j, \zeta_k) \det[\mathbf{J}(\xi_i, \eta_j, \zeta_k)]. \quad (2.70)$$

Infine, il vettore delle forze di volume può essere approssimato come

$$\mathbf{F}_{e,\Omega} \simeq \sum_{i=1}^{n_g} \sum_{j=1}^{n_g} \sum_{k=1}^{n_g} w_i w_j w_k \mathbf{N}_s^T(\xi_i, \eta_j, \zeta_k) \mathbf{f}(\xi_i, \eta_j, \zeta_k) \det [\mathbf{J}(\xi_i, \eta_j, \zeta_k)]. \quad (2.71)$$

In questo modo il calcolo degli integrali elementari viene ricondotto alla valutazione delle funzioni di forma, della matrice $\mathbf{B}$, della matrice costitutiva $\mathbf{D}$ e del determinante del Jacobiano in un numero finito di punti di integrazione, questi integrali solitamente arrivano a convergenza dopo pochi termini e in letteratura è noto il numero di punti di gauss da utilizzare per ogni tipologia di elemento.

Integrazione delle forze superficiali

Nel caso di carichi distribuiti applicati su una superficie dell'elemento, il contributo al vettore delle forze elementari è dato da

$$\mathbf{F}_{e,S} = \int_{S_e} \mathbf{N}_s^T \mathbf{p} dS \quad (2.72)$$

dove $\mathbf{p}$ è il vettore delle pressioni o trazioni superficiali applicate.

La superficie fisica viene descritta mediante una mappa dipendente da due coordinate naturali. Ad esempio, se la faccia è identificata da $\zeta = \text{costante}$, si ha

$$\mathbf{x} = \mathbf{x}(\xi, \eta) \quad (2.73)$$

Il differenziale di superficie non viene trasformato tramite $\det(\mathbf{J})$, ma tramite il Jacobiano superficiale:

$$dS = J_S d\xi d\eta \quad (2.74)$$

con

$$J_S = \left\| \frac{\partial \mathbf{x}}{\partial \xi} \times \frac{\partial \mathbf{x}}{\partial \eta} \right\| \quad (2.75)$$

$$\mathbf{F}_{e,S} = \int_{-1}^1 \int_{-1}^1 \mathbf{N}_s^T(\xi, \eta) \mathbf{p}(\xi, \eta) J_S(\xi, \eta) d\xi d\eta \quad (2.76)$$

$$\mathbf{F}_{e,S} \simeq \sum_{i=1}^{n_g} \sum_{j=1}^{n_g} w_i w_j \mathbf{N}_s^T(\xi_i, \eta_j) \mathbf{p}(\xi_i, \eta_j) J_S(\xi_i, \eta_j) \quad (2.77)$$

Il Jacobiano superficiale rappresenta quindi il fattore di scala che trasforma l'area infinitesima del dominio naturale nella corrispondente area infinitesima della superficie fisica. A differenza degli integrali di volume, in cui compare $\det(\mathbf{J})$, negli integrali superficiali compare la norma del prodotto vettoriale tra i due vettori tangenti alla superficie.

2.8 Modellazione di un vincolo elastico superficiale

Un vincolo elastico distribuito su una superficie può essere modellato introducendo un insieme di molle distribuite sull'area vincolata. Tali molle generano una reazione proporzionale allo spostamento locale della superficie.

Si consideri una superficie vincolata Ω_v appartenente al bordo del dominio. Il campo di spostamento sulla superficie è interpolato tramite le funzioni di forma dell'elemento:

$$\mathbf{u} = \mathbf{N}_s \boldsymbol{\delta} \quad (2.78)$$

La forza elastica distribuita per unità di superficie si oppone allo spostamento secondo la legge

$$\mathbf{p}_v = -\mathbf{C} \mathbf{u} \quad (2.79)$$

dove $\mathbf{C}$ è la matrice diagonale delle rigidezze del vincolo elastico distribuito nelle tre direzioni cartesiane:

$$\mathbf{C} = \begin{bmatrix} k_x & 0 & 0 \\ 0 & k_y & 0 \\ 0 & 0 & k_z \end{bmatrix} \quad (2.80)$$

L'energia potenziale elastica aggiuntiva associata al vincolo si ottiene integrando il lavoro delle forze di reazione sulla superficie Ω_v :

$$\Delta \Pi_v = \frac{1}{2} \int_{\Omega_v} \mathbf{u}^T \mathbf{C} \mathbf{u} dS \quad (2.81)$$

Sostituendo l'interpolazione (2.78) si ottiene la forma discreta:

$$\Delta \Pi_v = \frac{1}{2} \boldsymbol{\delta}^T \underbrace{\left[\int_{\Omega_v} \mathbf{N}_s^T \mathbf{C} \mathbf{N}_s dS \right]}_{\Delta \mathbf{K}_v} \boldsymbol{\delta} \quad (2.82)$$

La matrice di rigidezza del vincolo elastico superficiale è quindi:

$$\Delta \mathbf{K}_v = \int_{\Omega_v} \mathbf{N}_s^T \mathbf{C} \mathbf{N}_s dS \quad (2.83)$$

Tale contributo viene sommato alla matrice di rigidezza globale della struttura, modificando l'equazione del moto come segue:

$$\mathbf{M} \ddot{\boldsymbol{\delta}} + (\mathbf{K} + \Delta \mathbf{K}_v) \boldsymbol{\delta} = \mathbf{F} \quad (2.84)$$

Scegliendo k_x , k_y , k_z sufficientemente grandi rispetto alla rigidezza della struttura (metodo della penalità), il vincolo cinematico è approssimato con la precisione desiderata senza modificare la struttura del sistema di equazioni.

3 Elemento solido isoparametrico Hexa20

Nel presente lavoro viene utilizzato una tipologia di elemento *Serendipity*, che si differenzia dalla tipologia di elemento Lagrangiano presente in 2.3 dal momento che non presenta nodi nel mezzo delle facce, ma solamente sugli spigoli dell'elemento. In particolare l'elemento trattato in letteratura viene definito **Hexa20**, ovvero un brick tridimensionale che presenta 20 nodi.

3.1 Descrizione dell'elemento

L'elemento *Hexa20* è un elemento finito tridimensionale esaedrico costituito da 20 nodi: 8 nodi di vertice e 12 nodi intermedi, posti sui lati dell'elemento.

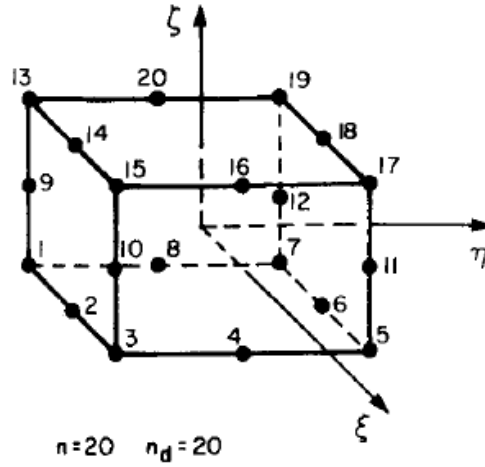


Figura 3.1: Elemento esaedrico quadratico a 20 nodi.

L'elemento è definito nel dominio naturale

$$-1 \leq \xi \leq 1, \quad -1 \leq \eta \leq 1, \quad -1 \leq \zeta \leq 1. \quad (3.1)$$

Il campo di spostamento viene interpolato mediante le funzioni di forma $N_i(\xi, \eta, \zeta)$:

$$\mathbf{u}(\xi, \eta, \zeta) = \sum_{i=1}^{20} N_i(\xi, \eta, \zeta) \mathbf{u}_i \quad (3.2)$$

dove

$$\mathbf{u}_i = \begin{bmatrix} u_i \\ v_i \\ w_i \end{bmatrix} \quad (3.3)$$

rappresenta il vettore degli spostamenti del nodo i -esimo. L'elemento viene detto quadratico perché lungo ciascun lato sono presenti tre nodi: due nodi estremi e un nodo intermedio. Questo permette di descrivere l'andamento dello spostamento lungo il lato mediante un polinomio di secondo grado.

Ad esempio, lungo un generico lato dell'elemento, il campo di spostamento può essere approssimato come

$$u(\xi) = a_0 + a_1\xi + a_2\xi^2. \quad (3.4)$$

La presenza dei nodi intermedi consente quindi all'elemento di rappresentare lati curvi e campi di spostamento quadratici lungo gli spigoli. L'elemento Hexa20 è tuttavia un elemento quadratico incompleto. Infatti, un elemento esaedrico quadratico completo nel dominio tridimensionale richiederebbe 27 nodi: 8 nodi di vertice, 12 nodi sugli spigoli, 6 nodi al centro delle facce e 1 nodo interno al volume.

L'elemento Hexa20, invece, possiede solamente i nodi di vertice e i nodi intermedi sugli spigoli. Mancano quindi i nodi al centro delle facce e il nodo interno.

Per questo motivo l'interpolazione non contiene tutti i termini quadratici completi associati ad un elemento *Lagrangiano* completo a 27 nodi. In particolare, l'Hexa20 è un elemento serendipity: mantiene un'interpolazione quadratica sugli spigoli e sulle facce, ma riduce il numero di gradi di libertà eliminando alcuni termini interni della base polinomiale.

3.2 Shape functions

Le funzioni di forma dell'elemento Hexa20 sono definite nel dominio naturale $(\xi, \eta, \zeta) \in [-1, 1]^3$. L'elemento possiede 8 nodi di vertice e 12 nodi intermedi sugli spigoli.

Per i nodi di vertice vale la forma generale

$$N_i = \frac{1}{8}(1 + \xi\xi_i)(1 + \eta\eta_i)(1 + \zeta\zeta_i)(-2 + \xi\xi_i + \eta\eta_i + \zeta\zeta_i) \quad (3.5)$$

$$\frac{\partial N_i}{\partial \xi} = \frac{1}{8}\xi_i(1 + \eta\eta_i)(1 + \zeta\zeta_i)(-1 + 2\xi\xi_i + \eta\eta_i + \zeta\zeta_i) \quad (3.6)$$

$$\frac{\partial N_i}{\partial \eta} = \frac{1}{8}\eta_i(1 + \xi\xi_i)(1 + \zeta\zeta_i)(-1 + \xi\xi_i + 2\eta\eta_i + \zeta\zeta_i) \quad (3.7)$$

$$\frac{\partial N_i}{\partial \zeta} = \frac{1}{8}\zeta_i(1 + \xi\xi_i)(1 + \eta\eta_i)(-1 + \xi\xi_i + \eta\eta_i + 2\zeta\zeta_i) \quad (3.8)$$

dove $\xi_i, \eta_i, \zeta_i = \pm 1$.

Per i nodi intermedi sugli spigoli paralleli a ξ , cioè per $\xi_i = 0$, si ha

$$N_i = \frac{1}{4}(1 - \xi^2)(1 + \eta\eta_i)(1 + \zeta\zeta_i) \quad (3.9)$$

$$\frac{\partial N_i}{\partial \xi} = -\frac{1}{2}\xi(1 + \eta\eta_i)(1 + \zeta\zeta_i) \quad (3.10)$$

$$\frac{\partial N_i}{\partial \eta} = \frac{1}{4}\eta_i(1 - \xi^2)(1 + \zeta\zeta_i) \quad (3.11)$$

$$\frac{\partial N_i}{\partial \zeta} = \frac{1}{4}\zeta_i(1 - \xi^2)(1 + \eta\eta_i) \quad (3.12)$$

Per i nodi intermedi sugli spigoli paralleli a η , cioè per $\eta_i = 0$, si ha

$$N_i = \frac{1}{4}(1 + \xi\xi_i)(1 - \eta^2)(1 + \zeta\zeta_i) \quad (3.13)$$

$$\frac{\partial N_i}{\partial \xi} = \frac{1}{4}\xi_i(1 - \eta^2)(1 + \zeta\zeta_i) \quad (3.14)$$

$$\frac{\partial N_i}{\partial \eta} = -\frac{1}{2}\eta(1 + \xi\xi_i)(1 + \zeta\zeta_i) \quad (3.15)$$

$$\frac{\partial N_i}{\partial \zeta} = \frac{1}{4}\zeta_i(1 + \xi\xi_i)(1 - \eta^2) \quad (3.16)$$

Per i nodi intermedi sugli spigoli paralleli a ζ , cioè per $\zeta_i = 0$, si ha

$$N_i = \frac{1}{4}(1 + \xi\xi_i)(1 + \eta\eta_i)(1 - \zeta^2) \quad (3.17)$$

$$\frac{\partial N_i}{\partial \xi} = \frac{1}{4}\xi_i(1 + \eta\eta_i)(1 - \zeta^2) \quad (3.18)$$

$$\frac{\partial N_i}{\partial \eta} = \frac{1}{4}\eta_i(1 + \xi\xi_i)(1 - \zeta^2) \quad (3.19)$$

$$\frac{\partial N_i}{\partial \zeta} = -\frac{1}{2}\zeta(1 + \xi\xi_i)(1 + \eta\eta_i) \quad (3.20)$$

La numerazione adottata è riportata nella Tabella 3.1.

Tabella 3.1: Coordinate naturali dei nodi dell'elemento Hexa20.

Nodo	ξ_i	η_i	ζ_i
1	-1	-1	-1
2	0	-1	-1
3	1	-1	-1
4	1	0	-1
5	1	1	-1
6	0	1	-1
7	-1	1	-1
8	-1	0	-1
9	-1	-1	0
10	1	-1	0
11	1	1	0
12	-1	1	0
13	-1	-1	1
14	0	-1	1
15	1	-1	1
16	1	0	1
17	1	1	1
18	0	1	1
19	-1	1	1
20	-1	0	1

4 Implementazione dello Script MATLAB

Il programma è strutturato attorno a un menu principale. Nei paragrafi seguenti si descrive il funzionamento dei sette bottoni principali.

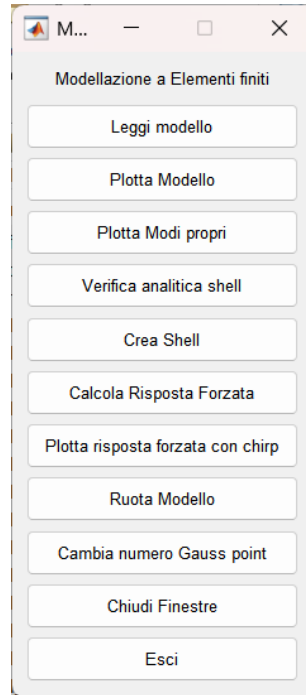


Figura 4.1: Menù principale

4.1 Leggi modello

Il bottone *Leggi modello* è il punto di ingresso dell'intera pipeline: alla sua pressione vengono eseguiti in sequenza la lettura della geometria, l'assemblaggio delle matrici strutturali e la soluzione del problema agli autovalori.

Lettura del file .el

Una finestra di dialogo consente di selezionare un file con estensione `.el`, il cui protocollo è il seguente:

- **Prima riga:** quattro interi n_d , n_{el} , n_v , n_f che specificano rispettivamente il numero di nodi, di elementi, di condizioni di vincolo e di condizioni di carico.
- **Sezione nodi** (n_d righe): coordinate cartesiane (x, y, z) di ciascun nodo.
- **Sezione elementi** (n_{el} blocchi): per ciascun elemento Hexa20, i 20 indici nodali globali seguiti dalle proprietà di materiale ρ , ν , E .
- **Sezione vincoli** (n_v blocchi): per ciascun vincolo elastico, l'indice dell'elemento interessato, gli intervalli parametrici che identificano la faccia vincolata, e i tre coefficienti di rigidità k_x, k_y, k_z .
- **Sezione forzanti** (n_f blocchi): struttura analoga ai vincoli, con la pressione superficiale p_x, p_y, p_z al posto delle rigidità.

I dati vengono organizzati in una struttura MATLAB che raccoglie coordinate nodali, connettività elementale e proprietà di materiale.

Assemblaggio di $\mathbf{K}$, $\mathbf{M}$, $\mathbf{F}$

Il programma richiede all'utente il numero di punti di Gauss per direzione n_g e procede all'assemblaggio delle matrici globali di rigidezza $\mathbf{K}$ e di massa $\mathbf{M}$ tramite integrazione di Gauss-Legendre, come descritto nelle sezioni 2.6–2.7. Se sono presenti condizioni al contorno, viene inoltre assemblato il vettore globale delle forze $\mathbf{F}$ e aggiunto il termine vincolante alla matrice $\mathbf{K}$.

Soluzione del problema agli autovalori

Immediatamente dopo l'assemblaggio, il programma risolve il problema agli autovalori generalizzato:

$$\mathbf{K} \boldsymbol{\phi}_i = \lambda_i \mathbf{M} \boldsymbol{\phi}_i, \quad i = 1, \dots, n \quad (4.1)$$

dove $\lambda_i = \omega_i^2$ e $\boldsymbol{\phi}_i$ è il corrispondente modo proprio. Le frequenze proprie in Hz si ricavano come:

$$f_i = \frac{\omega_i}{2\pi} = \frac{\sqrt{\lambda_i}}{2\pi} \quad (4.2)$$

L'utente specifica il numero n di modi da calcolare; il programma utilizza la funzione MATLAB `eigs` con l'opzione `'smallestabs'`, che restituisce gli n autovalori di modulo minimo, il quale numero n viene chiesto all'utente, utilizzando matrici $\mathbf{K}$ e $\mathbf{M}$ sparse, in modo tale che MATLAB non salvi né manipoli tutti gli zeri della matrice; infatti nei problemi FEM con gradi di libertà elevati le matrici globali presentano un numero di righe e colonne pari al numero di gradi libertà n , andando a creare $n \cdot n$ coefficienti all'interno delle matrici, di cui un gran numero sono zeri visto un nodo comunica con pochi nodi. Se si utilizzassero matrici piene, il solver dovrebbe andare a memorizzare tutti questi coefficienti, andando a occupare spazi enormi di RAM, motivo per il quale si utilizzano le matrici sparse che sono in grado di salvare solo i valori dei coefficienti non nulli e la riga e la colonna in cui si trovano, rendendo efficiente, nel nostro caso, il calcolo degli autovalori e degli autovettori.

Gli autovalori quasi nulli (sotto la soglia di 0.01 Hz) vengono poi azzerati, in quanto artefatti numerici dei modi di corpo rigido per poi essere ordinati in frequenza crescente assieme ai relativi autovettori.

4.2 Plotta modello

La visualizzazione della mesh avviene sfruttando le funzioni di forma isoparametriche dell'elemento Hexa20. Per ciascuno dei 12 spigoli di ogni elemento, due delle coordinate naturali (ξ, η, ζ) vengono fissate agli estremi ± 1 , mentre la terza viene campionata in N punti uniformi nell'intervallo $[-1, 1]$. Ogni punto in coordinate naturali viene mappato nello spazio fisico tramite la trasformazione isoparametrica:

$$\mathbf{x}(\xi, \eta, \zeta) = \sum_{i=1}^{20} N_i(\xi, \eta, \zeta) \mathbf{x}_i \quad (4.3)$$

dove $\mathbf{x}_i$ sono le coordinate nodali dell'elemento. Poiché le stesse funzioni di forma N_i utilizzate per approssimare il campo di spostamento descrivono anche la geometria (formulazione isoparametrica), esse sono in grado di interpolare esattamente qualsiasi punto interno all'elemento: il bordo visualizzato non è quindi una semplice spezzata tra nodi, ma la curva isoparametrica reale. Il numero di punti di campionamento N per spigolo è scelto dall'utente.

4.3 Plotta modi propri

Anche la visualizzazione dei modi propri si basa sul mapping isoparametrico: gli spostamenti nodali dell'autovettore ϕ_i vengono sommati alle coordinate dei nodi, e la mesh deformata viene tracciata con lo stesso meccanismo descritto sopra. Le shape functions interpolano quindi la deformazione in ogni punto degli spigoli, non soltanto nei nodi.

Prima di essere sommato alle coordinate, l'autovettore viene scalato secondo:

$$\tilde{\phi}_i = \frac{s_c \cdot L}{w_{\max}} \phi_i \quad (4.4)$$

dove L è la dimensione caratteristica della struttura (norma della diagonale del bounding box), $w_{\max}$ è la massima norma euclidea del triplo di spostamento (u, v, w) tra tutti i nodi del modo, e s_c è uno *scale factor* adimensionale scelto dall'utente (default 0.1). Questo garantisce che la deformazione visualizzata sia sempre proporzionale alle dimensioni della struttura, indipendentemente dal modo considerato o dalla normalizzazione degli autovettori.

Sono disponibili due modalità di visualizzazione:

- **Massima deformazione:** viene tracciata staticamente la mesh deformata di $\tilde{\phi}_i$, sovrapposta alla mesh indeformata in tratteggio. Rappresenta lo snapshot alla massima ampiezza del modo.
- **Animato:** la struttura viene animata per più cicli con n_{frame} fotogrammi per ciclo. Ad ogni fotogramma k l'ampiezza è modulata da un fattore sinusoidale:

$$\mathbf{u}(k) = \sin\left(\frac{2\pi(k-1)}{n_{\text{frame}}}\right) \tilde{\phi}_i \quad (4.5)$$

simulando l'oscillazione armonica del modo alla propria frequenza naturale.

4.4 Calcola risposta forzata

È stato poi implementato un bottone che permette all'utente di visualizzare la risposta forzata dell'intera struttura in modo animato per l'istante temporale sul quale la forza agisce su di essa, per fare ciò sono presenti due metodi, il metodo strutturale e l'approccio modale.

4.4.1 Metodo diretto (modello strutturale)

Il metodo diretto integra numericamente l'equazione del moto nella sua forma completa:

$$\mathbf{M} \ddot{\mathbf{x}}(t) + \mathbf{C} \dot{\mathbf{x}}(t) + \mathbf{K} \mathbf{x}(t) = \mathbf{F}_0 \alpha(t) \quad (4.6)$$

dove $\mathbf{F}_0 \in \mathbb{R}^{n_{\text{dof}}}$ è il vettore spaziale della forza assemblato dalle condizioni di carico del file `.e1`, e $\alpha(t)$ è la forzante temporale scalare scelta dall'utente. Il sistema del secondo ordine viene convertito nel sistema di stato del primo ordine introducendo $\mathbf{y} = [\mathbf{x}; \dot{\mathbf{x}}]^\top$:

$$\dot{\mathbf{y}} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{M}^{-1}\mathbf{K} & -\mathbf{M}^{-1}\mathbf{C} \end{bmatrix} \mathbf{y} + \begin{bmatrix} \mathbf{0} \\ \mathbf{M}^{-1}\mathbf{F}_0 \end{bmatrix} \alpha(t) \quad (4.7)$$

La risoluzione avviene con un solver ODE di MATLAB scelto dall'utente (ad es. `ode45` o `ode23s`), a partire da condizioni iniziali nulle. Il vantaggio di questo approccio è la generalità: non richiede assunzioni sullo smorzamento e funziona per qualsiasi matrice $\mathbf{C}$. Lo svantaggio è il costo computazionale, che scala con $2n_{\text{dof}}$ e può risultare proibitivo per modelli con molti gradi di libertà.

Tipologie di forzante temporale. L'utente può scegliere tra sei tipologie di forzante $\alpha(t)$, parametrizzabili interattivamente:

- **Random:** segnale casuale a distribuzione uniforme nell'intervallo $[A_{\min}, A_{\max}]$, campionato a frequenza f_c per N campioni.
- **Armonica:** senoide a frequenza, ampiezza e fase specificate dall'utente:

$$\alpha(t) = A \sin(2\pi f t + \varphi) \quad (4.8)$$

- **Serie di armoniche:** somma di n sinusoidi con ampiezze, frequenze e fasi estratte casualmente negli intervalli $[A_{\min}, A_{\max}]$, $[f_{\min}, f_{\max}]$ e $[-\pi, \pi]$.
- **Serie di armoniche smorzate:** come il caso precedente, ma ogni componente è moltiplicata per un inviluppo esponenziale decrescente con smorzamento ζ_i estratto casualmente in $[\zeta_{\min}, \zeta_{\max}]$:

$$\alpha(t) = \sum_{i=1}^n A_i e^{-\zeta_i \omega_i t} \sin(\omega_i t + \varphi_i) \quad (4.9)$$

- **Chirp:** sweep lineare di frequenza da f_0 a f_1 nel tempo totale di registrazione, utile per eccitare uniformemente una banda di frequenza.
- **Pseudo impulso:** segnale rettangolare di durata Δt centrato in t_0 , con ampiezza $A = I/\Delta t$ tale da garantire impulso unitario (o di valore I scelto dall'utente) al tendere di $\Delta t \rightarrow 0$.

4.4.2 Metodo per sovrapposizione modale

Il metodo modale sfrutta i modi propri $\Phi = [\phi_1, \dots, \phi_n]$ già calcolati per disaccoppiare le equazioni del moto. Introducendo le coordinate modali $\mathbf{q}(t)$ tramite la trasformazione:

$$\mathbf{x}(t) = \Phi \mathbf{q}(t) \quad (4.10)$$

e premoltiplicando per $\Phi^\top$, si ottiene un sistema di n equazioni scalari disaccoppiate (nell'ipotesi di smorzamento proporzionale):

$$m_i \ddot{q}_i(t) + c_i \dot{q}_i(t) + k_i q_i(t) = f_i \alpha(t), \quad i = 1, \dots, n \quad (4.11)$$

dove le grandezze modali sono definite come:

$$m_i = \boldsymbol{\phi}_i^\top \mathbf{M} \boldsymbol{\phi}_i, \quad k_i = \boldsymbol{\phi}_i^\top \mathbf{K} \boldsymbol{\phi}_i, \quad f_i = \boldsymbol{\phi}_i^\top \mathbf{F}_0 \quad (4.12)$$

Lo smorzamento modale viene introdotto tramite il rapporto di smorzamento ζ scelto dall'utente, che fornisce:

$$c_i = 2\zeta\omega_i m_i \quad (4.13)$$

Le n equazioni scalari vengono integrate con lo stesso solver ODE del metodo diretto, a partire da condizioni iniziali nulle in coordinate modali. La risposta fisica viene poi recuperata tramite:

$$\mathbf{x}(t) = \sum_{i=1}^n \boldsymbol{\phi}_i q_i(t) \quad (4.14)$$

Il vantaggio rispetto al metodo diretto è la drastica riduzione della dimensione del sistema integrato: si passa da $2n_{\text{dof}}$ a $2n$ equazioni, con $n \ll n_{\text{dof}}$, rendendo il metodo molto più efficiente per strutture con molti gradi di libertà, a patto che la risposta sia ben rappresentata dagli n modi calcolati. I modi rigidi (frequenza < 0.1 Hz) vengono automaticamente esclusi dalla sovrapposizione.

In entrambi i casi, la risposta calcolata viene infine animata con il medesimo meccanismo isoparametrico descritto per la visualizzazione dei modi propri.

4.5 Ruota modello

Il bottone *Ruota modello* applica una rotazione rigida all'intero modello FEM, richiedendo all'utente tre angoli $\theta_x, \theta_y, \theta_z$ in gradi. La trasformazione viene eseguita applicando la rotazione composta $\mathbf{R} = \mathbf{R}_x \mathbf{R}_y \mathbf{R}_z$ alle coordinate di tutti i nodi, aggiornando la struttura geometrica senza modificare la connettività elementale né le proprietà di materiale.

Dopo la rotazione, il programma ricalcola le matrici $\mathbf{K}$ e $\mathbf{M}$ e risolve nuovamente il problema agli autovalori. Poiché una rotazione rigida è una trasformazione ortogonale che preserva distanze e volumi, le frequenze proprie della struttura devono rimanere invariate. Questo comportamento costituisce una verifica di consistenza del modello: se le frequenze proprie non cambiano al variare dell'orientamento nello spazio, significa che il solver sta integrando correttamente le matrici elementali, che l'assemblaggio globale è coerente e che il modello è impostato correttamente.

4.6 Crea Shell

Il bottone *Crea Shell* genera automaticamente la mesh a elementi finiti di un guscio cilindrico cavo, discretizzato con elementi solidi Serendipity Hexa20, e scrive il corrispondente file `.el`. L'utente fornisce i parametri geometrici (R, h, L), le proprietà di materiale (ρ, ν, E), la discretizzazione (n_θ, n_z, n_r) e la rigidezza di penalità k_{pen} per i vincoli alle estremità.

Parametrizzazione della geometria

Il cilindro ha asse coincidente con z , raggio medio R , spessore h e lunghezza L . Un punto generico sulla superficie è individuato dalle coordinate cilindriche (θ, z, r) , con:

$$x = r \cos \theta, \quad y = r \sin \theta, \quad z = z \quad (4.15)$$

dove i domini delle tre variabili sono:

$$\theta \in [0, 2\pi), \quad z \in [0, L], \quad r \in \left[R - \frac{h}{2}, R + \frac{h}{2} \right] \quad (4.16)$$

Le tre direzioni fisiche vengono associate alle coordinate naturali dell'elemento Hexa20 come segue:

$$\xi \longleftrightarrow \theta, \quad \eta \longleftrightarrow z, \quad \zeta \longleftrightarrow r \quad (4.17)$$

Griglia parametrica raddoppiata

Poiché l'elemento Hexa20 è un elemento Serendipity quadratico, ogni elemento occupa due intervalli nella griglia parametrica sottostante. Si introduce pertanto una griglia raddoppiata con indici interi:

$$a \in \{0, 1, \dots, N_A - 1\}, \quad b \in \{0, 1, \dots, N_B\}, \quad c \in \{0, 1, \dots, N_C\} \quad (4.18)$$

dove $N_A = 2n_\theta$, $N_B = 2n_z$, $N_C = 2n_r$. Le coordinate fisiche associate al punto (a, b, c) della griglia sono:

$$\theta_a = \frac{2\pi a}{N_A}, \quad z_b = \frac{L b}{N_B}, \quad r_c = R - \frac{h}{2} + \frac{h c}{N_C} \quad (4.19)$$

e le coordinate cartesiane del nodo corrispondente sono:

$$x = r_c \cos \theta_a, \quad y = r_c \sin \theta_a, \quad z = z_b \quad (4.20)$$

Regola di selezione Serendipity

Non tutti i punti della griglia raddoppiata sono nodi dell'elemento Hexa20: l'elemento Serendipity esclude il nodo di volume centrale presente nell'elemento Lagrangiano a 27 nodi. La condizione che seleziona i soli nodi Serendipity è:

$$\text{mod}(a, 2) + \text{mod}(b, 2) + \text{mod}(c, 2) \leq 1 \quad (4.21)$$

Questa condizione ammette solo i punti della griglia per i quali *al più una* delle tre coordinate è dispari (cioè si trova a metà intervallo). In pratica vengono inclusi:

- i **vertici** dell'elemento: $\text{mod}(a, 2) = \text{mod}(b, 2) = \text{mod}(c, 2) = 0$, somma = 0;
- i **nodi di spigolo** (un indice dispari): somma = 1;
- esclusi i **nodi di faccia** (due indici dispari, somma = 2) e il **nodo di volume** (somma = 3).

Ai nodi che soddisfano la condizione viene assegnato un indice globale progressivo n_d , memorizzato nella mappa tridimensionale:

$$\text{gid}(a + 1, b + 1, c + 1) = n_d \quad (4.22)$$

Periodicità circonferenziale

La direzione θ è periodica: il punto $a = N_A$ coincide con $a = 0$. Questo è gestito applicando l'operazione di modulo all'indice a ogni volta che viene calcolata la connettività:

$$a \leftarrow \text{mod}(a, N_A) \quad (4.23)$$

In questo modo la chiusura del cilindro è garantita senza duplicazione di nodi: il primo e l'ultimo strato circonferenziale condividono esattamente gli stessi nodi globali.

Costruzione della connettività elementale

Gli elementi sono indicizzati da tre indici $e_\theta \in \{0, \dots, n_\theta - 1\}$, $e_z \in \{0, \dots, n_z - 1\}$, $e_r \in \{0, \dots, n_r - 1\}$. Per ciascun elemento, le posizioni dei 20 nodi locali nelle coordinate naturali $(\xi, \eta, \zeta) \in \{-1, 0, 1\}^3$ sono note a priori e raccolte nella matrice $\mathbf{nat} \in \mathbb{Z}^{20 \times 3}$, riportata in tabella 4.1.

Nodo locale	ξ	η	ζ
1	-1	-1	-1
2	0	-1	-1
3	1	-1	-1
4	1	0	-1
5	1	1	-1
6	0	1	-1
7	-1	1	-1
8	-1	0	-1
9	-1	-1	0
10	1	-1	0
11	1	1	0
12	-1	1	0
13	-1	-1	1
14	0	-1	1
15	1	-1	1
16	1	0	1
17	1	1	1
18	0	1	1
19	-1	1	1
20	-1	0	1

Tabella 4.1: Coordinate naturali dei 20 nodi locali dell'elemento Hexa20 (convenzione Dhatt & Touzot [2]).

Per l' m -esimo nodo locale dell'elemento (e_θ, e_z, e_r) , gli indici nella griglia raddoppiata sono:

$$a_m = \text{mod}(2e_\theta + \mathbf{nat}(m, 1) + 1, N_A), \quad b_m = 2e_z + \mathbf{nat}(m, 2) + 1, \quad c_m = 2e_r + \mathbf{nat}(m, 3) + 1 \quad (4.24)$$

e l'indice globale del nodo locale m è:

$$\text{indx}(m) = \text{gid}(a_m + 1, b_m + 1, c_m + 1) \quad (4.25)$$

Il vettore $\mathbf{indx} \in \mathbb{Z}^{20}$ costituisce la connettività dell'elemento, da cui si ricava il vettore degli indici globali dei gradi di libertà:

$$\text{idofs}(3(k-1) + j) = 3(\mathbf{indx}(k) - 1) + j, \quad k = 1, \dots, 20, \quad j = 1, 2, 3 \quad (4.26)$$

Condizioni al contorno: appoggio semplice tipo shear diaphragm

Alle estremità $z = 0$ e $z = L$ vengono imposte condizioni di appoggio semplice senza vincolo assiale (*shear diaphragm*), coerenti con il caso di riferimento di Blevins [1] (sez. 12.2.3):

$$u_r = 0, \quad u_\theta = 0, \quad N_z = 0 \quad \text{su } z = 0 \text{ e } z = L \quad (4.27)$$

Poiché il codice utilizza elementi volumici tridimensionali, i vincoli non sono imposti a livello di grado di libertà nodale ma per via di **penalità superficiale**: su ciascuna faccia di estremità ($\eta = -1$ per $z = 0$, $\eta = +1$ per $z = L$) di ogni elemento del primo e dell'ultimo strato assiale viene applicata una distribuzione di molle con rigidezza k_{pen} nelle direzioni x e y e rigidezza nulla in direzione z :

$$\mathbf{k}_{\text{vinc}} = \begin{bmatrix} k_{\text{pen}} \\ k_{\text{pen}} \\ 0 \end{bmatrix} \quad (4.28)$$

Il contributo di questi vincoli alla matrice globale di rigidezza è calcolato e assemblato dalla funzione `assembla_vinc` tramite integrazione di Gauss sulla faccia vincolata, esattamente come avviene per le matrici elementali. Il valore consigliato per k_{pen} è:

$$k_{\text{pen}} \sim 10^3 \div 10^6 \cdot \frac{E}{h} \quad (4.29)$$

Un valore troppo basso non vincola efficacemente la struttura; un valore troppo alto degrada il condizionamento numerico di $\mathbf{K}$. La scelta ottimale dipende dal rapporto h/R e dalla finezza della mesh.

4.7 Verifica analitica Shell

Forma dei modi propri

Per un guscio cilindrico semplicemente appoggiato alle estremità con condizione *shear diaphragm* (sezione 12.2.3 di Blevins [1]), il campo di spostamento che soddisfa simultaneamente le equazioni del moto e le condizioni al contorno ha la forma:

$$u = A \cos \frac{j\pi x}{L} \cos i\theta \cos \omega t, \quad v = B \sin \frac{j\pi x}{L} \sin i\theta \cos \omega t, \quad w = C \sin \frac{j\pi x}{L} \cos i\theta \cos \omega t \quad (4.30)$$

dove $i = 0, 1, 2, \dots$ è il numero di **onde circolari** e $j = 1, 2, 3, \dots$ è il numero di **mezzo onde assiali**. Ogni coppia (i, j) individua una famiglia di modi propri distinta.

Equazione caratteristica di Donnell

Sostituendo la forma modale nell'equazione del moto della teoria di Donnell per gusci cilindrici sottili, si ottiene il sistema lineare omogeneo (eq. 12-14 di Blevins [1]):

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} A \\ B \\ C \end{bmatrix} = \mathbf{0} \quad (4.31)$$

Per avere soluzioni non banali il determinante deve essere nullo, e nell'ipotesi di guscio sottile ($k \ll 1$) il problema si riduce a un polinomio cubico in λ^2 :

$$\lambda^6 - K_2 \lambda^4 + K_1 \lambda^2 - K_0 = 0 \quad (4.32)$$

dove λ è il parametro adimensionale di frequenza legato alla pulsazione ω dalla relazione:

$$\lambda^2 = \frac{\mu(1 - \nu^2) R^2 \omega^2}{E} \quad (4.33)$$

I coefficienti del polinomio sono definiti a partire da due parametri adimensionali fondamentali:

$$\xi = \frac{j\pi R}{L}, \quad k = \frac{h^2}{12R^2}, \quad s = i^2 + \xi^2 \quad (4.34)$$

come:

$$K_2 = 1 + \frac{1}{2}(3 - \nu) s + k s^2 \quad (4.35)$$

$$K_1 = \frac{1 - \nu}{2} \left[(3 + 2\nu) \xi^2 + i^2 + s^2 + \frac{3 - \nu}{1 - \nu} k s^3 \right] \quad (4.36)$$

$$K_0 = \frac{1 - \nu}{2} [(1 - \nu^2) \xi^4 + k s^4] \quad (4.37)$$

Le tre radici e il modo fondamentale

Per ogni coppia (i, j) il polinomio (4.32) ha tre radici reali positive in λ^2 , che corrispondono a tre famiglie di modi:

- la radice più piccola è associata al modo **flessionale** (dominato da w), che ha la frequenza più bassa ed è il modo di interesse strutturale;
- le due radici maggiori corrispondono ai modi **membranal**i (assiale e circonferenziale), con frequenze molto più elevate.

La frequenza naturale associata alla radice minima $\lambda_{\min}$ è:

$$f_{ij} = \lambda_{\min} \cdot \frac{1}{2\pi R} \sqrt{\frac{E}{\rho(1 - \nu^2)}} \quad (4.38)$$

Degenerazione per $i \geq 1$: doppietti modali

Per $i = 0$ il modo è assisimmetrico: la distribuzione di spostamento è uniforme in θ e ogni frequenza f_{0j} appartiene a un unico modo. Per $i \geq 1$, invece, la forma $\cos i\theta$ e la forma $\sin i\theta$ hanno la stessa frequenza ma orientamento angolare sfasato di $\pi/(2i)$: si tratta di due modi fisicamente distinti ma degeneri in frequenza. Pertanto ogni f_{ij} con $i \geq 1$ compare **due volte** nello spettro della struttura reale.

Implementazione nello script

La funzione `freq_analitica_shell` esplora sistematicamente tutte le coppie (i, j) con $i \in \{0, \dots, i_{\max}\}$ e $j \in \{1, \dots, j_{\max}\}$. Per ciascuna coppia risolve il polinomio cubico (4.32) tramite la funzione MATLAB `roots`, estrae le radici reali positive e calcola f_{ij} dalla radice minima. I risultati sono organizzati nella matrice `freq_tab` di dimensioni $(i_{\max} + 1) \times j_{\max}$.

La funzione `confronta_freq_shell` costruisce poi lo spettro analitico ordinato applicando la regola di degenerazione: le frequenze con $i = 0$ compaiono una volta, quelle con $i \geq 1$ vengono duplicate prima dell'ordinamento, in modo da avere una sequenza direttamente comparabile con quella FEM. Il confronto finale riporta per ciascun modo l'errore percentuale:

$$\epsilon_m = \frac{f_m^{\text{FEM}} - f_m^{\text{anal}}}{f_m^{\text{anal}}} \times 100\% \quad (4.39)$$

e al termine viene calcolato l'errore medio su tutti i modi confrontati:

$$\bar{\epsilon} = \frac{1}{n} \sum_{m=1}^n |\epsilon_m| \quad (4.40)$$

4.8 Plotta risposta forzata con chirp

Una volta calcolata la risposta forzata con forzante di tipo chirp, è disponibile una funzione di verifica che permette di confrontare visivamente i picchi della risposta in frequenza con le frequenze proprie del modello modale. Questa analisi costituisce una verifica di consistenza interna del codice: se il modello è corretto e il solver ha integrato bene le equazioni del moto, i picchi della FRF devono coincidere con le frequenze proprie calcolate dal problema agli autovalori.

Perché il chirp

Il segnale chirp è uno sweep sinusoidale a frequenza linearmente crescente da f_0 a f_1 :

$$\alpha(t) = \sin\left(2\pi \left(f_0 + \frac{f_1 - f_0}{2T} t\right) t\right) \quad (4.41)$$

Il suo spettro $|F(f)|$ è pressoché piatto nella banda $[f_0, f_1]$: ogni frequenza viene eccitata con la stessa intensità. Questa proprietà lo rende ideale per l'identificazione modale, perché la risposta in frequenza è direttamente proporzionale alla funzione di risposta in frequenza della struttura senza necessità di correzioni spettrali particolari.

Calcolo della FRF

Prima di chiedere il nodo di analisi, il programma plotta il modello con la numerazione dei nodi tramite `plotta_modello_nodi`, in modo che l'utente possa identificare visivamente il nodo di interesse. L'utente sceglie poi il nodo n e la direzione $d \in \{x, y, z\}$, che individuano il grado di libertà globale:

$$i_{\text{dof}} = 3(n - 1) + d \quad (4.42)$$

La storia temporale dello spostamento $x(t)$ al grado di libertà scelto viene estratta dalla matrice di risposta `risp.x`. Poiché il solver ODE restituisce un vettore tempi a passo variabile, il segnale viene prima ricampionato su una griglia uniforme di $N = 2^{\lceil \log_2 n_t \rceil}$ punti (potenza di 2 per efficienza della FFT) tramite interpolazione cubica `pchip`. La frequenza di campionamento risultante è $f_c = 1/\Delta t$.

Si calcola poi la trasformata di Fourier discreta monolaterale della risposta:

$$X(f) = \text{FFT}\{x(t)\}, \quad f \in \left[0, \frac{f_c}{2}\right] \quad (4.43)$$

e analogamente quella della forzante $\alpha(t)$ ricampionata sulla stessa griglia:

$$F(f) = \text{FFT}\{\alpha(t)\} \quad (4.44)$$

La **Frequency Response Function** (FRF) è definita come il rapporto tra i due spettri:

$$G(f) = \frac{X(f)}{F(f)} \quad (4.45)$$

La normalizzazione per $F(f)$ rimuove l'effetto dell'ampiezza della forzante e lascia emergere la sola risposta strutturale. Per evitare divisioni per zero nelle zone fuori banda (dove $|F(f)| \approx 0$), si applica una soglia di tolleranza $\varepsilon = 10^{-6} \cdot \max |F(f)|$.

Visualizzazione e verifica

Il plot è composto da due pannelli sovrapposti. Il pannello superiore mostra $|G(f)|$ in scala logaritmica (stile Bode), con linee verticali tratteggiate rosse in corrispondenza di ciascuna frequenza propria f_i calcolata dal modello modale. Il pannello inferiore mostra $|F(f)|$, permettendo di verificare che la banda del chirp copra effettivamente le frequenze proprie di interesse: se $|F(f)| \approx 0$ in corrispondenza di una frequenza propria, quel modo non è stato eccitato e il picco corrispondente non apparirà nella FRF. L'asse delle frequenze è limitato a $1.1 f_1$ per escludere la zona fuori banda dove la FRF non ha significato fisico.

La verifica è considerata soddisfacente quando i picchi di $|G(f)|$ cadono in corrispondenza delle linee rosse tratteggiate. È importante osservare che non tutti i modi propri producono necessariamente un picco visibile nella FRF: se la forzante è applicata in una direzione ortogonale alla forma modale, il modo non viene eccitato e il picco corrispondente non compare nella risposta. La coincidenza tra i picchi osservati e le frequenze proprie attese conferma che le frequenze calcolate per via algebrica dal problema agli autovalori coincidono con quelle che emergono dall'integrazione temporale delle equazioni del moto, validando simultaneamente l'assemblaggio delle matrici, la soluzione modale e il solver ODE.

5 Risultati numerici e verifiche

Prima di presentare i risultati modali e dinamici, è necessario verificare che le matrici assemblate dal codice siano fisicamente corrette e numericamente ben condizionate. Le tre verifiche descritte nei paragrafi seguenti sono indipendenti tra loro e testano aspetti distinti del codice: la correttezza dell'integrazione di Gauss, la coerenza energetica della matrice di rigidezza e l'invarianza delle frequenze proprie per trasformazioni isometriche.

5.1 Verifica della matrice di massa: conservazione della massa totale

La matrice di massa globale è definita come:

$$\mathbf{M} = \sum_{e=1}^{n_{el}} \int_{V_e} \rho \mathbf{N}_e^T \mathbf{N}_e dV_e \quad (5.1)$$

dove $\mathbf{N}_e$ è la matrice delle funzioni di forma dell'elemento e . Una proprietà fondamentale delle funzioni di forma è la **partizione dell'unità**:

$$\sum_{i=1}^{20} N_i(\xi, \eta, \zeta) = 1 \quad \forall (\xi, \eta, \zeta) \in [-1, 1]^3 \quad (5.2)$$

Questa proprietà garantisce che un moto di corpo rigido (traslazione uniforme in una direzione) sia rappresentato esattamente dall'elemento. Considerando il vettore di spostamento rigido in direzione x :

$$\mathbf{u}_x = [\underbrace{1, 0, 0}_{\text{nodo 1}}, \underbrace{1, 0, 0}_{\text{nodo 2}}, \dots, \underbrace{1, 0, 0}_{\text{nodo } n_d}]^T \quad (5.3)$$

il prodotto $\mathbf{u}_x^T \mathbf{M} \mathbf{u}_x$ rappresenta l'energia cinetica associata a una traslazione rigida con velocità unitaria. Sfruttando la partizione dell'unità, questo prodotto deve essere uguale alla massa totale della struttura:

$$\mathbf{u}_x^T \mathbf{M} \mathbf{u}_x = \sum_{e=1}^{n_{el}} \int_{V_e} \rho \left(\sum_{i=1}^{20} N_i \right)^2 dV_e = \sum_{e=1}^{n_{el}} \int_{V_e} \rho dV_e = M_{\text{tot}} \quad (5.4)$$

Per simmetria, lo stesso vale per le direzioni y e z . La verifica si esegue costruendo il vettore $\mathbf{u}_x$ e calcolando l'errore relativo:

$$\varepsilon_M = \frac{|\mathbf{u}_x^T \mathbf{M} \mathbf{u}_x - M_{\text{tot}}|}{M_{\text{tot}}} \times 100\% \quad (5.5)$$

Applicazione al modello dell'arco

Il modello di verifica è un settore di anello cilindrico pieno (arco) discretizzato con 5 elementi Hexa20, mostrato in Figura 5.1, con le proprietà seguenti:

$$R_{\text{in}} = 4 \text{ m}, \quad R_{\text{out}} = 6 \text{ m}, \quad \Delta\theta = \frac{\pi}{2}, \quad L = 2 \text{ m}, \quad \rho = 7800 \text{ kg/m}^3 \quad (5.6)$$

Il volume analitico del settore è:

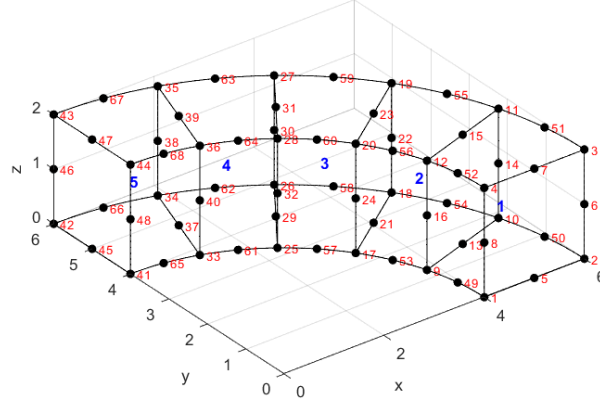


Figura 5.1: Modello dell'arco: settore di anello cilindrico pieno discretizzato con 5 elementi Hexa20. In rosso la numerazione dei nodi, in blu la numerazione degli elementi.

$$V = \int_0^{\pi/2} d\theta \int_{R_{in}}^{R_{out}} r dr \int_0^L dz = \frac{\pi}{2} \cdot \frac{R_{out}^2 - R_{in}^2}{2} \cdot L = \frac{\pi}{2} \cdot 10 \cdot 2 = 10\pi \approx 31.416 \text{ m}^3 \quad (5.7)$$

da cui la massa teorica:

$$M_{tot} = \rho \cdot V = 7800 \cdot 10\pi = 78000\pi \approx 245044.23 \text{ kg} \quad (5.8)$$

In Figura 5.2 è riportato il calcolo eseguito in MATLAB, dal quale si ottiene:

$$\mathbf{u}_x^T \mathbf{M} \mathbf{u}_x = 245037.17 \text{ kg} \quad (5.9)$$

```
>> v=zeros(68*3,1);
>> for i=1:size(v)
v(3*i-2)=1;
end
>> v=zeros(68*3,1);
>> for i=1:68
v(3*i-2)=1;
end
>> v'*s.M*v

ans =

2.450371712359997e+05

; >> |
```

Figura 5.2: Calcolo di $\mathbf{u}_x^T \mathbf{M} \mathbf{u}_x$ in MATLAB per il modello dell'arco.

L'errore relativo rispetto al valore analitico è:

$$\varepsilon_M = \frac{|245037.17 - 245044.23|}{245044.23} \times 100\% \approx 0.003\% \quad (5.10)$$

	Valore [kg]
$\mathbf{u}_x^T \mathbf{M} \mathbf{u}_x$ (FEM)	245 037.17
$M_{\text{tot}} = 78000\pi$ (analitico)	245 044.23
Errore relativo ε_M	0.003%

Tabella 5.1: Verifica della conservazione della massa totale – modello arco, 5 elementi Hexa20.

L'errore dello 0.003% è imputabile all'errore di quadratura di Gauss su una geometria curva discretizzata con soli 5 elementi: i bordi degli elementi Hexa20, essendo polinomi quadratici, approssimano la curvatura circolare con un piccolo scarto geometrico che si riduce al raffinamento della mesh. Il risultato conferma che l'integrazione numerica e l'assemblaggio della matrice di massa sono corretti.

5.2 Verifica della matrice di rigidezza: semidefinita positività

La matrice di rigidezza globale è semidefinita positiva per costruzione fisica: l'energia di deformazione elastica è sempre non negativa. Formalmente:

$$\mathbf{u}^T \mathbf{K} \mathbf{u} \geq 0 \quad \forall \mathbf{u} \in \mathbb{R}^{n_{\text{dof}}} \quad (5.11)$$

con uguaglianza se e solo se $\mathbf{u}$ è un modo di corpo rigido (traslazione o rotazione pura). La verifica viene eseguita su due classi di vettori di test.

Modi di corpo rigido. Le tre traslazioni rigide $\mathbf{u}_x$, $\mathbf{u}_y$, $\mathbf{u}_z$ corrispondono fisicamente a un moto in cui la struttura trasla come un corpo rigido senza deformarsi: l'energia interna deve essere nulla. In termini di indici globali, il vettore $\mathbf{u}_x$ ha valore 1 agli indici $3i - 2$ (DOF in x del nodo i) e 0 altrove; analogamente per $\mathbf{u}_y$ (indici $3i - 1$) e $\mathbf{u}_z$ (indici $3i$). I risultati ottenuti sono:

$$\mathbf{u}_x^T \mathbf{K} \mathbf{u}_x = -4.60 \times 10^{-4} \text{ N/m}, \quad \mathbf{u}_y^T \mathbf{K} \mathbf{u}_y = 1.52 \times 10^{-3} \text{ N/m}, \quad \mathbf{u}_z^T \mathbf{K} \mathbf{u}_z = 1.23 \times 10^{-3} \text{ N/m} \quad (5.12)$$

Questi valori, rapportati all'ordine di grandezza di $\mathbf{K}$ (stimato tramite i vettori casuali, $\sim 10^{12}$), danno un errore relativo:

$$\frac{|\mathbf{u}_x^T \mathbf{K} \mathbf{u}_x|}{\mathbf{v}^T \mathbf{K} \mathbf{v}} \sim \frac{10^{-4}}{10^{12}} \sim 10^{-16} \approx \varepsilon_{\text{macchina}} \quad (5.13)$$

I risultati sono quindi compatibili con zero a meno della precisione macchina ($\varepsilon_{\text{macchina}} \approx 2.22 \times 10^{-16}$), confermando che la matrice non introduce energia artificiosa per moti rigidi.

Vettori casuali. Per due vettori casuali $\mathbf{v}_1, \mathbf{v}_2 \sim \mathcal{U}[0, 1]^{n_{\text{dof}}}$ si verifica che il prodotto sia strettamente positivo:

$$\mathbf{v}_1^T \mathbf{K} \mathbf{v}_1 = 6.521 \times 10^{12} \text{ N/m} > 0, \quad \mathbf{v}_2^T \mathbf{K} \mathbf{v}_2 = 3.876 \times 10^{12} \text{ N/m} > 0 \quad (5.14)$$

Tutti i test sono superati: i modi di corpo rigido producono valori a livello della precisione macchina, mentre i vettori generici producono energia di deformazione strettamente

Vettore di test	$\mathbf{u}^T \mathbf{K} \mathbf{u}$ [N/m]	Atteso	Esito
Traslazione x	-4.60×10^{-4}	≈ 0	✓
Traslazione y	1.52×10^{-3}	≈ 0	✓
Traslazione z	1.23×10^{-3}	≈ 0	✓
Vettore casuale 1	6.521×10^{12}	> 0	✓
Vettore casuale 2	3.876×10^{12}	> 0	✓

Tabella 5.2: Verifica della semidefinita positività di $\mathbf{K}$ – modello arco, 5 elementi Hexa20.

positiva. La matrice $\mathbf{K}$ è pertanto semidefinita positiva, coerentemente con la fisica del problema.

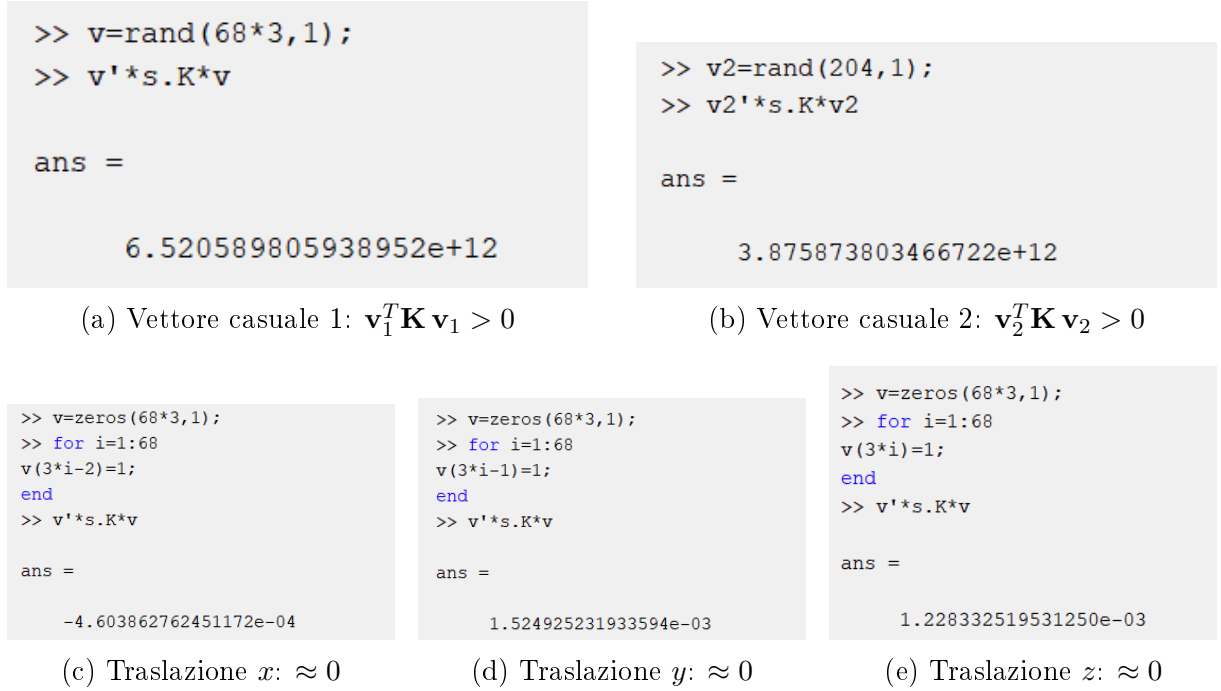


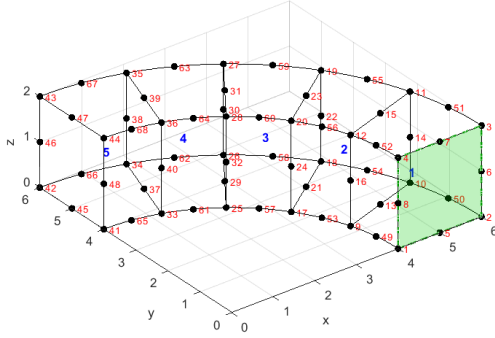
Figura 5.3: Calcoli MATLAB per la verifica della semidefinita positività di $\mathbf{K}$: in alto i vettori casuali (risultato > 0), in basso le traslazioni rigide (risultato ≈ 0 a livello di precisione macchina).

5.3 Verifica per rotazione del modello: invarianza delle frequenze proprie

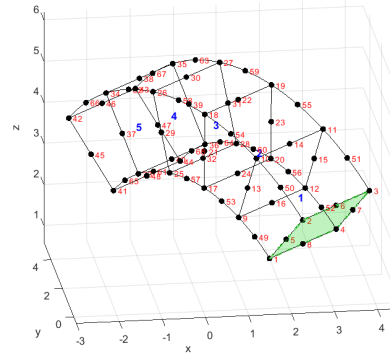
Come descritto nella sezione dedicata al bottone *Ruota modello* 4.5, le frequenze proprie di una struttura sono invarianti per trasformazioni isometriche. La verifica viene condotta sullo stesso modello dell'arco, al quale è stato aggiunto un vincolo elastico sulla faccia evidenziata in verde in Figura 5.4: questo elimina i modi di corpo rigido e rende il problema agli autovalori ben posto, con tutte le frequenze strettamente positive.

La verifica consiste nel calcolare le prime 20 frequenze proprie prima e dopo una rotazione di 45° su ogni asse e confrontare i due spettri tramite l'errore relativo:

$$\varepsilon_{f_i} = \frac{|f_i^{\text{rot}} - f_i^{\text{orig}}|}{f_i^{\text{orig}}} \times 100\% \quad (5.15)$$



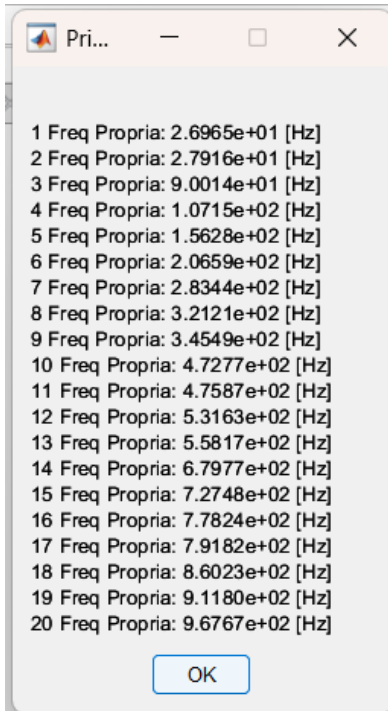
(a) Modello originale con vincolo sulla faccia verde.



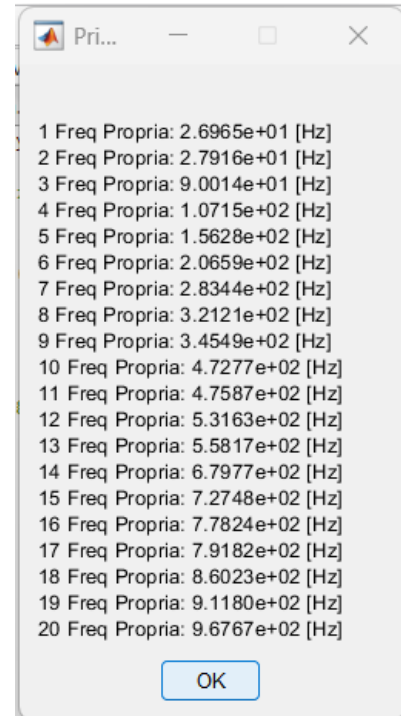
(b) Stesso modello ruotato di 45° su ogni asse.

Figura 5.4: Modello dell'arco vincolato prima e dopo la rotazione di 45° su ogni asse.

Le due finestre di output del programma sono riportate in Figura 5.5: le frequenze sono identiche alla precisione di visualizzazione, confermando l'invarianza dello spettro per rotazione.



(a) Prime 20 frequenze proprie
– orientamento originale.



(b) Prime 20 frequenze proprie
– modello ruotato di 45°.

Figura 5.5: Confronto delle frequenze proprie prima e dopo la rotazione.

Tutti gli errori sono al di sotto della precisione macchina ($\varepsilon_{\text{mac}} \approx 2.22 \times 10^{-16}$), confermando che la rotazione è implementata correttamente: le matrici $\mathbf{K}$ e $\mathbf{M}$ vengono riassemblate in modo consistente sulla geometria ruotata, e lo spettro dinamico della struttura è invariante per trasformazioni isometriche come atteso dalla teoria.

Modo	f_i^{orig} [Hz]	f_i^{rot} [Hz]	ε_{f_i} [%]
1	26.965	26.965	$< \varepsilon_{\text{mac}}$
2	27.916	27.916	$< \varepsilon_{\text{mac}}$
3	90.014	90.014	$< \varepsilon_{\text{mac}}$
4	107.15	107.15	$< \varepsilon_{\text{mac}}$
5	156.28	156.28	$< \varepsilon_{\text{mac}}$
6	206.59	206.59	$< \varepsilon_{\text{mac}}$
7	283.44	283.44	$< \varepsilon_{\text{mac}}$
8	321.21	321.21	$< \varepsilon_{\text{mac}}$
9	345.49	345.49	$< \varepsilon_{\text{mac}}$
10	472.77	472.77	$< \varepsilon_{\text{mac}}$

Tabella 5.3: Confronto delle prime 10 frequenze proprie prima e dopo rotazione di 45 – modello arco vincolato.

Le tre verifiche condotte – conservazione della massa, semidefinita positività della matrice di rigidezza e invarianza delle frequenze proprie per rotazione – dimostrano congiuntamente che il codice assembla correttamente le matrici strutturali e che il solutore agli autovalori produce risultati fisicamente consistenti. Tuttavia, queste verifiche sono di natura *interna*: esse accertano la coerenza del modello con sé stesso, ma non ne garantiscono l'accuratezza rispetto alla realtà fisica. Per questo motivo, nella sezione seguente si confrontano le frequenze proprie calcolate dal codice FEM con quelle previste dalla teoria analitica di Donnell per gusci cilindrici, fornendo una validazione *esterna* del modello.

5.4 Confronto con la teoria analitica di Donnell

La validazione esterna del codice viene condotta su uno shell cilindrico con le seguenti proprietà:

$$R = 1 \text{ m}, \quad h = 0.02 \text{ m}, \quad L = 4 \text{ m}, \quad \frac{h}{R} = 0.02, \quad \frac{L}{R} = 4 \quad (5.16)$$

$$\rho = 7800 \text{ kg/m}^3, \quad \nu = 0.3, \quad E = 2.1 \times 10^{11} \text{ Pa} \quad (5.17)$$

Il rapporto $h/R = 0.02$ rientra ampiamente nel dominio di validità della teoria di Donnell ($h/R \ll 1$), rendendo il confronto significativo.

Calcolo esplicito delle frequenze analitiche

Per il modello di verifica, i parametri adimensionali della teoria di Donnell assumono i valori:

$$k = \frac{h^2}{12R^2} = \frac{(0.02)^2}{12 \cdot 1^2} = 3.33 \times 10^{-5} \quad (5.18)$$

$$c_f = \frac{1}{2\pi R} \sqrt{\frac{E}{\rho(1-\nu^2)}} = \frac{1}{2\pi} \sqrt{\frac{2.1 \times 10^{11}}{7800 \cdot (1-0.3^2)}} \approx 866.6 \text{ Hz} \quad (5.19)$$

Per ogni coppia (i, j) si calcola innanzitutto il parametro ξ , che misura il numero di mezze onde assiali rapportato alla geometria:

$$\xi = \frac{j\pi R}{L} = \frac{j\pi \cdot 1}{4} = \frac{j\pi}{4} \quad (5.20)$$

e la quantità ausiliaria $s = i^2 + \xi^2$. Per il modo fondamentale $i = 3$, $j = 1$:

$$\xi = \frac{\pi}{4} \approx 0.7854, \quad s = 3^2 + 0.7854^2 = 9 + 0.617 = 9.617 \quad (5.21)$$

I coefficienti del polinomio caratteristico cubico in λ^2 sono:

$$K_2 = 1 + \frac{1}{2}(3 - 0.3) \cdot 9.617 + 3.33 \times 10^{-5} \cdot 9.617^2 = 1 + 13.46 + 3.08 \times 10^{-3} \approx 14.46 \quad (5.22)$$

$$K_1 = \frac{1 - 0.3}{2} \left[(3 + 2 \cdot 0.3) \cdot 0.617 + 9 + 9.617^2 + \frac{3 - 0.3}{1 - 0.3} \cdot 3.33 \times 10^{-5} \cdot 9.617^3 \right] \approx 42.03 \quad (5.23)$$

$$K_0 = \frac{1 - 0.3}{2} [(1 - 0.3^2) \cdot 0.617^2 + 3.33 \times 10^{-5} \cdot 9.617^4] \approx 0.1237 \quad (5.24)$$

Il polinomio cubico in λ^2 :

$$\lambda^6 - 14.46 \lambda^4 + 42.03 \lambda^2 - 0.1237 = 0 \quad (5.25)$$

ha tre radici reali positive. La radice minima, associata al modo flessionale, è:

$$\lambda_{\min}^2 \approx 2.94 \times 10^{-3} \quad \Rightarrow \quad \lambda_{\min} \approx 0.05422 \quad (5.26)$$

La frequenza naturale corrispondente è:

$$f_{3,1} = \lambda_{\min} \cdot c_f = 0.05422 \times 866.6 \approx 67.59 \text{ Hz} \quad (5.27)$$

in perfetto accordo con la frequenza fondamentale riportata dal programma. Poiché $i = 3 \geq 1$, questo modo è degenere: esiste una coppia di modi con la stessa frequenza, uno proporzionale a $\cos 3\theta$ e uno a $\sin 3\theta$, corrispondenti rispettivamente ai modi 1 e 2 della tabella FEM. La frequenza fondamentale analitica è $f_{\min} = 67.59 \text{ Hz}$, corrispondente ai modi con $i = 3$ onde circolari e $j = 1$ mezza onda assiale.

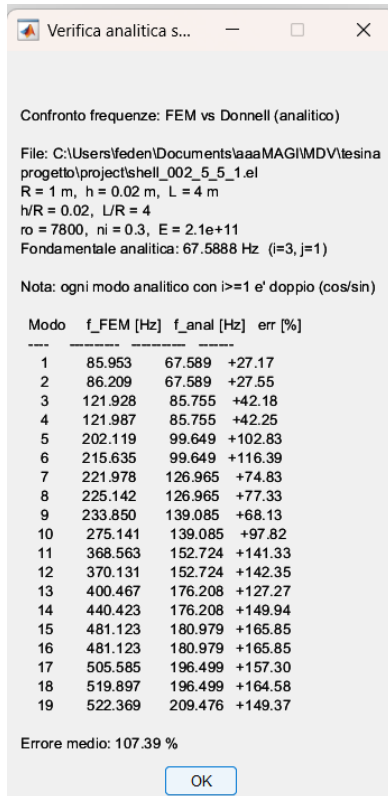
Lo stesso shell è stato discretizzato con sei mesh di densità crescente, identificate dalla convenzione **shell_002_** n_θ n_z n_r , dove n_θ , n_z e n_r sono il numero di elementi in direzione circonferenziale, assiale e radiale rispettivamente. I risultati del confronto sono riportati in Tabella 5.4.

Convergenza al raffinamento. Il trend di convergenza è netto: l'errore medio cala da oltre il 100% con la mesh più grossolana ($5 \times 5 \times 1$) fino al 2.47% con la mesh più fine ($25 \times 20 \times 2$). La riduzione dell'errore al crescere del numero di elementi è monotona e conferma che il codice converge correttamente alla soluzione al raffinamento della mesh.

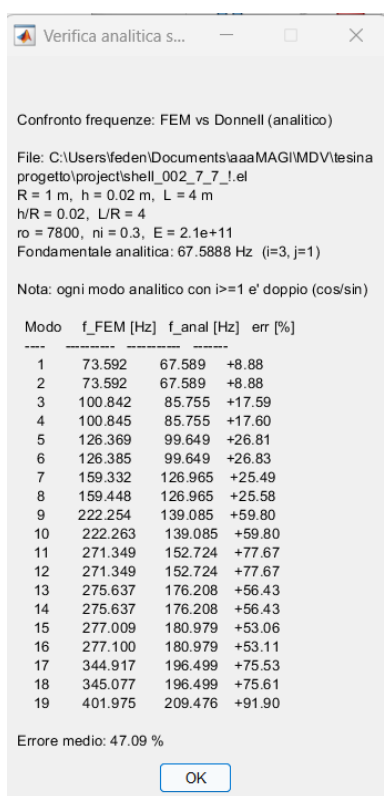
Mesh	$n_\theta \times n_z \times n_r$	Elementi totali	$\bar{\varepsilon}$ [%]
shell_002_5_5_1	$5 \times 5 \times 1$	25	107.39
shell_002_7_7_1	$7 \times 7 \times 1$	49	47.09
shell_002_10_10_1	$10 \times 10 \times 1$	100	13.05
shell_002_12_12_2	$12 \times 12 \times 2$	288	7.27
shell_002_15_15_2	$15 \times 15 \times 2$	450	3.27
shell_002_25_20_2	$25 \times 20 \times 2$	1000	2.47

Tabella 5.4: Convergenza dell'errore medio sulle prime 20 frequenze proprie al raffinamento della mesh.

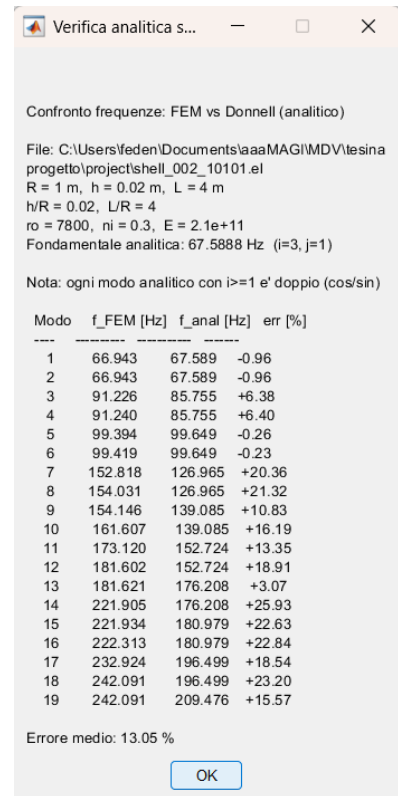
Errore residuo sotto il 3%. Il fatto che anche con 1000 elementi l'errore sia al 2.47% può essere giustificato dal fatto che sia il modello FEM che la teoria di Donnel siano due modelli che hanno delle approssimazioni. Il fatto che comunque raffinando la mesh l'errore converga raggiungendo un 2 – 3% è segno che c'è una correlazione fra la teoria analitica e il codice.



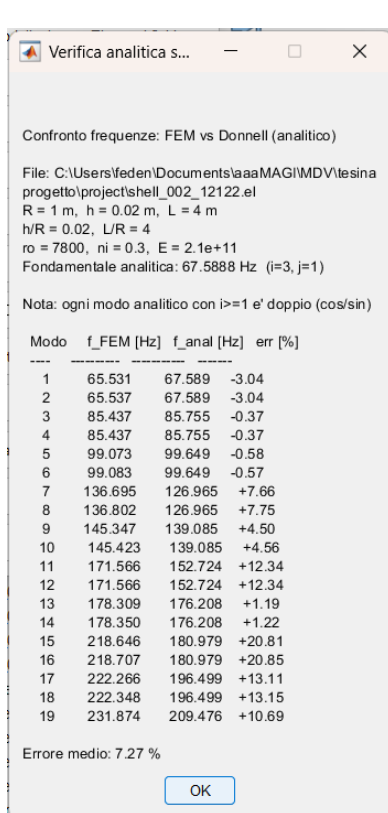
(a) $5 \times 5 \times 1 - \bar{\varepsilon} = 107.39\%$



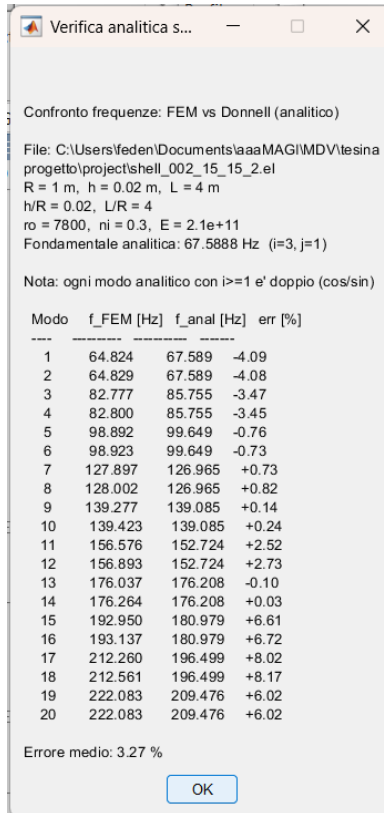
(b) $7 \times 7 \times 1 - \bar{\varepsilon} = 47.09\%$



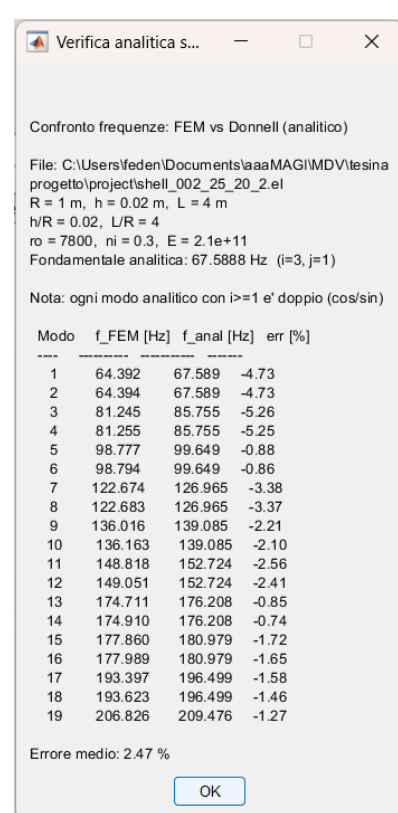
(c) $10 \times 10 \times 1 - \bar{\varepsilon} = 13.05\%$



(d) $12 \times 12 \times 2 - \bar{\varepsilon} = 7.27\%$



(e) $15 \times 15 \times 2 - \bar{\varepsilon} = 3.27\%$



(f) $25 \times 20 \times 2 - \bar{\varepsilon} = 2.47\%$

Figura 5.6: Confronto FEM vs Donnell per le sei mesh al crescere della discretizzazione.

5.5 Verifica della risposta forzata tramite FRF chirp

Per verificare la correttezza dell'implementazione della risposta forzata, si è applicata una forzante di tipo chirp al modello e si è calcolata la funzione di risposta in frequenza (FRF) tramite il rapporto tra gli spettri di Fourier dello spostamento e della forza:

$$H(f) = \frac{\mathcal{F}\{x(t)\}}{\mathcal{F}\{F(t)\}} \quad (5.28)$$

Il segnale chirp copre linearmente la banda $[f_0, f_1] = [0, 800]$ Hz, con frequenza di campionamento $f_c = 4000$ Hz e $N = 20000$ campioni, per una durata totale di $T = N/f_c = 5$ s. La scelta di f_c soddisfa ampiamente il criterio di Nyquist, in quanto $f_c = 4000 \gg 2f_1 = 1600$ Hz, garantendo che l'intero contenuto in frequenza del segnale sia campionato correttamente. La forzante è applicata in direzione x con ampiezza $A = 1000$ N sui nodi della faccia di estremità, come visibile in figura 5.8. In figura 5.7 è riportata la FRF calcolata in un nodo della struttura lungo la direzione y , insieme allo spettro della forzante. Lo spettro della forzante chirp risulta sostanzialmente piatto nella banda $[0, 800]$ Hz, confermando che tutte le frequenze del range di interesse sono eccitate con energia uniforme. In corrispondenza di ciascuna frequenza naturale del sistema si osserva chiaramente un picco nella FRF, il che conferma la correttezza del calcolo: il solver identifica correttamente i modi di vibrare della struttura e la funzione di risposta in frequenza è impostata correttamente.

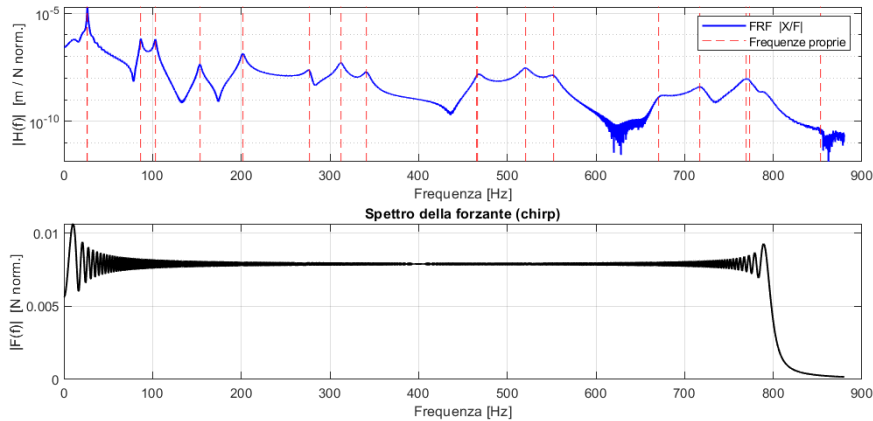


Figura 5.7: FRF calcolata per il nodo selezionato (direzione y). Le linee tratteggiate rosse indicano le frequenze proprie del sistema. Ogni picco dell'ampiezza corrisponde a una frequenza naturale.

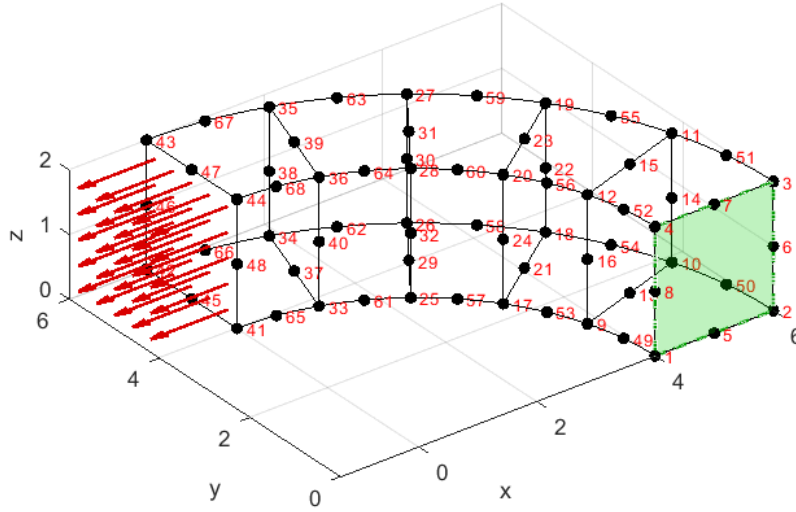


Figura 5.8: Modello FEM con visualizzazione delle condizioni al contorno: vincolo di appoggio semplice (faccia verde) e forzante chirp applicata ai nodi della faccia opposta (freccie rosse).

6 Conclusioni

Nel presente lavoro è stato sviluppato e verificato un codice MATLAB per l'analisi modale e dinamica di strutture tridimensionali mediante elementi finiti solidi isoparametrici Hexa20. Il codice consente di assemblare le matrici globali di rigidezza e massa, risolvere il problema agli autovalori, calcolare la risposta forzata tramite approccio strutturale e modale, e visualizzare sia la mesh che le forme modali.

Le verifiche numeriche condotte hanno confermato la correttezza dell'implementazione: la conservazione della massa totale è stata verificata con un errore relativo dello 0.003%, la matrice di rigidezza è risultata semidefinita positiva a livello di precisione macchina, le frequenze proprie sono risultate invarianti per rotazione rigida del modello, e il confronto con la teoria analitica di Donnell per gusci cilindrici ha mostrato una convergenza monotona dell'errore medio fino al 2.47% con 1000 elementi. La verifica tramite FRF chirp ha infine confermato la consistenza interna del solver ODE con il problema agli autovalori.

Nonostante i risultati ottenuti, il codice presenta alcune limitazioni intrinseche che è opportuno evidenziare.

Tipologia di elemento. Il codice supporta esclusivamente l'elemento solido isoparametrico Hexa20. Non sono implementate tipologie alternative come elementi tetraedrici, prismatici, o elementi di guscio dedicati (shell elements), che sarebbero più efficienti per strutture sottili o geometrie complesse.

Assenza di un meshatore interno. Il programma legge modelli già discretizzati dal file `.el`: non è possibile partire da una geometria CAD arbitraria e generare automaticamente la mesh. L'unica geometria generabile internamente è il guscio cilindrico tramite

il bottone *Crea Shell*. Per qualsiasi altra geometria è necessario produrre il file `.el` esternamente.

Solo analisi dinamiche lineari. Il codice è limitato all'analisi modale libera non smorzata e alla risposta forzata non smorzata (o con smorzamento proporzionale nel metodo modale). Non sono implementate analisi statiche, analisi non lineari geometriche o materiali, né analisi con smorzamento strutturale generico (smorzamento non proporzionale).

Assenza di post-processing tensionale. Il codice calcola esclusivamente il campo di spostamento e le forme modali. Non è previsto il calcolo e la visualizzazione del campo di tensione e di deformazione, che sarebbero indispensabili per una progettazione strutturale completa.

Efficienza computazionale. Il codice è implementato in MATLAB, un linguaggio interpretato ad alto livello. Sebbene MATLAB sfrutti routine ottimizzate per l'algebra lineare sparsa (in particolare per la fattorizzazione LU e il solutore agli autovalori `eigs`), il ciclo di assemblaggio delle matrici elementari rimane in gran parte interpretato. Per modelli di grandi dimensioni (dell'ordine di 10^5 – 10^6 gradi di libertà), le prestazioni risulterebbero proibitive rispetto a codici compilati in C++ o Fortran. Parallelamente, l'integrazione temporale delle equazioni del moto scala con $2n_{\text{dof}}$, rendendo il metodo diretto inapplicabile a modelli con molti gradi di libertà senza una drastica riduzione modale.

Metodo della penalità per i vincoli. I vincoli elastici superficiali sono imposti tramite il metodo della penalità, che richiede una scelta accurata della rigidità k_{pen} : un valore troppo basso non vincola efficacemente la struttura, mentre un valore troppo alto degrada il condizionamento numerico della matrice $\mathbf{K}$, con possibili effetti negativi sulla precisione della soluzione agli autovalori.

Geometria del generatore di shell. Il generatore di mesh è limitato a gusci cilindrici circolari a sezione costante. Non sono supportate geometrie a sezione variabile, gusci conici, toroidali o di forma generale.

In prospettiva, gli sviluppi futuri potrebbero riguardare l'estensione ad altre tipologie di elementi (tetraedri, prismi, shell), l'implementazione di un modulo di post-processing tensionale, l'introduzione di smorzamento non proporzionale e l'ottimizzazione delle performance tramite vettorizzazione delle operazioni di assemblaggio o migrazione verso un linguaggio compilato.

Riferimenti bibliografici

- [1] R. D. Blevins, *Formulas for Natural Frequency and Mode Shape*, Van Nostrand Reinhold, New York, 1979.
- [2] G. Dhatt, G. Touzot, *The Finite Element Method Displayed*, John Wiley & Sons, Chichester, 1984.
- [3] R. D. Cook, D. S. Malkus, M. E. Plesha, *Concepts and Applications of Finite Element Analysis*, 3^a ed., John Wiley & Sons, New York, 1989.
- [4] G. Catania, *Appunti delle lezioni di Laboratorio di meccanica delle vibrazioni e sperimentazione sui sistemi meccanici M e Meccanica delle vibrazioni M*, DIN – Dipartimento di Ingegneria Industriale, Università di Bologna, Italia.

A Codice MATLAB completo

```
1 close all hidden
2 clc
3 clear all
4 fig = 0;
5 mydir = './modelli';
6 s = [];
7 stit = 'Modellazione_a_Elementi_finiti';
8 vs = {'Leggi_modello', ...
9       'Plotta_modello', ...
10      'Plotta_modi_propri',...
11      'Verifica_analitica_shell',...
12      'Crea_Shell',...
13      'Calcola_Risposta_Forzata',...
14      'Plotta_risposta_forzata_con_chirp',...
15      'Ruota_modello', ...
16      'Cambia_numero_Gauss_point',...
17      'Chiudi_Finestre',...
18      'Esci'};
19 ripeti = 1;
20 while ripeti
21     scelta = menu(stit, vs);
22     switch scelta
23         case 1
24             s = leggi_modello(mydir);
25             ng = get_Ngauss();
26             s = crea_modello_fem(s,ng);
27             s = calcola_modello_modale(s);
28         case 2
29             if isempty(s)
30                 error_window('Devi_prima_leggere_il_modello.', 'logo.jpg');
31             else
32                 fig = various_plot(s,fig);
33             end
34         case 3
35             if isempty(s)
36                 error_window('Devi_prima_leggere_il_modello.', 'logo.jpg');
37             else
38                 fig = plotta_modi(s,fig);
39             end
40         case 4
41             if isempty(s)
42                 error_window('Devi_prima_leggere_il_modello.', 'logo.jpg');
43             elseif ~isfield(s,'file') || ~contains(lower(s.file), 'shell')
44                 print_window( ...
45                     {'Il_modello_che_stai_esaminando_non_e' 'uno_shell.', ...
46                      '' , ...
47                      'La_verifica_analitica(Blevins,Donnell)', ...
48                      'e' 'applicabile_solo_a_modelli_il_cui_nome', ...
49                      'file_contiene"shell".'}, ...
50                      'Verifica_non_applicabile');
51             else
52                 confronta_freq_shell(s, s.freq, s.file);
53             end
54         case 5
55             R_sh = query_double_d('Raggio_medio_R[m]', 0.01, 100, 1.0);
56             h_sh = query_double_d('Spessore_h[m]', 1e-4, R_sh, 0.05);
57             L_sh = query_double_d('Lunghezza_L[m]', 0.01, 1000, 4.0);
58             nc_sh = query_integer_d('N_elementi_in_circonferenza', 2, 200, 12);
59             na_sh = query_integer_d('N_elementi_in_direzione_assiale', 1, 200, 8);
60             nt_sh = query_integer_d('N_elementi_nello_spessore', 1, 10, 1);
61             ro_sh = query_double_d('Densita' '[kg/m^3]', 1, 30000, 7800);
62             ni_sh = query_double_d('Coefficiente_di_Poisson', 0, 0.5, 0.3);
63             E_sh = query_double_d('Modulo_di_Young[Pa]', 1e6, 1e13, 2.1e11);
64             kp_sh = query_double_d('Rigidezza_penalita' '[N/m^3]', 0, 1e30, 1e6*
65                                     E_sh/h_sh);
66             nome_default = 'shell_ss.el';
67             risposta = inputdlg( ...
68                 {'Nome_file_el(deve_contenere"shell"per_la_verifica_analitica):'},
69                 ...
70                 'Salva_modello', 1, {nome_default});
```

```

69         if isempty(risposta)
70             error('!!');
71         end
72         nome_file_sh = char(risposta);
73         if ~endsWith(nome_file_sh, '.el')
74             nome_file_sh = [nome_file_sh '.el'];
75         end
76         if ~contains(lower(nome_file_sh), 'shell')
77             print_window( ...
78                 {'Attenzione: il nome file non contiene "shell".', ...
79                  'La verifica analitica non sara' 'disponibile.'}, ...
80                 'Avviso');
81         end
82         s = crea_modello_shell(R_sh, h_sh, L_sh, nc_sh, na_sh, nt_sh, ...
83             ro_sh, ni_sh, E_sh, kp_sh, nome_file_sh);
84         print_window( ...
85             {sprintf('Shell creato: %s', nome_file_sh), ...
86               }, ...
87             sprintf('Nodi: %d, Elementi: %d, Vincoli: %d', s.nd, s.nel, s.nv), ...
88             sprintf('R= %.4g, h= %.4g, L= %.4g', R_sh, h_sh, L_sh), ...
89             sprintf('Mesh: %dx%dx%d', nc_sh, na_sh, nt_sh)}, ...
90             'Shell creato');
91     case 6
92         [risp, fig] = risposta_forzata(s, fig);
93     case 7
94         if isempty(risp)
95             error_window('Devi prima calcolare risposta forzata.', 'logo.jpg');
96         else
97             verifica_chirp_frf(risp, s)
98         end
99     case 8
100         if isempty(s)
101             error_window('Devi prima leggere il modello.', 'logo.jpg');
102         else
103             s = rotate_model(s, ng);
104         end
105     case 9
106         ng = get_Ngauss(true);
107     case 10
108         close all hidden
109     case 11
110         ripeti = 0;
111     case 0
112         ripeti = 0;
113     otherwise
114         msg = sprintf('Opzione %d non implementata', scelta);
115         error_window(msg, 'logo.jpg');
116     end
117 end
118
119 function [x,w]= gauleg(x1,x2,n)
120
121     DMACHEPS= 1.0e-16;
122     m=(n+1)/2;
123     xm=0.5*(x2+x1);
124     x1=0.5*(x2-x1);
125     for i=1:m
126         z=cos(pi*(i-0.25)/(n+0.5));
127         ancora= 1;
128         while ancora
129             p1=1.0;
130             p2=0.0;
131             for j=1:n
132                 p3=p2;
133                 p2=p1;
134                 p1=((2.0*j-1.0)*z*p2-(j-1.0)*p3)/j;
135             end
136             pp=n*(z*p1-p2)/(z*z-1.0);
137             z1=z;
138             z=z1-p1/pp;
139             if abs(z-z1) <= DMACHEPS
140                 ancora= 0;
141             end
142         end
143     end
144     x=x1+xm*(1-x1);
145     w=w*(xm-x1);
146     w=w/2;
147     w=w/(x2-x1);
148 end

```

```

142         end
143     end
144     x(i)=xm-xl*z;
145     x(n+1-i)=xm+xl*z;
146     w(i)=2.0*xl/((1.0-z*z)*pp*pp);
147     w(n+1-i)=w(i);
148 end
149
150
151 function [st,file] = leggi_modello(loc_dir)
152
153     st.ndxel= 20;  st.ndofsxel= st.ndxel*3;
154     [file,~]= query_file(loc_dir, '*.el', 'r', 'Open*.el_file');
155     st.file=file;
156     pf = fopen(file, 'r');
157     st.nd = fscanf(pf, '%d', 1);
158     st.nel = fscanf(pf, '%d', 1);
159     st.nv = fscanf(pf, '%d', 1);
160     st.nf = fscanf(pf, '%d', 1);
161
162     for i=1:st.nd
163         p(i).x= fscanf(pf, '%f', 1);
164         p(i).y= fscanf(pf, '%f', 1);
165         p(i).z= fscanf(pf, '%f', 1);
166     end
167     st.p=p; clear p;
168
169     for i=1:st.nel
170         el(i).indx= fscanf(pf, '%d', st.ndxel);
171         el(i).idofs= el2idofs(el(i).indx, st.ndofsxel);
172         el(i).ro= fscanf(pf, '%f', 1);
173         el(i).ni= fscanf(pf, '%f', 1);
174         el(i).E= fscanf(pf, '%f', 1);
175     end
176     st.el=el; clear el;
177
178     if st.nv>0
179         for i=1:st.nv
180             vinc(i).el= fscanf(pf, '%d', 1);
181             vinc(i).u= fscanf(pf, '%f', 2);
182             vinc(i).v= fscanf(pf, '%f', 2);
183             vinc(i).w= fscanf(pf, '%f', 2);
184             vinc(i).k= fscanf(pf, '%f', 3);
185         end
186         st.vinc= vinc; clear vinc
187     end
188     if st.nf>0
189         for i=1:st.nf
190             F(i).el= fscanf(pf, '%d', 1);
191             F(i).u= fscanf(pf, '%f', 2);
192             F(i).v= fscanf(pf, '%f', 2);
193             F(i).w= fscanf(pf, '%f', 2);
194             F(i).p= fscanf(pf, '%f', 3);
195         end
196         st.F= F; clear F;
197     end
198     fclose(pf);
199 end
200
201 function v= el2idofs(vnod,n)
202     v=zeros(n,1);  nn=length(vnod);
203     k=0;
204     for i=1:nn
205         ndi= vnod(i);  ii= 3*(ndi-1);
206         for j=1:3
207             k=k+1;  v(k)= ii+j;
208         end
209     end
210 end
211
212
213 function N = get_Ngauss(reset)
214

```

```

215 persistent N_saved
216
217 if nargin < 1
218     reset = false;
219 end
220
221 if reset
222     N_saved = [];
223 end
224
225 if isempty(N_saved)
226
227     vs = 'Con quanti punti di Gauss vuoi integrare?';
228     xmin = 1;
229     xmax = 5;
230     xdef = 3;
231
232     N_saved = query_integer_d(vs, xmin, xmax, xdef);
233     if isempty(N_saved), return; end
234
235 end
236
237 N = N_saved;
238
239 end
240
241
242 function D = matxD(v,E)
243
244 zer=zeros(3); k=(1-2*v)/2;
245 vec=[k,k,k];
246 dg=diag(vec);
247 mat=[1-v v v; v 1-v v; v v 1-v];
248 mat2=[mat zer; zer dg];
249 a=E/((1+v)*(1-2*v));
250 D=a*mat2;
251 end
252
253
254 function [Jprod,N] = crea_N_dN_new(xi_gp, eta_gp, zita_gp)
255
256 Jprod = zeros(3,20);
257 N      = zeros(3,60);
258
259 N1      = 1/8*(1-xi_gp)*(1-eta_gp)*(1-zita_gp)*(-2-xi_gp-eta_gp-zita_gp);
260 dN1x    = -1/8*(1-eta_gp)*(1-zita_gp)*(-1-2*xi_gp-eta_gp-zita_gp);
261 dN1e    = -1/8*(1-xi_gp)*(1-zita_gp)*(-1-xi_gp-2*eta_gp-zita_gp);
262 dN1z    = -1/8*(1-eta_gp)*(1-xi_gp)*(-1-xi_gp-eta_gp-2*zita_gp);
263
264 N2      = 1/4*(1-xi_gp^2)*(1-eta_gp)*(1-zita_gp);
265 dN2x    = -1/2*xi_gp*(1-eta_gp)*(1-zita_gp);
266 dN2e    = -1/4*(1-xi_gp^2)*(1-zita_gp);
267 dN2z    = -1/4*(1-xi_gp^2)*(1-eta_gp);
268
269 N3      = 1/8*(1+xi_gp)*(1-eta_gp)*(1-zita_gp)*(-2+xi_gp-eta_gp-zita_gp);
270 dN3x    = 1/8*(1-eta_gp)*(1-zita_gp)*(-1+2*xi_gp-eta_gp-zita_gp);
271 dN3e    = -1/8*(1+xi_gp)*(1-zita_gp)*(-1+xi_gp-2*eta_gp-zita_gp);
272 dN3z    = -1/8*(1-eta_gp)*(1+xi_gp)*(-1+xi_gp-eta_gp-2*zita_gp);
273
274 N4      = 1/4*(1-eta_gp^2)*(1+xi_gp)*(1-zita_gp);
275 dN4x    = 1/4*(1-eta_gp^2)*(1-zita_gp);
276 dN4e    = -1/2*eta_gp*(1+xi_gp)*(1-zita_gp);
277 dN4z    = -1/4*(1-eta_gp^2)*(1+xi_gp);
278
279 N5      = 1/8*(1+xi_gp)*(1+eta_gp)*(1-zita_gp)*(-2+xi_gp+eta_gp-zita_gp);
280 dN5x    = 1/8*(1+eta_gp)*(1-zita_gp)*(-1+2*xi_gp+eta_gp-zita_gp);
281 dN5e    = 1/8*(1+xi_gp)*(1-zita_gp)*(-1+xi_gp+2*eta_gp-zita_gp);
282 dN5z    = -1/8*(1+eta_gp)*(1+xi_gp)*(-1+xi_gp+eta_gp-2*zita_gp);
283
284 N6      = 1/4*(1-xi_gp^2)*(1+eta_gp)*(1-zita_gp);
285 dN6x    = -1/2*xi_gp*(1+eta_gp)*(1-zita_gp);
286 dN6e    = 1/4*(1-xi_gp^2)*(1-zita_gp);
287 dN6z    = -1/4*(1-xi_gp^2)*(1+eta_gp);

```

```

288
289 N7 = 1/8*(1-xi_gp)*(1+eta_gp)*(1-zita_gp)*(-2-xi_gp+eta_gp-zita_gp);
290 dN7x = -1/8*(1+eta_gp)*(1-zita_gp)*(-1-2*xi_gp+eta_gp-zita_gp);
291 dN7e = 1/8*(1-xi_gp)*(1-zita_gp)*(-1-xi_gp+2*eta_gp-zita_gp);
292 dN7z = -1/8*(1+eta_gp)*(1-xi_gp)*(-1-xi_gp+eta_gp-2*zita_gp);
293
294 N8 = 1/4*(1-eta_gp^2)*(1-xi_gp)*(1-zita_gp);
295 dN8x = -1/4*(1-eta_gp^2)*(1-zita_gp);
296 dN8e = -1/2*eta_gp*(1-xi_gp)*(1-zita_gp);
297 dN8z = -1/4*(1-eta_gp^2)*(1-xi_gp);
298
299 N9 = 1/4*(1-zita_gp^2)*(1-xi_gp)*(1-eta_gp);
300 dN9x = -1/4*(1-zita_gp^2)*(1-eta_gp);
301 dN9e = -1/4*(1-zita_gp^2)*(1-xi_gp);
302 dN9z = -1/2*zita_gp*(1-xi_gp)*(1-eta_gp);
303
304 N10 = 1/4*(1-zita_gp^2)*(1+xi_gp)*(1-eta_gp);
305 dN10x = 1/4*(1-zita_gp^2)*(1-eta_gp);
306 dN10e = -1/4*(1-zita_gp^2)*(1+xi_gp);
307 dN10z = -1/2*zita_gp*(1+xi_gp)*(1-eta_gp);
308
309 N11 = 1/4*(1-zita_gp^2)*(1+xi_gp)*(1+eta_gp);
310 dN11x = 1/4*(1-zita_gp^2)*(1+eta_gp);
311 dN11e = 1/4*(1-zita_gp^2)*(1+xi_gp);
312 dN11z = -1/2*zita_gp*(1+xi_gp)*(1+eta_gp);
313
314 N12 = 1/4*(1-zita_gp^2)*(1-xi_gp)*(1+eta_gp);
315 dN12x = -1/4*(1-zita_gp^2)*(1+eta_gp);
316 dN12e = 1/4*(1-zita_gp^2)*(1-xi_gp);
317 dN12z = -1/2*zita_gp*(1-xi_gp)*(1+eta_gp);
318
319 N13 = 1/8*(1-xi_gp)*(1-eta_gp)*(1+zita_gp)*(-2-xi_gp-eta_gp+zita_gp);
320 dN13x = -1/8*(1-eta_gp)*(1+zita_gp)*(-1-2*xi_gp-eta_gp+zita_gp);
321 dN13e = -1/8*(1-xi_gp)*(1+zita_gp)*(-1-xi_gp-2*eta_gp+zita_gp);
322 dN13z = 1/8*(1-eta_gp)*(1-xi_gp)*(-1-xi_gp-eta_gp+2*zita_gp);
323
324 N14 = 1/4*(1-xi_gp^2)*(1-eta_gp)*(1+zita_gp);
325 dN14x = -1/2*xi_gp*(1-eta_gp)*(1+zita_gp);
326 dN14e = -1/4*(1-xi_gp^2)*(1+zita_gp);
327 dN14z = 1/4*(1-xi_gp^2)*(1-eta_gp);
328
329 N15 = 1/8*(1+xi_gp)*(1-eta_gp)*(1+zita_gp)*(-2+xi_gp-eta_gp+zita_gp);
330 dN15x = 1/8*(1-eta_gp)*(1+zita_gp)*(-1+2*xi_gp-eta_gp+zita_gp);
331 dN15e = -1/8*(1+xi_gp)*(1+zita_gp)*(-1+xi_gp-2*eta_gp+zita_gp);
332 dN15z = 1/8*(1-eta_gp)*(1+xi_gp)*(-1+xi_gp-eta_gp+2*zita_gp);
333
334 N16 = 1/4*(1-eta_gp^2)*(1+xi_gp)*(1+zita_gp);
335 dN16x = 1/4*(1-eta_gp^2)*(1+zita_gp);
336 dN16e = -1/2*eta_gp*(1+xi_gp)*(1+zita_gp);
337 dN16z = 1/4*(1-eta_gp^2)*(1+xi_gp);
338
339 N17 = 1/8*(1+xi_gp)*(1+eta_gp)*(1+zita_gp)*(-2+xi_gp+eta_gp+zita_gp);
340 dN17x = 1/8*(1+eta_gp)*(1+zita_gp)*(-1+2*xi_gp+eta_gp+zita_gp);
341 dN17e = 1/8*(1+xi_gp)*(1+zita_gp)*(-1+xi_gp+2*eta_gp+zita_gp);
342 dN17z = 1/8*(1+eta_gp)*(1+xi_gp)*(-1+xi_gp+eta_gp+2*zita_gp);
343
344 N18 = 1/4*(1-xi_gp^2)*(1+eta_gp)*(1+zita_gp);
345 dN18x = -1/2*xi_gp*(1+eta_gp)*(1+zita_gp);
346 dN18e = 1/4*(1-xi_gp^2)*(1+zita_gp);
347 dN18z = 1/4*(1-xi_gp^2)*(1+eta_gp);
348
349 N19 = 1/8*(1-xi_gp)*(1+eta_gp)*(1+zita_gp)*(-2-xi_gp+eta_gp+zita_gp);
350 dN19x = -1/8*(1+eta_gp)*(1+zita_gp)*(-1-2*xi_gp+eta_gp+zita_gp);
351 dN19e = 1/8*(1-xi_gp)*(1+zita_gp)*(-1-xi_gp+2*eta_gp+zita_gp);
352 dN19z = 1/8*(1+eta_gp)*(1-xi_gp)*(-1-xi_gp+eta_gp+2*zita_gp);
353
354 N20 = 1/4*(1-eta_gp^2)*(1-xi_gp)*(1+zita_gp);
355 dN20x = -1/4*(1-eta_gp^2)*(1+zita_gp);
356 dN20e = -1/2*eta_gp*(1-xi_gp)*(1+zita_gp);
357 dN20z = 1/4*(1-eta_gp^2)*(1-xi_gp);
358
359 Nvec = [N1,N2,N3,N4,N5,N6,N7,N8,N9,N10,N11,N12,N13,N14,N15,N16,N17,N18,N19,N20];
360 for q = 1:20

```

```

361     N(1, 3*q-2) = Nvec(q);
362     N(2, 3*q-1) = Nvec(q);
363     N(3, 3*q ) = Nvec(q);
364 end
365
366 Jprod(1,:) = [dN1x, dN2x, dN3x, dN4x, dN5x, dN6x, dN7x, dN8x, dN9x, dN10x, ...
367              dN11x,dN12x,dN13x,dN14x,dN15x,dN16x,dN17x,dN18x,dN19x,dN20x];
368 Jprod(2,:) = [dN1e, dN2e, dN3e, dN4e, dN5e, dN6e, dN7e, dN8e, dN9e, dN10e, ...
369              dN11e,dN12e,dN13e,dN14e,dN15e,dN16e,dN17e,dN18e,dN19e,dN20e];
370 Jprod(3,:) = [dN1z, dN2z, dN3z, dN4z, dN5z, dN6z, dN7z, dN8z, dN9z, dN10z, ...
371              dN11z,dN12z,dN13z,dN14z,dN15z,dN16z,dN17z,dN18z,dN19z,dN20z];
372
373 end
374
375
376 function [B] = Assembla_B(Js,Jprod,ndxel)
377
378 dN_glob=(Js)^(-1)*Jprod;
379 B = zeros(6,60);
380 for q=1:ndxel
381     B(1,3*q-2)=dN_glob(1,q);
382     B(2,3*q-1)=dN_glob(2,q);
383     B(3,3*q)=dN_glob(3,q);
384     B(4,3*q-2)=dN_glob(2,q);
385     B(4,3*q-1)=dN_glob(1,q);
386     B(5,3*q-1)=dN_glob(3,q);
387     B(5,3*q)=dN_glob(2,q);
388     B(6,3*q-2)=dN_glob(3,q);
389     B(6,3*q)=dN_glob(1,q);
390 end
391 end
392
393
394 function [dK,dM] = add_gauss_contribution(ui,vj,wk,coord,D,ro,ndxel)
395
396 [Jprod,N] = crea_N_dN_new(ui,vj,wk);
397
398 Js = Jprod * coord;
399
400 detJ = det(Js);
401 if detJ <= 0
402     fprintf('\nERRORE: detJ=%%.16e\n', detJ);
403     fprintf('Gauss point: xi=%%.6f, eta=%%.6f, zita=%%.6f\n', ui, vj, wk);
404     disp('Coordinate elemento:');
405     disp(coord);
406     error('Jacobiano negativo o nullo: controlla ordine nodi Hexa20');
407 end
408
409 B = Assembla_B(Js,Jprod,ndxel);
410
411 dK = B' * D * B * detJ;
412 dM = ro * N' * N * detJ;
413
414 end
415
416
417 function A = assemblaggio_matrici(A,Ai,indx)
418
419 n = length(indx);
420 for i=1:n
421     ii= indx(i);
422     for j=1:n
423         jj= indx(j);
424         A(ii,jj) = A(ii,jj) + Ai(i,j);
425     end
426 end
427 end
428
429
430 function s=crea_modello_fem(s,ng)
431
432 [xg,wg]= gauleg(-1.,1.,ng);
433 s.N= s.nd*3;

```

```

434 s.M = sparse(s.N, s.N);
435 s.K = sparse(s.N, s.N);
436 coord=zeros(20,3);
437 Kq=sparse(60,60); Mq=sparse(60,60);
438 for q=1:s.nel
439     Kq=sparse(60,60); Mq=sparse(60,60);
440     eq= s.el(q);
441     indx=eq.indx;      idofs= eq.idofs;
442     coord(:,1) = [s.p(indx).x]';
443     coord(:,2) = [s.p(indx).y]';
444     coord(:,3) = [s.p(indx).z]';
445     Dq=matxD(eq.ni,eq.E);
446     for i=1:ng
447         ui= xg(i); wui= wg(i);
448         for j=1:ng
449             vj= xg(j); wvj= wg(j);
450             for k=1:ng
451                 wk= xg(k); wwkw= wg(k);
452                 [dK,dM] = add_gauss_contribution(ui,vj,wk,coord,Dq,eq.ro,s.ndxel);
453                 Kq= Kq+wui*wvj*wwkw*dK;
454                 Mq= Mq+wui*wvj*wwkw*dM;
455                 clear wk; clear wwkw;
456             end
457             clear vj; clear wvj;
458         end
459         clear ui; clear wui;
460     end
461
462     s.K= assemblaggio_matrici(s.K,Kq,idofs);
463     s.M= assemblaggio_matrici(s.M,Mq,idofs);
464 end
465
466 if s.nv>0
467     s=assembla_vinc(s,ng);
468 end
469
470 if s.nf>0
471     s=assembla_forza(s,ng);
472 end
473
474 end
475
476
477 function s=assembla_vinc(s,ng)
478
479 for i=1:s.nv
480     vinc=s.vinc(i);
481     eq=s.el(vinc.el);
482     indx=eq.indx;
483     idofs=eq.idofs;
484     coord(:,1) = [s.p(indx).x]';
485     coord(:,2) = [s.p(indx).y]';
486     coord(:,3) = [s.p(indx).z]';
487
488     Kv=crea_vinc_elastico(vinc,coord,ng,s.ndxel);
489
490     s.K=assemblaggio_matrici(s.K,Kv,idofs);
491 end
492
493 end
494
495
496 function Kv = crea_vinc_elastico(vinc,coord,ng,ndxel)
497
498 Kv = zeros(ndxel*3,ndxel*3);
499
500 toll = 1.e-12;
501
502 u_fisso = abs(vinc.u(2)-vinc.u(1)) < toll;
503 v_fisso = abs(vinc.v(2)-vinc.v(1)) < toll;
504 w_fisso = abs(vinc.w(2)-vinc.w(1)) < toll;
505
506 n_fissi = u_fisso + v_fisso + w_fisso;

```

```

507
508     if n_fissi ~= 1
509         error('Il vincolo elastico deve essere applicato su una faccia: una sola coordinata naturale deve essere fissa.');
```

510

```

511     end
512     Ks = diag(vinc.k);
513
514     if u_fisso
515
516         u0 = vinc.u(1);
517
518         [xv,wv] = gauleg(vinc.v(1),vinc.v(2),ng);
519         [xw,ww] = gauleg(vinc.w(1),vinc.w(2),ng);
520
521         for j = 1:ng
522             vj = xv(j);
523             wvj = wv(j);
524
525             for k = 1:ng
526                 wk = xw(k);
527                 ww = ww(k);
528
529                 dKv = add_gauss_contribution_vincolo(u0,vj,wk,coord,Ks,'u');
530
531                 Kv = Kv + wvj*ww*dKv;
532
533             end
534         end
535
536     elseif v_fisso
537
538         v0 = vinc.v(1);
539
540         [xu,wu] = gauleg(vinc.u(1),vinc.u(2),ng);
541         [xw,ww] = gauleg(vinc.w(1),vinc.w(2),ng);
542
543         for i = 1:ng
544             ui = xu(i);
545             wui = wu(i);
546
547             for k = 1:ng
548                 wk = xw(k);
549                 ww = ww(k);
550
551                 dKv = add_gauss_contribution_vincolo(ui,v0,wk,coord,Ks,'v');
552
553                 Kv = Kv + wui*ww*dKv;
554
555             end
556         end
557
558     elseif w_fisso
559
560         w0 = vinc.w(1);
561
562         [xu,wu] = gauleg(vinc.u(1),vinc.u(2),ng);
563         [xv,wv] = gauleg(vinc.v(1),vinc.v(2),ng);
564
565         for i = 1:ng
566             ui = xu(i);
567             wui = wu(i);
568
569             for j = 1:ng
570                 vj = xv(j);
571                 wvj = wv(j);
572
573                 dKv = add_gauss_contribution_vincolo(ui,vj,w0,coord,Ks,'w');
574
575                 Kv = Kv + wui*wvj*dKv;
576
577             end
578         end

```

```

579
580     end
581
582 end
583
584
585 function dKv = add_gauss_contribution_vincolo(ui,vj,wk,coord,Ks,faccia)
586
587 [Jprod,N] = crea_N_dN_new(ui,vj,wk);
588
589 Js = Jprod * coord;
590
591 dx_du = Js(1,:);
592 dx_dv = Js(2,:);
593 dx_dw = Js(3,:);
594
595 switch faccia
596
597     case 'u'
598         dGamma = norm(cross(dx_dv,dx_dw));
599
600     case 'v'
601         dGamma = norm(cross(dx_du,dx_dw));
602
603     case 'w'
604         dGamma = norm(cross(dx_du,dx_dv));
605
606     otherwise
607         error('Faccia non riconosciuta.');

```

```

651
652 if u_fisso
653     u0=forza.u(1);
654     for i=1:length(xg)
655         vi=xg(i);
656         wvi=wg(i);
657         for j=1:length(xg)
658             vj=xg(j);
659             wvj=wg(j);
660             dfe=forza_gauss_superficie(u0,vi,vj,coord,t,'u');
661             fe=fe+dfe*wvj*wvi;
662         end
663     end
664 elseif v_fisso
665     v0=forza.v(1);
666     for i=1:length(xg)
667         vi=xg(i);
668         wvi=wg(i);
669         for j=length(xg)
670             vj=xg(j);
671             wvj=wg(j);
672             dfe=forza_gauss_superficie(vi,v0,vj,coord,t,'v');
673             fe=fe+dfe*wvj*wvi;
674         end
675     end
676 elseif w_fisso
677     w0=forza.w(1);
678     for i=1:length(xg)
679         vi=xg(i);
680         wvi=wg(i);
681         for j=length(xg)
682             vj=xg(j);
683             wvj=wg(j);
684             dfe=forza_gauss_superficie(vi,vj,w0,coord,t,'w');
685             fe=fe+dfe*wvj*wvi;
686         end
687     end
688 end
689
690 end
691
692 function dfe=forza_gauss_superficie(u,v,w,coord,t,tiposup)
693
694 [Jprod,N]=crea_N_dN_new(u,v,w);
695 Js=Jprod*coord;
696 switch tiposup
697     case 'u'
698         t1=Js(2,:);
699         t2=Js(3,:);
700
701     case 'v'
702         t1=Js(1,:);
703         t2=Js(3,:);
704
705     case 'w'
706         t1=Js(1,:);
707         t2=Js(2,:);
708 end
709 Jsup=norm(cross(t1,t2));
710 dfe=N.*t*Jsup;
711 end
712
713
714 function s = calcola_modello_modale(s)
715
716     n_modi = query_integer_d('Numero modi da calcolare', 1, 200, 20);
717     if isempty(n_modi), return; end
718     [Vs,D] = eigs(s.K, s.M, n_modi, 'smallestabs');
719     Vs = real(Vs);
720     D = real(D);
721     lambda = condiziona(diag(D));
722     s.freq = sqrt(lambda)/(2*pi);
723     s.freq(s.freq < 0.01) = 0;

```

```

724     [s.freq,idx]=sort(s.freq);
725     s.V=Vs(:,idx);
726
727 end
728
729 function x= condiziona(x)
730 n= length(x); xmax= max(abs(x));  tol= xmax*1e-16;
731 for i=1:n
732     if x(i)<tol
733         x(i)=0.;
734     end
735 end
736 end
737
738
739 function st = crea_modello_shell(R, h, L, n_circ, n_ax, n_thick, ro, ni, E, k_pen,
    file_el)
740
741     if nargin < 11
742         error('crea_modello_shell: servono 11 argomenti. ');
743     end
744     if n_circ < 2
745         error('n_circ deve essere >= 2 per chiudere il cilindro. ');
746     end
747
748     st.ndxel = 20;
749     st.ndofsxel = st.ndxel*3;
750
751     nat = [ -1 -1 -1;
752            0 -1 -1;
753            1 -1 -1;
754            1 0 -1;
755            1 1 -1;
756            0 1 -1;
757            -1 1 -1;
758            -1 0 -1;
759            -1 -1 0;
760            1 -1 0;
761            1 1 0;
762            -1 1 0;
763            -1 -1 1;
764            0 -1 1;
765            1 -1 1;
766            1 0 1;
767            1 1 1;
768            0 1 1;
769            -1 1 1;
770            -1 0 1];
771
772     NA = 2*n_circ;
773     NB = 2*n_ax;
774     NC = 2*n_thick;
775
776     gid = zeros(NA, NB+1, NC+1);
777     nd = 0;
778     p = struct('x',{},{},'y',{},{},'z',{},{ });
779     for c = 0:NC
780         r = R - h/2 + h*c/NC;
781         for b = 0:NB
782             zax = L*b/NB;
783             for a = 0:NA-1
784                 if mod(a,2)+mod(b,2)+mod(c,2) <= 1
785                     nd = nd + 1;
786                     gid(a+1,b+1,c+1) = nd;
787                     theta = 2*pi*a/NA;
788                     p(nd).x = r*cos(theta);
789                     p(nd).y = r*sin(theta);
790                     p(nd).z = zax;
791                 end
792             end
793         end
794     end
795     st.nd = nd;

```

```

796 st.p = p;
797
798 nel = n_circ*n_ax*n_thick;
799 el = struct('indx',{},'idofs',{},'ro',{},'ni',{},'E',{});
800 e = 0;
801 for et = 0:n_thick-1
802     for eb = 0:n_ax-1
803         for ec = 0:n_circ-1
804             e = e + 1;
805             indx = zeros(20,1);
806             for m = 1:20
807                 a = mod(2*ec + nat(m,1) + 1, NA);
808                 b = 2*eb + nat(m,2) + 1;
809                 c = 2*et + nat(m,3) + 1;
810                 indx(m) = gid(a+1, b+1, c+1);
811             end
812             if any(indx == 0)
813                 error('Connettivita'' non valida, elemento %d.',e);
814             end
815             el(e).indx = indx;
816             el(e).idofs = el2idofs_shell(indx, st.ndofsxel);
817             el(e).ro = ro;
818             el(e).ni = ni;
819             el(e).E = E;
820         end
821     end
822 end
823 st.nel = nel;
824 st.el = el;
825
826 if ~isempty(k_pen) && k_pen > 0
827     nv = 0;
828     vinc = struct('el',{},'u',{},'v',{},'w',{},'k',{});
829     for et = 0:n_thick-1
830         for ec = 0:n_circ-1
831             e_z0 = et*n_ax*n_circ + 0*n_circ + ec + 1;
832             nv = nv + 1;
833             vinc(nv).el = e_z0;
834             vinc(nv).u = [-1; 1];
835             vinc(nv).v = [-1; -1];
836             vinc(nv).w = [-1; 1];
837             vinc(nv).k = [k_pen; k_pen; 0];
838             e_zL = et*n_ax*n_circ + (n_ax-1)*n_circ + ec + 1;
839             nv = nv + 1;
840             vinc(nv).el = e_zL;
841             vinc(nv).u = [-1; 1];
842             vinc(nv).v = [1; 1];
843             vinc(nv).w = [-1; 1];
844             vinc(nv).k = [k_pen; k_pen; 0];
845         end
846     end
847     st.nv = nv;
848     st.vinc = vinc;
849 else
850     st.nv = 0;
851 end
852
853 st.nf = 0;
854
855 pf = fopen(file_el,'w');
856 if pf < 0, error('Impossibile aprire %s.',file_el); end
857
858 fprintf(pf,'%d %d %d %d\n', st.nd, st.nel, st.nv, st.nf);
859
860 for i = 1:st.nd
861     fprintf(pf,'% .10g % .10g % .10g\n', p(i).x, p(i).y, p(i).z);
862 end
863
864 for i = 1:st.nel
865     fprintf(pf,'%d ', el(i).indx);
866     fprintf(pf,'\n');
867     fprintf(pf,'% .10g % .10g % .10g\n', el(i).ro, el(i).ni, el(i).E);
868 end

```

```

869
870     if st.nv > 0
871         for i = 1:st.nv
872             fprintf(pf, '%d %10g %10g %10g %10g %10g %10g %10g %10g\n', ...
873                 st.vinc(i).el, ...
874                 st.vinc(i).u(1), st.vinc(i).u(2), ...
875                 st.vinc(i).v(1), st.vinc(i).v(2), ...
876                 st.vinc(i).w(1), st.vinc(i).w(2), ...
877                 st.vinc(i).k(1), st.vinc(i).k(2), st.vinc(i).k(3));
878         end
879     end
880
881     fclose(pf);
882 end
883
884 function v = el2idofs_shell(vnod, n)
885     v = zeros(n,1); nn = length(vnod); k = 0;
886     for i = 1:nn
887         ii = 3*(vnod(i)-1);
888         for j = 1:3
889             k = k+1; v(k) = ii+j;
890         end
891     end
892 end
893
894 function fig = various_plot(s,fig)
895
896     stit = 'Tipologia_Plot';
897
898     vs = {'Mesh', ...
899         'Mesh_+_nodi', ...
900         'Mesh_+_nodi_+_Numerazione_Elementi', ...
901         'Cambia_numero_punti_plottati', ...
902         'Chiudi_Finestre', ...
903         'Esci'};
904
905     ripeti = 1;
906
907     while ripeti
908
909         scelta = menu(stit, vs);
910
911         switch scelta
912
913             case 1
914                 if isempty(s)
915                     error_window('Devi prima leggere il modello.', 'logo.jpg');
916                 else
917                     fig = plotta_modello(s.el,s.p,fig,'k-');
918                     hold on
919                     plotta_facce_vincolate_e_caricate(s);
920                 end
921
922             case 2
923                 if isempty(s)
924                     error_window('Devi prima leggere il modello.', 'logo.jpg');
925                 else
926                     fig = plotta_modello_nodi(s.el,s.p,fig);
927                     hold on
928                     plotta_facce_vincolate_e_caricate(s);
929                 end
930
931             case 3
932                 if isempty(s)
933                     error_window('Devi prima leggere il modello.', 'logo.jpg');
934                 else
935                     fig = plotta_modello_nodi_elementi(s.el,s.p,fig);
936                     hold on
937                     plotta_facce_vincolate_e_caricate(s);
938                 end
939
940             case 4

```

```

942         get_Nplot(true);
943
944         case 5
945             close all hidden
946
947         case 6
948             ripeti=0;
949     end
950 end
951 end
952
953
954 function fig = plotta_modi(s,fig)
955
956     vs='Quante_frequenze_vuoi_stampare?';
957     ancora=1;
958     xmax=min(100,length(s.freq));
959     xmin=1;
960     xdef=10;
961     while ancora
962         n=query_integer_d(vs,xmin,xmax,xdef);
963         ancora=0;
964     end
965     if isempty(n)
966         return
967     end
968     stit=sprintf('Prime%d_Frequenze_Proprie',n);
969     for i=1:n
970         str{i}=sprintf('%d_Freq_Propria:%.4e[Hz]', i, s.freq(i));
971     end
972     print_window(str,stit);
973     fig = 0;
974
975     while ask_yes('Vuoi_plottare_un_modo_proprio?','si','no')
976         ripeti=1;
977         while ripeti
978             scelta=menu('Che_tipo_di_plot_vuoi?', 'Massima_deformazione', ...
979                 'Animato', 'Cambia_numero_punti_plottati', ...
980                 'Cambia_scale_factor',...
981                 'Chiudi_Finestre','Esci');
982             switch scelta
983                 case 1
984                     imodo = query_integer_d('Quale_modo_vuoi_plottare?',1,length(s.freq)
985                         ,1);
986                     if isempty(imodo)
987                         return
988                     end
989                     modo=s.V(:,imodo);
990                     vel=s.el;
991                     nodi=s.p;
992                     freq=s.freq(imodo);
993                     fig=plotta_modo_max_def(modo,vel,nodi,freq,imodo,fig);
994                 case 2
995                     imodo = query_integer_d('Quale_modo_vuoi_animare?',1,length(s.freq)
996                         ,1);
997                     if isempty(imodo)
998                         return
999                     end
1000                     modo=s.V(:,imodo);
1001                     vel=s.el;
1002                     nodi=s.p;
1003                     freq=s.freq(imodo);
1004                     fig=plotta_modo_animato(modo,vel,nodi,freq,imodo,fig);
1005                 case 3
1006                     get_Nplot(true);
1007                 case 4
1008                     get_sc(true);
1009                 case 5
1010                     close all hidden
1011                 case 6
1012                     ripeti=0;

```

```

1013         end
1014     end
1015
1016     end
1017 end
1018
1019
1020 function [out,fig]= risposta_forzata(st,fig)
1021
1022 switch(menu('Risposta_forzata',{'via_Modello_strutturale','via_Modello_modale'}));
1023     case 1
1024         [out,fig]=risposta_forzata_strutturale(st,fig);
1025
1026     case 2
1027         [out,fig]=risposta_forzata_modale(st,fig);
1028
1029     otherwise
1030         return
1031 end
1032
1033 end
1034
1035
1036 function [risp,fig] = risposta_forzata_strutturale(s,fig)
1037
1038     vel=s.el;
1039     nodi=s.p;
1040     if ~s.nf
1041         risp = [];
1042         error_window('No_Forces!!!','logo.jpg');
1043         return
1044     end
1045
1046     M = s.M;
1047     K = s.K;
1048
1049     if isfield(s,'C') && ~isempty(s.C)
1050         C = s.C;
1051     else
1052         C = zeros(size(M));
1053     end
1054
1055     n = size(M,1);
1056
1057     F0 = s.Forza(:);
1058
1059     if length(F0) ~= n
1060         error('Dimensione_errata: s.Forza deve avere dimensione ndof_x1. ');
1061     end
1062
1063     ft = scegli_forzante_temporale();
1064
1065     T = ft.t(:);
1066
1067     x0 = zeros(n,1);
1068     dx0 = zeros(n,1);
1069     y0 = [x0; dx0];
1070
1071     [odesol,options] = scelta_ode_solver();
1072
1073     [t,y] = odesol(@(t,y) ode_forzata_strutturale(t,y,M,C,K,F0,ft), ...
1074         T,y0,options);
1075
1076     risp.t = t;
1077     risp.y = y;
1078     risp.x = y(:,1:n);
1079     risp.dx = y(:,n+1:2*n);
1080     risp.F0 = F0;
1081     risp.ft = ft;
1082
1083     fig = plotta_risposta_forzata_animata(risp,vel,nodi,fig);
1084
1085 end

```

```

1086
1087 function dydt = ode_forzata_strutturale(t,y,M,C,K,F0,ft)
1088
1089     n = size(M,1);
1090
1091     x = y(1:n);
1092     dx = y(n+1:2*n);
1093
1094     alfa = interp1(ft.t,ft.x,t,'linear',0);
1095
1096     Ft = F0 * alfa;
1097
1098     ddx = M \ (Ft - C*dx - K*x);
1099
1100     dydt = [dx; ddx];
1101
1102 end
1103
1104
1105 function [risp,fig] = risposta_forzata_modale(s,fig)
1106
1107     vel = s.el;
1108     nodi = s.p;
1109
1110     if ~s.nf
1111         risp = [];
1112         error_window('No_Forces!!!','logo.jpg');
1113         return
1114     end
1115
1116     V = s.V;
1117     freq = s.freq(:);
1118     ndof = size(V,1);
1119
1120     idx_flex = find(freq > 0.1);
1121     V_flex = V(:, idx_flex);
1122     omega = 2*pi*freq(idx_flex);
1123     nmodi = length(omega);
1124
1125     mm = diag(V_flex' * s.M * V_flex);
1126     km = diag(V_flex' * s.K * V_flex);
1127
1128     zeta = query_double_d('Smorzamento_modale_zeta', 0, 1, 0.01);
1129     cm = 2*zeta*omega(:) .* mm;
1130
1131     F0 = s.Forza(:);
1132     Fm = V_flex' * F0;
1133
1134     ft = scegli_forzante_temporale();
1135     T = ft.t(:);
1136
1137     q0 = zeros(2*nmodi, 1);
1138
1139     [odesol, options] = scelta_ode_solver();
1140
1141     [t, yq] = odesol(@(t,y) ode_modale(t, y, mm, cm, km, Fm, ft), ...
1142         T, q0, options);
1143
1144     q = yq(:, 1:nmodi);
1145     dq = yq(:, nmodi+1:2*nmodi);
1146
1147     x = q * V_flex';
1148     dx = dq * V_flex';
1149
1150     risp.t = t;
1151     risp.y = [x dx];
1152     risp.x = x;
1153     risp.dx = dx;
1154     risp.F0 = F0;
1155     risp.ft = ft;
1156
1157     fig = plotta_risposta_forzata_animata(risp, vel, nodi, fig);
1158

```

```

1159 end
1160
1161 function dydt = ode_modale(t, y, mm, cm, km, Fm, ft)
1162
1163     n = length(mm);
1164     q = y(1:n);
1165     dq = y(n+1:2*n);
1166
1167     alpha = interp1(ft.t, ft.x, t, 'linear', 0);
1168
1169     ddq = (Fm*alpha - cm.*dq - km.*q) ./ mm;
1170
1171     dydt = [dq; ddq];
1172 end
1173
1174
1175 function sig = scegli_forzante_temporale()
1176
1177     v = {'Random', ...
1178         'Armonica', ...
1179         'Serie_armoniche', ...
1180         'Serie_armoniche_smorzate', ...
1181         'Chirp', ...
1182         'Pseudo_impulso', ...
1183         'Esci'};
1184
1185     scelta = menu('Scegli_forzante_temporale',v);
1186
1187     switch scelta
1188     case 1
1189         sig = genera_segnale_casuale();
1190
1191     case 2
1192         sig = genera_segnale_armonico();
1193
1194     case 3
1195         sig = genera_serie_armoniche();
1196
1197     case 4
1198         sig = genera_serie_armoniche_smorzate();
1199
1200     case 5
1201         sig = genera_segnale_chirp();
1202
1203     case 6
1204         sig = genera_segnale_pseudo_impulso_area();
1205
1206     otherwise
1207         error('Scelta_forzante annullata. ');
1208     end
1209
1210     sig.t = (0:sig.N-1)/sig.fc;
1211
1212 end
1213
1214 function s=genera_segnale_casuale()
1215     stitle= 'Genera_segnale_random';
1216     vs= {'N','fc[Hz]','Amin','Amax'};
1217     ancora= 1;
1218     xmin= [2,1e-6,-1e6,-1.e6];
1219     xmax= [100000000,1e6,1e6,1.e6];
1220     xdef= [1024,1000.,-100,100.];
1221     while ancora
1222         out= query_multiple_exit(stitle,vs,xmin,xmax,xdef);
1223         s.N= out(1); s.fc= out(2); Amin= out(3); Amax= out(4);
1224         if Amax>Amin ancora=0; end
1225     end
1226     tmax= (s.N-1)/s.fc; t= linspace(0,tmax,s.N);
1227     s.x= Amin+(Amax-Amin)*rand(size(t));
1228
1229 end
1230
1231 function s=genera_segnale_armonico()

```

```

1232     stitle= 'Genera_segnale_armonico';
1233     vs= {'N','fc[Hz]','Amplitude','Frequency[Hz]','Phase[deg]'};
1234     xmin= [2,1e-6,1e-6,1.e-6,0.];
1235     xmax= [100000000,1e6,1e10,1.e5,360.];
1236     xdef= [1024,1000.,10.,100.,30.];
1237     out= query_multiple_exit(stitle,vs,xmin,xmax,xdef);
1238     s.N= out(1); s.fc= out(2); A= out(3); w= 2*pi*out(4); ph= pi*out(5)/180;
1239     tmax= (s.N-1)/s.fc; t= linspace(0,tmax,s.N);
1240     s.x= A*sin(w*t+ph);
1241
1242 end
1243
1244 function s=genera_serie_armoniche()
1245     stitle= 'Genera_segnale_serie_armoniche';
1246     vs= {'N','fc[Hz]','Numero_componenti','Amin','Amax',...
1247         'fmin[Hz]','fmax[Hz]'};
1248     ancora= 1;
1249     xmin= [2,1e-6,1,1e-6,1e-6,1e-6,1e-6];
1250     xmax= [100000000,1e6,1000,1e6,1e6,1e5,1e5];
1251     xdef= [1024,1000.,10,1.,10.,1,400.];
1252     while ancora
1253         out= query_multiple_exit(stitle,vs,xmin,xmax,xdef);
1254         s.N= out(1); s.fc= out(2); n= out(3);
1255         Amin= out(4); Amax= out(5);
1256         wmin= 2*pi*out(6); wmax= 2*pi*out(7);
1257         if Amax>Amin & wmax>wmin
1258             ancora=0;
1259         end
1260     end
1261
1262     tmax= (s.N-1)/s.fc; t= linspace(0,tmax,s.N);
1263     vA= Amin+(Amax-Amin)*rand(n,1);
1264     vw= wmin+(wmax-wmin)*rand(n,1);
1265     vph= -pi+2*pi*rand(n,1);
1266     s.x= zeros(size(t));
1267     for i=1:n
1268         s.x= s.x+vA(i)*sin(vw(i)*t+vph(i));
1269     end
1270
1271 end
1272
1273 function s=genera_serie_armoniche_smorzate()
1274     stitle= 'Genera_segnale_serie_armoniche_smorzate';
1275     vs= {'N','fc[Hz]','Numero_componenti','Amin','Amax',...
1276         'fmin[Hz]','fmax[Hz]','zita_min[%]','zita_max[%]'};
1277     ancora= 1;
1278     xmin= [2,1e-6,1,1e-6,1e-6,1e-6,1e-6,1e-3,1e-3];
1279     xmax= [100000000,1e6,1000,1e6,1e6,1e5,1e5,100.,100.];
1280     xdef= [1024,1000.,10,1.,10.,1,400.,1.e-3,1.e-2];
1281     while ancora
1282         out= query_multiple_exit(stitle,vs,xmin,xmax,xdef);
1283         s.N= out(1); s.fc= out(2); n= out(3);
1284         Amin= out(4); Amax= out(5);
1285         wmin= 2*pi*out(6); wmax= 2*pi*out(7);
1286         zmin= out(8); zmax= out(9);
1287         if Amax>Amin & wmax>wmin & zmax>zmin & wmax<pi*s.fc
1288             ancora=0;
1289         end
1290     end
1291
1292     tmax= (s.N-1)/s.fc; t= linspace(0,tmax,s.N);
1293     vA= Amin+(Amax-Amin)*rand(n,1);
1294     vw= wmin+(wmax-wmin)*rand(n,1);
1295     vzita= zmin+(zmax-zmin)*rand(n,1);
1296     vph= -pi+2*pi*rand(n,1);
1297     s.x= zeros(size(t));
1298     for i=1:n
1299         s.x= s.x+vA(i)*exp(-vzita(i)*vw(i)*t).*sin(vw(i)*t+vph(i));
1300     end
1301
1302 end
1303
1304 function s=genera_segnale_chirp()

```

```

1305     stitle= 'Genera_segnale_chirp';
1306     vs= {'N','fc[Hz]','f0[Hz]','f1[Hz]'};
1307     ancora= 1;
1308     xmin= [2,1e-6,0.,1e-6];
1309     xmax= [100000000,1e6,1e6,1e6];
1310     xdef= [1024,1000.,0.,100.];
1311     while ancora
1312         out= query_multiple_exit(stitle,vs,xmin,xmax,xdef);
1313         s.N= out(1); s.fc= out(2); f0= out(3); f1= out(4);
1314         fmax= 0.5*s.fc;
1315         if f1<fmax & f0<f1
1316             ancora=0;
1317         end
1318     end
1319     time= linspace(0,(s.N-1)/s.fc,s.N);
1320     s.x= chirp(time,f0,time(end),f1);
1321     s.X= fft(s.x);
1322     s.tipo='chirp';
1323     s.f0 = f0;
1324     s.f1 = f1;
1325
1326 end
1327
1328 function s = genera_segnale_pseudo_impulso_area()
1329
1330 stitle = 'Genera_segnale_pseudo_impulso';
1331
1332 vs = {'N','fc[Hz]','Impulso_I','t0[s]','dt_impulso[s]'};
1333
1334 xmin = [2, 1e-6, -1e10, 0, 1e-12];
1335 xmax = [100000000, 1e6, 1e10, 1e10, 1e10];
1336 xdef = [1024, 1000., 1., 0.01, 0.001];
1337
1338 ancora = 1;
1339
1340 while ancora
1341
1342     out = query_multiple_exit(stitle,vs,xmin,xmax,xdef);
1343
1344     if isempty(out)
1345         out = xdef;
1346     end
1347
1348     s.N = round(out(1));
1349     s.fc = out(2);
1350     Iimp = out(3);
1351     t0 = out(4);
1352     dt = out(5);
1353
1354     tmax = (s.N-1)/s.fc;
1355
1356     if t0 >= 0 && t0 < tmax && dt > 0 && t0+dt <= tmax
1357         ancora = 0;
1358     else
1359         error_window('Pseudo_impulso_fuori_dal_range_temporale','logo.jpg');
1360     end
1361
1362 end
1363
1364 tmax = (s.N-1)/s.fc;
1365 t = linspace(0,tmax,s.N);
1366
1367 A = Iimp/dt;
1368
1369 s.x = zeros(size(t));
1370 indx = t >= t0 & t <= t0+dt;
1371 s.x(indx) = A;
1372
1373 s.t = t;
1374
1375 end
1376
1377

```

```

1378 function [sol,opt] = scelta_ode_solver()
1379
1380     vsol = {'ode45','ode15s','ode23','ode113','ode23t','ode23tb','ode23s'};
1381
1382     is = menu('Scelta_solutore_ODE',vsol);
1383
1384     if is == 0
1385         is = 1;
1386     end
1387
1388     sol = str2func(vsol{is});
1389
1390     RelTol = 1e-3;
1391     AbsTol = 1e-6;
1392
1393     vstr = {'AbsTol','RelTol'};
1394     xmin = [eps,eps];
1395     xmax = [1e10,1];
1396     xdef = [AbsTol,RelTol];
1397
1398     out = query_multiple_exit('Ode_Options',vstr,xmin,xmax,xdef);
1399
1400     if isempty(out)
1401         out = xdef;
1402     end
1403
1404     AbsTol = out(1);
1405     RelTol = out(2);
1406
1407     opt = odeset('RelTol',RelTol,'AbsTol',AbsTol);
1408
1409 end
1410
1411
1412 function fig = plotta_risposta_forzata_animata(risp,vel,nodi,fig)
1413
1414     sc = get_sc();
1415     L = dimensione_struttura(nodi);
1416     wmax = max_response_disp(risp.x);
1417     if wmax < eps
1418         scale_factor = 1;
1419     else
1420         scale_factor = sc*L/wmax;
1421     end
1422     [xlim0,ylim0,zlim0] = limiti_animazione(nodi,risp,scale_factor);
1423     nd = length(nodi);
1424     N = get_Nplot();
1425     nt = length(risp.t);
1426     nframe = 80;
1427     ncycle = 1;
1428     fig = set_fig(fig,'Risposta_forzata_animata');
1429     set(fig,'ToolBar','figure')
1430     set(fig,'MenuBar','figure')
1431     if nt > nframe
1432         vk = round(linspace(1,nt,nframe));
1433     else
1434         vk = 1:nt;
1435     end
1436     for c = 1:ncycle
1437         for iframe = 1:length(vk)
1438             if ~ishandle(fig)
1439                 return
1440             end
1441             k = vk(iframe);
1442             xk = risp.x(k,:) * scale_factor;
1443             nodi_def = nodi;
1444             for i = 1:nd
1445                 ii = (i-1)*3;
1446                 nodi_def(i).x = nodi(i).x + xk(ii+1);
1447                 nodi_def(i).y = nodi(i).y + xk(ii+2);
1448                 nodi_def(i).z = nodi(i).z + xk(ii+3);
1449             end
1450             cla

```

```

1451         hold on
1452         for i = 1:length(vel)
1453             plotta_el(vel(i), nodi, N, 'k--');
1454         end
1455         for i = 1:length(vel)
1456             plotta_el(vel(i), nodi_def, N, 'r-');
1457         end
1458         grid on
1459         view(3)
1460         xlim(xlim0); ylim(ylim0); zlim(zlim0);
1461         drawnow
1462     end
1463 end
1464 end
1465
1466 function verifica_chirp_frf(risp, s)
1467
1468     if isempty(risp) || isempty(risp.x)
1469         error_window('Nessuna risposta disponibile.', 'logo.jpg');
1470         return
1471     end
1472
1473     if ~isfield(risp, 'ft') || ~isfield(risp.ft, 'tipo') || ~strcmp(risp.ft.tipo, 'chirp'
1474         )
1475         print_window({'La verifica FRF richiede la forzante di tipo chirp.', ...
1476             'Selezionare prima il chirp come forzante temporale.'}, ...
1477             'Tipo forzante non valido');
1478         return
1479     end
1480
1481     ndof = size(risp.x, 2);
1482     n_nodi = ndof / 3;
1483
1484     while 1
1485         switch menu('FRF Chirp', {'Seleziona nodo e plotta FRF', 'Esci'})
1486             case 0
1487                 break
1488             case 1
1489                 fig_nodi = plotta_modello_nodi(s.el, s.p, 0);
1490                 hold on
1491                 plotta_facce_vincolate_e_caricate(s);
1492                 hold off
1493
1494                 print_window( ...
1495                     {'Il modello e stato plottato con la', ...
1496                     'numerazione dei nodi.', '', ...
1497                     'Ruota la figura per identificare', ...
1498                     'il nodo su cui eseguire la FRF', ...
1499                     'poi premi OK per continuare.'}, ...
1500                     'Scegli nodo');
1501
1502                 nodo = query_integer_d('Numero nodo per la FRF', 1, n_nodi, 1);
1503
1504                 if isempty(nodo), continue; end
1505
1506                 dir = menu('Direzione spostamento', ...
1507                     'x(dof_1)', 'y(dof_2)', 'z(dof_3)');
1508                 if dir == 0, continue; end
1509
1510                 frf_plot(risp, s, nodo, dir, fig_nodi);
1511
1512             case 2
1513                 break
1514         end
1515     end
1516 end
1517
1518 function frf_plot(risp, s, nodo, dir, fig)
1519
1520     dirs = {'x', 'y', 'z'};
1521     idof = (nodo - 1)*3 + dir;
1522

```

```

1523 dt = 1 / risp.ft.fc;
1524 N = risp.ft.N;
1525 fc = risp.ft.fc;
1526
1527 X = fft(risp.x(:, idof)) / N;
1528 F = fft(risp.ft.x) / N;
1529 freq = (0:N-1) * fc / N;
1530 H = X ./ F;
1531
1532 f_max_plot = min(risp.ft.f1 * 1.1, fc / 2);
1533 idx = freq > 0 & freq <= f_max_plot;
1534
1535 fn = s.freq;
1536
1537 fig_title = sprintf('FRF_%d_nodo_%d_dir_%s', nodo, dirs{dir});
1538 fig = set_fig(fig, fig_title);
1539
1540 subplot(2, 1, 1)
1541 semilogy(freq(idx), abs(H(idx)), 'b-', 'LineWidth', 1.2)
1542 hold on
1543 for k = 1:length(fn)
1544     if fn(k) <= f_max_plot
1545         xline(fn(k), 'r--', 'LineWidth', 0.8);
1546     end
1547 end
1548 hold off
1549 xlabel('Frequenza [Hz]')
1550 ylabel('|H(f)|_{m/N_{norm.}}')
1551 legend('FRF_{X/F}', 'Frequenze_propriie', 'Location', 'northeast')
1552 grid on
1553
1554 subplot(2, 1, 2)
1555 plot(freq(idx), abs(F(idx)), 'k-', 'LineWidth', 1.2)
1556 xlabel('Frequenza [Hz]')
1557 ylabel('|F(f)|_{N_{norm.}}')
1558 title('Spettro della forzante (chirp)')
1559 grid on
1560
1561 drawnow
1562 end
1563
1564 function confronta_freq_shell(st, freq_fem, nome_file)
1565
1566     if ~contains(lower(nome_file), 'shell')
1567         print_window( ...
1568             {'Il modello che stai esaminando non e' uno shell.', ...
1569             }, ...
1570             sprintf('File caricato: %s', nome_file), ...
1571             {'La verifica analitica (Blevins, Donnell)', ...
1572             'e' applicabile solo a modelli il cui nome', ...
1573             'file contiene "shell".'}, ...
1574             'Verifica non applicabile');
1575         return
1576     end
1577
1578     nd = st.nd;
1579     x = zeros(nd,1);
1580     y = zeros(nd,1);
1581     z = zeros(nd,1);
1582     for i = 1:nd
1583         x(i) = st.p(i).x;
1584         y(i) = st.p(i).y;
1585         z(i) = st.p(i).z;
1586     end
1587
1588     r_all = sqrt(x.^2 + y.^2);
1589     R = (max(r_all) + min(r_all)) / 2;
1590     h = max(r_all) - min(r_all);
1591     L = max(z) - min(z);
1592
1593     ro = st.el(1).ro;

```

```

1596     ni = st.el(1).ni;
1597     E = st.el(1).E;
1598
1599     i_max = 20;
1600     j_max = 5;
1601     [freq_tab, f_min, ij_min] = freq_analitica_shell(R, h, L, ro, ni, E, i_max, j_max);
1602
1603     f_an = [];
1604     for j = 1:j_max
1605         if ~isnan(freq_tab(1,j))
1606             f_an(end+1) = freq_tab(1,j);
1607         end
1608     end
1609     for i = 1:i_max
1610         for j = 1:j_max
1611             if ~isnan(freq_tab(i+1,j))
1612                 f_an(end+1) = freq_tab(i+1,j);
1613                 f_an(end+1) = freq_tab(i+1,j);
1614             end
1615         end
1616     end
1617     f_an = sort(f_an(:));
1618
1619     toll_zero = 1e-2;
1620     freq_fem = freq_fem(freq_fem > toll_zero);
1621     freq_fem = sort(freq_fem(:));
1622
1623     n_confronto = min([length(freq_fem), length(f_an), 20]);
1624
1625     str = {};
1626     str{end+1} = 'Confronto_frequenze: FEM vs Donnell (analitico)';
1627     str{end+1} = '';
1628     str{end+1} = sprintf('File: %s', nome_file);
1629     str{end+1} = sprintf('R = %.4g m, h = %.4g m, L = %.4g m', R, h, L);
1630     str{end+1} = sprintf('h/R = %.4g, L/R = %.4g', h/R, L/R);
1631     str{end+1} = sprintf('ro = %.4g, ni = %.4g, E = %.4g', ro, ni, E);
1632     str{end+1} = sprintf('Fondamentale analitica: %.4f Hz (i=%d, j=%d)', ...
1633         f_min, ij_min(1), ij_min(2));
1634     str{end+1} = '';
1635     str{end+1} = 'Nota: ogni modo analitico con i >= 1 e j >= 1 doppio (cos/sin)';
1636     str{end+1} = '';
1637     str{end+1} = 'Modo FEM [Hz] anal [Hz] err [%]';
1638     str{end+1} = '-----';
1639
1640     err_vec = zeros(n_confronto, 1);
1641     for im = 1:n_confronto
1642         err = (freq_fem(im) - f_an(im)) / f_an(im) * 100;
1643         err_vec(im) = abs(err);
1644         str{end+1} = sprintf('%.4d %.10f %.11f %.2f', ...
1645             im, freq_fem(im), f_an(im), err);
1646     end
1647
1648     str{end+1} = '';
1649     str{end+1} = sprintf('Errore medio: %.2f%%', mean(err_vec));
1650
1651     print_window(str, 'Verifica_analitica_shell');
1652 end
1653
1654
1655 function [freq_tab, f_min, ij_min] = freq_analitica_shell(R, h, L, ro, ni, E, i_max,
    j_max)
1656
1657     if nargin < 7 || isempty(i_max), i_max = 20; end
1658     if nargin < 8 || isempty(j_max), j_max = 5; end
1659
1660     k = h^2 / (12*R^2);
1661     c_freq = 1/(2*pi*R) * sqrt(E / (ro*(1-ni^2)));
1662
1663     freq_tab = zeros(i_max+1, j_max);
1664     f_min = Inf;
1665     ij_min = [0, 1];
1666
1667     for j = 1:j_max

```

```

1668     xi = j*pi*R / L;
1669     for i = 0:i_max
1670         s = i^2 + xi^2;
1671
1672         K2 = 1 + 0.5*(3 - ni)*s + k*s^2;
1673         K1 = 0.5*(1 - ni) * ( (3 + 2*ni)*xi^2 + i^2 + s^2 ...
1674             + (3 - ni)/(1 - ni) * k * s^3 );
1675         K0 = 0.5*(1 - ni) * ( (1 - ni^2)*xi^4 + k*s^4 );
1676
1677         p = [1, -K2, K1, -K0];
1678         rad = roots(p);
1679         rad = real(rad(abs(imag(rad)) < 1e-12*max(abs(rad)+1)));
1680         rad = rad(rad > 0);
1681
1682         if isempty(rad)
1683             freq_tab(i+1, j) = NaN;
1684             continue
1685         end
1686
1687         lambda_min = sqrt(min(rad));
1688         f_ij = lambda_min * c_freq;
1689
1690         freq_tab(i+1, j) = f_ij;
1691
1692         if f_ij < f_min
1693             f_min = f_ij;
1694             ij_min = [i, j];
1695         end
1696     end
1697 end
1698
1699 str = {};
1700 str{end+1} = 'Frequenze analitiche - Donnell shell theory';
1701 str{end+1} = '';
1702 str{end+1} = sprintf('R=%g m, h=%g m, L=%g m', R, h, L);
1703 str{end+1} = sprintf('h/R=%g, L/R=%g, k=%g e', h/R, L/R, k);
1704 str{end+1} = sprintf('rho=%g kg/m3, ni=%g g, E=%g Pa', ro, ni, E);
1705 str{end+1} = '';
1706
1707 riga = 'ij';
1708 for j = 1:j_max
1709     riga = [riga, sprintf('j=%-5d', j)];
1710 end
1711 str{end+1} = riga;
1712
1713 sep = '----';
1714 for j = 1:j_max
1715     sep = [sep, '-----'];
1716 end
1717 str{end+1} = sep;
1718
1719 for i = 0:i_max
1720     riga = sprintf('i=%2d', i);
1721     for j = 1:j_max
1722         f = freq_tab(i+1, j);
1723         if isnan(f)
1724             riga = [riga, sprintf('s', '---')];
1725         else
1726             riga = [riga, sprintf('f', f)];
1727         end
1728     end
1729     str{end+1} = riga;
1730 end
1731
1732 str{end+1} = '';
1733 str{end+1} = sprintf('Frequenza fondamentale: f_min=%g Hz (i=%d, j=%d)', ...
1734     f_min, ij_min(1), ij_min(2));
1735
1736 print_window(str, 'Frequenze analitiche shell');
1737 end
1738
1739
1740 function st = rotate_model(st, ng, thx, thy, thz)

```

```

1741
1742     if nargin < 3
1743         thx = 0;
1744         thy = 0;
1745         thz = 0;
1746     end
1747
1748     stitle = 'Ruota_Modello';
1749
1750     theta = char(952);
1751     vs = { [theta '_x'], [theta '_y'], [theta '_z'] };
1752
1753     xmin = [0., 0., 0.];
1754     xmax = [360, 360, 360];
1755     xdef = [thx, thy, thz];
1756
1757     out = query_multiple_exit(stitle, vs, xmin, xmax, xdef);
1758
1759     thx = deg2rad(out(1));
1760     thy = deg2rad(out(2));
1761     thz = deg2rad(out(3));
1762
1763     Rx = Rx_new(thx);
1764     Ry = Ry_new(thy);
1765     Rz = Rz_new(thz);
1766
1767     R = Rx * Ry * Rz;
1768
1769     nodes = zeros(st.nd,3);
1770
1771     for i = 1:st.nd
1772         nodes(i,1) = st.p(i).x;
1773         nodes(i,2) = st.p(i).y;
1774         nodes(i,3) = st.p(i).z;
1775     end
1776
1777     nodes_rot = (R * nodes)';
1778
1779     for i = 1:st.nd
1780         st.p(i).x = nodes_rot(i,1);
1781         st.p(i).y = nodes_rot(i,2);
1782         st.p(i).z = nodes_rot(i,3);
1783     end
1784     st=crea_modello_fem(st,ng);
1785     st=calcola_modello_modale(st);
1786 end
1787
1788
1789 function fig = plotta_modello(vel,nodi,fig,stile)
1790
1791     N = get_Nplot();
1792
1793     fig = set_fig(fig,'Plot_mesh');
1794
1795     hold on
1796     grid on
1797     axis equal
1798     view(3)
1799
1800     set(fig,'ToolBar','figure')
1801     set(fig,'MenuBar','figure')
1802
1803     xlabel('x')
1804     ylabel('y')
1805     zlabel('z')
1806
1807     for i = 1:length(vel)
1808         plotta_el(vel(i),nodi,N,stile);
1809     end
1810
1811     axis equal
1812     view(3)
1813

```

```

1814     rotate3d(fig,'on')
1815     zoom(fig,'off')
1816     pan(fig,'off')
1817
1818     drawnow
1819
1820     hold off
1821
1822 end
1823
1824
1825 function fig = plotta_modello_nodi(vel,nodi,fig)
1826
1827     N = get_Nplot();
1828     fig = set_fig(fig,'Plot_mesh_con_indx_nodi');
1829
1830     hold on
1831     grid on
1832     axis equal
1833     view(3)
1834
1835     set(fig,'ToolBar','figure')
1836     set(fig,'MenuBar','figure')
1837
1838     xlabel('x')
1839     ylabel('y')
1840     zlabel('z')
1841
1842     for i = 1:length(vel)
1843         plotta_el(vel(i),nodi,N,'k-');
1844     end
1845
1846     for i = 1:length(nodi)
1847
1848         plot3(nodi(i).x,nodi(i).y,nodi(i).z,'ko', ...
1849             'MarkerFaceColor','k', ...
1850             'MarkerSize',5);
1851
1852         text(nodi(i).x,nodi(i).y,nodi(i).z, ...
1853             ['_ _' num2str(i)], ...
1854             'FontSize',8, ...
1855             'Color','r');
1856
1857     end
1858
1859     axis equal
1860     view(3)
1861
1862     rotate3d(fig,'on')
1863     zoom(fig,'off')
1864     pan(fig,'off')
1865
1866     drawnow
1867     hold off
1868
1869 end
1870
1871
1872 function fig = plotta_modello_nodi_elementi(vel,nodi,fig)
1873
1874     N = get_Nplot();
1875
1876     fig = set_fig(fig,'Plot_mesh_con_indx_nodi_e_elementi');
1877
1878     hold on
1879     grid on
1880     axis equal
1881     view(3)
1882
1883     set(fig,'ToolBar','figure')
1884     set(fig,'MenuBar','figure')
1885
1886     xlabel('x')

```

```

1887 ylabel('y')
1888 zlabel('z')
1889
1890 for i = 1:length(vel)
1891
1892     plotta_el(vel(i),nodi,N,'k-');
1893
1894     pto = par2pto(0,0,0,vel(i),nodi);
1895
1896     text(pto.x,pto.y,pto.z, ...
1897          num2str(i), ...
1898          'FontSize',10, ...
1899          'Color','b', ...
1900          'FontWeight','bold', ...
1901          'HorizontalAlignment','center');
1902
1903 end
1904
1905 for i = 1:length(nodi)
1906
1907     plot3(nodi(i).x,nodi(i).y,nodi(i).z,'ko', ...
1908           'MarkerFaceColor','k', ...
1909           'MarkerSize',5);
1910
1911     text(nodi(i).x,nodi(i).y,nodi(i).z, ...
1912          ['□□' num2str(i)], ...
1913          'FontSize',8, ...
1914          'Color','r');
1915
1916 end
1917
1918 axis equal
1919 view(3)
1920
1921 rotate3d(fig,'on')
1922 zoom(fig,'off')
1923 pan(fig,'off')
1924
1925 drawnow
1926 hold off
1927
1928 end
1929
1930
1931 function plotta_modello_corrente(vel,nodi,N,stile)
1932
1933     hold on
1934     grid on
1935
1936     view(3)
1937     xlabel('x')
1938     ylabel('y')
1939     zlabel('z')
1940     for i = 1:length(vel)
1941         plotta_el(vel(i),nodi,N,stile);
1942     end
1943
1944     view(3)
1945     rotate3d(gcf,'on')
1946     zoom(gcf,'off')
1947     pan(gcf,'off')
1948     drawnow
1949 end
1950
1951
1952 function plotta_el(el,p,N,stile)
1953
1954     par = linspace(-1,1,N);
1955
1956     eta = -1; zita = -1;
1957     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
1958     for i = 1:N
1959         csi = par(i);

```

```

1960         pto = par2pto(csi,eta,zita,el,p);
1961         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
1962     end
1963     plot3(X,Y,Z,stile);
1964
1965     eta = 1; zita = -1;
1966     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
1967     for i = 1:N
1968         csi = par(i);
1969         pto = par2pto(csi,eta,zita,el,p);
1970         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
1971     end
1972     plot3(X,Y,Z,stile);
1973
1974     eta = -1; zita = 1;
1975     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
1976     for i = 1:N
1977         csi = par(i);
1978         pto = par2pto(csi,eta,zita,el,p);
1979         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
1980     end
1981     plot3(X,Y,Z,stile);
1982
1983     eta = 1; zita = 1;
1984     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
1985     for i = 1:N
1986         csi = par(i);
1987         pto = par2pto(csi,eta,zita,el,p);
1988         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
1989     end
1990     plot3(X,Y,Z,stile);
1991
1992     csi = -1; zita = -1;
1993     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
1994     for i = 1:N
1995         eta = par(i);
1996         pto = par2pto(csi,eta,zita,el,p);
1997         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
1998     end
1999     plot3(X,Y,Z,stile);
2000
2001     csi = 1; zita = -1;
2002     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
2003     for i = 1:N
2004         eta = par(i);
2005         pto = par2pto(csi,eta,zita,el,p);
2006         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
2007     end
2008     plot3(X,Y,Z,stile);
2009
2010     csi = -1; zita = 1;
2011     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
2012     for i = 1:N
2013         eta = par(i);
2014         pto = par2pto(csi,eta,zita,el,p);
2015         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
2016     end
2017     plot3(X,Y,Z,stile);
2018
2019     csi = 1; zita = 1;
2020     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
2021     for i = 1:N
2022         eta = par(i);
2023         pto = par2pto(csi,eta,zita,el,p);
2024         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
2025     end
2026     plot3(X,Y,Z,stile);
2027
2028     csi = -1; eta = -1;
2029     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
2030     for i = 1:N
2031         zita = par(i);
2032         pto = par2pto(csi,eta,zita,el,p);

```

```

2033         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
2034     end
2035     plot3(X,Y,Z,stile);
2036
2037     csi = 1; eta = -1;
2038     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
2039     for i = 1:N
2040         zita = par(i);
2041         pto = par2pto(csi,eta,zita,el,p);
2042         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
2043     end
2044     plot3(X,Y,Z,stile);
2045
2046     csi = 1; eta = 1;
2047     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
2048     for i = 1:N
2049         zita = par(i);
2050         pto = par2pto(csi,eta,zita,el,p);
2051         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
2052     end
2053     plot3(X,Y,Z,stile);
2054
2055     csi = -1; eta = 1;
2056     X = zeros(N,1); Y = zeros(N,1); Z = zeros(N,1);
2057     for i = 1:N
2058         zita = par(i);
2059         pto = par2pto(csi,eta,zita,el,p);
2060         X(i) = pto.x; Y(i) = pto.y; Z(i) = pto.z;
2061     end
2062     plot3(X,Y,Z,stile);
2063
2064 end
2065
2066
2067 function fig=plotta_modo_max_def(modo,vel,nodi,f,m,fig)
2068
2069 sc = get_sc();
2070 L= dimensione_struttura(nodi); scale_factor= sc*L;
2071 wmax=max_modo_disp(modo);
2072 modo= modo*scale_factor/wmax;
2073 nd= length(nodi);
2074 N = get_Nplot();
2075 fig = set_fig(fig, sprintf('Modo_%d-f=%f[Hz]',m,f));
2076 hold on
2077 set(fig,'ToolBar','figure')
2078 set(fig,'MenuBar','figure')
2079 plotta_modello_corrente(vel,nodi,N,'k--');
2080
2081 for i=1:nd
2082     ii=(i-1)*3;
2083     nodi(i).x= nodi(i).x+modo(ii+1);
2084     nodi(i).y= nodi(i).y+modo(ii+2);
2085     nodi(i).z= nodi(i).z+modo(ii+3);
2086 end
2087 plotta_modello_corrente(vel,nodi,N,'r-');
2088 end
2089
2090 function w=max_modo_disp(v)
2091 nv= length(v); n=nv/3; w=0;
2092 for i=1:n
2093     ii= (i-1)*3;
2094     Li=norm(v(ii+1:ii+3));
2095     if Li>w w=Li; end
2096 end
2097 end
2098
2099 function L=dimensione_struttura_local(nodi)
2100 nd=length(nodi);
2101 x=zeros(nd,1);
2102 y=zeros(nd,1);
2103 z=zeros(nd,1);
2104
2105 for i=1:nd

```

```

2106         x(i)=nodi(i).x;
2107         y(i)=nodi(i).y;
2108         z(i)=nodi(i).z;
2109     end
2110     dx=max(x)-min(x);
2111     dy=max(y)-min(y);
2112     dz=max(z)-min(z);
2113     L=sqrt(dx^2+dy^2+dz^2);
2114 end
2115
2116
2117 function fig=plotta_modo_animato(modo,vel,nodi,f,m,fig)
2118
2119 sc = get_sc();
2120 L= dimensione_struttura(nodi); scale_factor= sc*L;
2121 wmax=max_mode_disp(modo);
2122 modo= modo*scale_factor/wmax;
2123 nd= length(nodi);
2124 N = get_Nplot();
2125 nframe=40;
2126 fig = set_fig(fig, sprintf('Modo_%d-f=%f[Hz]',m,f));
2127 set(fig,'ToolBar','figure')
2128 set(fig,'MenuBar','figure')
2129 xn = [nodi.x]; yn = [nodi.y]; zn = [nodi.z];
2130 max_def = max_mode_disp(modo) ;
2131 margine = 0.1 * L;
2132 xlim0 = [min(xn)-max_def-margine, max(xn)+max_def+margine];
2133 ylim0 = [min(yn)-max_def-margine, max(yn)+max_def+margine];
2134 zlim0 = [min(zn)-max_def-margine, max(zn)+max_def+margine];
2135 ncycle=5;
2136 ncycle = 5;
2137 for c = 1:ncycle
2138     for iframe = 1:nframe
2139         if ~ishandle(fig)
2140             return
2141         end
2142         theta = 2*pi*(iframe-1)/nframe;
2143         alpha = sin(theta);
2144         nodi_def = nodi;
2145         for i = 1:nd
2146             ii = (i-1)*3;
2147             nodi_def(i).x = nodi(i).x + alpha*modo(ii+1);
2148             nodi_def(i).y = nodi(i).y + alpha*modo(ii+2);
2149             nodi_def(i).z = nodi(i).z + alpha*modo(ii+3);
2150         end
2151         cla
2152         hold on
2153         for i = 1:length(vel)
2154             plotta_el(vel(i), nodi, N, 'k--');
2155         end
2156         for i = 1:length(vel)
2157             plotta_el(vel(i), nodi_def, N, 'r-');
2158         end
2159         grid on
2160         view(3)
2161         xlim(xlim0); ylim(ylim0); zlim(zlim0);
2162         drawnow
2163     end
2164 end
2165
2166 end
2167
2168
2169 function plotta_facce_vincolate_e_caricate(st)
2170
2171     nv = 0; nf = 0;
2172     if isfield(st,'nv'), nv = st.nv; end
2173     if isfield(st,'nf'), nf = st.nf; end
2174     if nv == 0 && nf == 0, return; end
2175
2176     N = get_Nplot();
2177     N_arr = 5;
2178     par = linspace(-1, 1, N);

```

```

2179 par_a = linspace(-0.8, 0.8, N_arr);
2180 toll = 1e-12;
2181 hold on
2182
2183 for iv = 1:nv
2184     v = st.vinc(iv);
2185     el = st.el(v.el);
2186     xp = []; yp = []; zp = [];
2187
2188     [xp, yp, zp] = campiona_faccia(v.u, v.v, v.w, el, st.p, par, toll);
2189     if isempty(xp), continue; end
2190
2191     patch(xp, yp, zp, [0 0.8 0], ...
2192           'FaceAlpha', 0.25, ...
2193           'EdgeColor', [0 0.6 0], ...
2194           'LineStyle', '--', ...
2195           'LineWidth', 1.5);
2196 end
2197
2198 if nf > 0
2199     L_str = dimensione_struttura(st.p);
2200
2201     for q = 1:nf
2202         v = st.F(q);
2203         el = st.el(v.el);
2204         px = v.p(1); py = v.p(2); pz = v.p(3);
2205         p_norm = sqrt(px^2 + py^2 + pz^2);
2206         if p_norm < 1e-30, continue; end
2207
2208         ux = px/p_norm; uy = py/p_norm; uz = pz/p_norm;
2209         scale = 0.15 * L_str;
2210
2211         u_fisso = abs(v.u(2) - v.u(1)) < toll;
2212         v_fisso = abs(v.v(2) - v.v(1)) < toll;
2213         w_fisso = abs(v.w(2) - v.w(1)) < toll;
2214
2215         Xo = []; Yo = []; Zo = [];
2216         for ia = 1:N_arr
2217             for ib = 1:N_arr
2218                 a = par_a(ia); b = par_a(ib);
2219                 if u_fisso
2220                     pto = par2pto(v.u(1), a, b, el, st.p);
2221                 elseif v_fisso
2222                     pto = par2pto(a, v.v(1), b, el, st.p);
2223                 elseif w_fisso
2224                     pto = par2pto(a, b, v.w(1), el, st.p);
2225                 else
2226                     continue
2227                 end
2228                 Xo(end+1) = pto.x;
2229                 Yo(end+1) = pto.y;
2230                 Zo(end+1) = pto.z;
2231             end
2232         end
2233
2234         if isempty(Xo), continue; end
2235
2236         quiver3(Xo, Yo, Zo, ...
2237                ux*scale*ones(size(Xo)), ...
2238                uy*scale*ones(size(Yo)), ...
2239                uz*scale*ones(size(Zo)), ...
2240                0, ...
2241                'Color', [0.85 0 0], ...
2242                'LineWidth', 1.5, ...
2243                'MaxHeadSize', 0.5);
2244     end
2245 end
2246
2247 hold off
2248 end
2249
2250 function [xp, yp, zp] = campiona_faccia(u_range, v_range, w_range, el, p, par, toll)
2251     xp = []; yp = []; zp = [];

```

```

2252     N = length(par);
2253
2254     u_fisso = abs(u_range(2) - u_range(1)) < toll;
2255     v_fisso = abs(v_range(2) - v_range(1)) < toll;
2256     w_fisso = abs(w_range(2) - w_range(1)) < toll;
2257
2258     if u_fisso
2259         u0 = u_range(1);
2260         for i = 1:N, pto=par2pto(u0,par(i),-1,el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2261             zp(end+1)=pto.z; end
2262         for i = 1:N, pto=par2pto(u0,1,par(i),el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2263             zp(end+1)=pto.z; end
2264         for i = N:-1:1, pto=par2pto(u0,par(i),1,el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2265             zp(end+1)=pto.z; end
2266         for i = N:-1:1, pto=par2pto(u0,-1,par(i),el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2267             zp(end+1)=pto.z; end
2268
2269     elseif v_fisso
2270         v0 = v_range(1);
2271         for i = 1:N, pto=par2pto(par(i),v0,-1,el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2272             zp(end+1)=pto.z; end
2273         for i = 1:N, pto=par2pto(1,v0,par(i),el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2274             zp(end+1)=pto.z; end
2275         for i = N:-1:1, pto=par2pto(par(i),v0,1,el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2276             zp(end+1)=pto.z; end
2277         for i = N:-1:1, pto=par2pto(-1,v0,par(i),el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2278             zp(end+1)=pto.z; end
2279
2280     elseif w_fisso
2281         w0 = w_range(1);
2282         for i = 1:N, pto=par2pto(par(i),-1,w0,el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2283             zp(end+1)=pto.z; end
2284         for i = 1:N, pto=par2pto(1,par(i),w0,el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2285             zp(end+1)=pto.z; end
2286         for i = N:-1:1, pto=par2pto(par(i),1,w0,el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2287             zp(end+1)=pto.z; end
2288         for i = N:-1:1, pto=par2pto(-1,par(i),w0,el,p); xp(end+1)=pto.x; yp(end+1)=pto.y;
2289             zp(end+1)=pto.z; end
2290
2291     end
2292 end
2293
2294 function L = dimensione_struttura(nodi)
2295     nd = length(nodi);
2296     x = [nodi.x]; y = [nodi.y]; z = [nodi.z];
2297     L = sqrt((max(x)-min(x))^2 + (max(y)-min(y))^2 + (max(z)-min(z))^2);
2298 end
2299
2300 function pto = par2pto(csi,eta,zita,el,p)
2301
2302     vN = vN_Hexa20(csi,eta,zita);
2303
2304     pto.x = 0;
2305     pto.y = 0;
2306     pto.z = 0;
2307
2308     for i = 1:20
2309
2310         ig = el.indx(i);
2311
2312         pto.x = pto.x + vN(i)*p(ig).x;
2313         pto.y = pto.y + vN(i)*p(ig).y;
2314         pto.z = pto.z + vN(i)*p(ig).z;
2315
2316     end
2317 end
2318
2319 function vN = vN_Hexa20(xi,eta,zita)
2320
2321     vN = zeros(20,1);
2322
2323     vN(1) = 1/8*(1-xi)*(1-eta)*(1-zita)*(-2-xi-eta-zita);

```

```

2313     vN(3) = 1/8*(1+xi)*(1-eta)*(1-zita)*(-2+xi-eta-zita);
2314
2315     vN(5) = 1/8*(1+xi)*(1+eta)*(1-zita)*(-2+xi+eta-zita);
2316
2317     vN(7) = 1/8*(1-xi)*(1+eta)*(1-zita)*(-2-xi+eta-zita);
2318
2319     vN(13) = 1/8*(1-xi)*(1-eta)*(1+zita)*(-2-xi-eta+zita);
2320
2321     vN(15) = 1/8*(1+xi)*(1-eta)*(1+zita)*(-2+xi-eta+zita);
2322
2323     vN(17) = 1/8*(1+xi)*(1+eta)*(1+zita)*(-2+xi+eta+zita);
2324
2325     vN(19) = 1/8*(1-xi)*(1+eta)*(1+zita)*(-2-xi+eta+zita);
2326
2327     vN(2) = 1/4*(1-xi^2)*(1-eta)*(1-zita);
2328
2329     vN(6) = 1/4*(1-xi^2)*(1+eta)*(1-zita);
2330
2331     vN(14) = 1/4*(1-xi^2)*(1-eta)*(1+zita);
2332
2333     vN(18) = 1/4*(1-xi^2)*(1+eta)*(1+zita);
2334
2335     vN(4) = 1/4*(1+xi)*(1-eta^2)*(1-zita);
2336
2337     vN(8) = 1/4*(1-xi)*(1-eta^2)*(1-zita);
2338
2339     vN(16) = 1/4*(1+xi)*(1-eta^2)*(1+zita);
2340
2341     vN(20) = 1/4*(1-xi)*(1-eta^2)*(1+zita);
2342
2343     vN(9) = 1/4*(1-xi)*(1-eta)*(1-zita^2);
2344
2345     vN(10) = 1/4*(1+xi)*(1-eta)*(1-zita^2);
2346
2347     vN(11) = 1/4*(1+xi)*(1+eta)*(1-zita^2);
2348
2349     vN(12) = 1/4*(1-xi)*(1+eta)*(1-zita^2);
2350
2351 end
2352
2353
2354 function Nplot = get_Nplot(reset)
2355
2356 persistent Nplot_saved
2357
2358 if nargin < 1
2359     reset = false;
2360 end
2361
2362 if reset
2363     Nplot_saved = [];
2364 end
2365
2366 if isempty(Nplot_saved)
2367
2368     vs = 'Quanti_punti_per_spigolo_vuoi_plottare?';
2369     xmin = 2;
2370     xmax = 10000;
2371     xdef = 100;
2372
2373     Nplot_saved = query_integer_d(vs, xmin, xmax, xdef);
2374     if isempty(Nplot_saved), return; end
2375
2376 end
2377
2378 Nplot = Nplot_saved;
2379
2380 end
2381
2382
2383 function sc = get_sc(reset)
2384
2385

```

```

2386 persistent sc_saved
2387
2388 if nargin < 1
2389     reset = false;
2390 end
2391
2392 if reset
2393     sc_saved = [];
2394 end
2395
2396 if isempty(sc_saved)
2397
2398     vs = 'Inserisci scale_factor per il plot';
2399     xmin = 0;
2400     xmax = 100;
2401     xdef = 0.1;
2402
2403     sc_saved = query_double_d(vs, xmin, xmax, xdef);
2404     if isempty(sc_saved), return; end
2405
2406 end
2407
2408 sc = sc_saved;
2409
2410 end
2411
2412
2413 function [xlim0,ylim0,zlim0] = limiti_animazione(nodi,risp,scale_factor)
2414
2415 nd = length(nodi);
2416
2417 x0 = zeros(nd,1);
2418 y0 = zeros(nd,1);
2419 z0 = zeros(nd,1);
2420
2421 for i = 1:nd
2422     x0(i) = nodi(i).x;
2423     y0(i) = nodi(i).y;
2424     z0(i) = nodi(i).z;
2425 end
2426
2427 max_def = max_response_disp(risp.x) * scale_factor;
2428
2429 xmin = min(x0) - max_def;
2430 xmax = max(x0) + max_def;
2431
2432 ymin = min(y0) - max_def;
2433 ymax = max(y0) + max_def;
2434
2435 zmin = min(z0) - max_def;
2436 zmax = max(z0) + max_def;
2437
2438 Lx = xmax - xmin;
2439 Ly = ymax - ymin;
2440 Lz = zmax - zmin;
2441
2442 L = max([Lx,Ly,Lz]);
2443
2444 if L < eps
2445     L = 1;
2446 end
2447
2448 margine = 0.1*L;
2449
2450 xlim0 = [xmin-margine xmax+margine];
2451 ylim0 = [ymin-margine ymax+margine];
2452 zlim0 = [zmin-margine zmax+margine];
2453
2454 end
2455
2456
2457 function wmax = max_response_disp(X)
2458

```

```

2459 [nt,ndof] = size(X);
2460
2461 nd = ndof/3;
2462
2463 wmax = 0;
2464
2465 for k = 1:nt
2466
2467     for i = 1:nd
2468
2469         ii = (i-1)*3;
2470
2471         ux = X(k,ii+1);
2472         uy = X(k,ii+2);
2473         uz = X(k,ii+3);
2474
2475         Li = norm([ux uy uz]);
2476
2477         if Li > wmax
2478             wmax = Li;
2479         end
2480
2481     end
2482
2483 end
2484
2485 end
2486
2487
2488 function w=max_mode_disp_standalone(v)
2489 nv= length(v); n=nv/3; w=0;
2490 for i=1:n
2491     ii= (i-1)*3;
2492     Li=norm(v(ii+1:ii+3));
2493     if Li>w w=Li; end
2494 end
2495 end
2496
2497
2498 function L=dimensione_struttura_standalone(nodi)
2499 nd=length(nodi);
2500 x=zeros(nd,1);
2501 y=zeros(nd,1);
2502 z=zeros(nd,1);
2503
2504 for i=1:nd
2505     x(i)=nodi(i).x;
2506     y(i)=nodi(i).y;
2507     z(i)=nodi(i).z;
2508 end
2509 dx=max(x)-min(x);
2510 dy=max(y)-min(y);
2511 dz=max(z)-min(z);
2512 L=sqrt(dx^2+dy^2+dz^2);
2513 end
2514
2515
2516 function R = Rx_new(th)
2517
2518 R = [1 0 0 ;
2519     0 cos(th) -sin(th);
2520     0 sin(th) cos(th)];
2521 end
2522
2523
2524 function R = Ry_new(th)
2525
2526 R = [cos(th) 0 sin(th) ;
2527     0 1 0;
2528     -sin(th) 0 cos(th)];
2529 end
2530
2531

```

```

2532 function R = Rz_new(th)
2533
2534 R = [cos(th) -sin(th) 0;
2535      sin(th) cos(th) 0;
2536      0 0 1];
2537 end
2538
2539
2540 function [file,name]= query_file(dir,mask,permission,title)
2541
2542     if nargin<4
2543         title= 'Get_File';
2544     end
2545     if nargin <3
2546         permission= 'r';
2547     end
2548     if nargin<2
2549         mask= '.*';
2550     end
2551     if nargin<1
2552         dir= '.\';
2553     end
2554
2555     switch permission
2556     case 'r'
2557         [fname,fpath]= uigetfile(sprintf('%s\\%s',dir,mask),title);
2558     case 'w'
2559         [fname,fpath]= uiputfile(sprintf('%s\\%s',dir,mask),title);
2560     otherwise
2561         error('Error in query_file()');
2562     end
2563     if ischar(fpath) && ischar(fname)
2564         file=sprintf('%s%s',fpath, fname);
2565         indx= strfind(fname,mask);
2566         if ~isempty(indx)
2567             name= fname(1:indx-1);
2568         else
2569             name= fname;
2570         end
2571     else
2572         file= []; name=[];
2573     end
2574 end
2575
2576
2577 function x= query_integer_d(s,xmin,xmax,xdef)
2578
2579 ss= sprintf('%s_[%d,%d]:',s,xmin,xmax);
2580
2581 while(1)
2582     answer= char(inputdlg(ss,'Query',1,cellstr(sprintf('%d',xdef))));
2583     if(length(answer)==0)
2584         x = []; return;
2585     end
2586     x= sscanf(answer,'%d');
2587     if x<=xmax & x>=xmin
2588         break
2589     end
2590 end
2591 return
2592
2593
2594 function x = query_double_d(s,xmin,xmax,xdef)
2595
2596 ss = sprintf('%s_[%g,%g]:',s,xmin,xmax);
2597
2598 while 1
2599
2600     answer = inputdlg(ss,'Query',1,cellstr(sprintf('%g',xdef)));
2601
2602     if isempty(answer)
2603         x = []; return;
2604     end

```

```

2605
2606     answer = char(answer);
2607
2608     x = sscanf(answer, '%f');
2609
2610     if ~isempty(x) && x <= xmax && x >= xmin
2611         break
2612     end
2613
2614 end
2615
2616 end
2617
2618
2619 function x= query_multiple_exit(stitle,s,xmin,xmax,xdef)
2620
2621     lineNo= 1;
2622     n= length(xmin);
2623     for i=1:n
2624         if isfinite(xdef(i))
2625             def(i)= cellstr(sprintf('%f',xdef(i)));
2626         else
2627             def(i)= cellstr(sprintf(''));
2628         end
2629         ss(i)= cellstr(sprintf('%s_[%f,%f]:_',char(s(i)),xmin(i),xmax(i)));
2630     end
2631
2632     again= 1;
2633     while again
2634         again= 0;
2635         answer= char(inputdlg(ss,stitle,lineNo,def));
2636         if(isempty(answer))
2637             x=[]; break
2638         end
2639         x= str2num(answer);
2640         for i=1:n
2641             if x(i)>xmax(i) | x(i)<xmin(i)
2642                 again= 1;
2643                 break
2644             end
2645         end
2646     end
2647 end
2648
2649
2650 function print_window(str,stit)
2651     msgbox(str,stit);
2652 return
2653
2654
2655 function error_window(msg,icon_file)
2656
2657     if nargin<2 | ~exist(icon_file)
2658         msgbox(msg,'Error','error');
2659     else
2660         msgbox(msg,'Error','custom',imread(icon_file));
2661     end
2662 end
2663
2664
2665 function fig= set_fig(fig,title)
2666
2667     fig= fig+1; figure(fig);
2668     set(gcf,'Name',title,'NumberTitle','off');
2669 end
2670
2671
2672 function x= ask_yes(s,yes,no)
2673     ss= sprintf('%s_[%s,%s]:_',s,yes,no);
2674     answer=questdlg(s,'Query',yes,no,yes);
2675     if strcmp(answer,no)
2676         x=0;
2677     else

```

```
2678         x=1;
2679     end
2680 end
```

Listing 1: Codice MATLAB completo sviluppato per il programma FEM